SAPIENS

Sapiens Decision Best Practices

March 2024

Contact: info@sapiens.com

Proprietary Material Notice

This document contains certain content or material, including text, graphics, images, and logos, that are either exclusively owned by Sapiens International Corporation N.V and its affiliates ("Sapiens") or are subject to rights of use granted to Sapiens.

All materials are protected by national and/or international copyright laws in respect of each of these entities and may be used by the recipient solely for its own internal review. Any other use, including the reproduction, incorporation, modification, distribution, transmission, republication, creation of a derivative work or display of this document and/or the content or material contained herein, is strictly prohibited without the express prior written authorization of Sapiens.

Contents

1 Introduction

1.1 About This Document

Sapiens is committed to enabling our clients to derive the greatest value from their Sapiens Decision investment. This commitment includes sharing best practices with, and amongst, our client community on an ongoing basis, as Sapiens Decision technology and capabilities evolve. The best practices contained within this document have been compiled by Sapiens, with input and submissions from our clients.

In the spirit of "Partnering for Success", we see the ongoing development of this guide as a collaborative endeavor between Sapiens and our clients. We welcome best practice submissions at any time. Please send your submissions to gil.segal@sapiens.com.

1.2 How to Use This Document

The best practices contained in this document may not apply to all situations. One of the goals of this document is to help you make an informed decision about when to follow the best practice and when to go down a different path.

The document is structured by major topics areas:

- Workspace
- Glossary
- Modeling
- Testing
- Governance
- Release Management
- Decision Selection
- Requirements Traceability
- Communities
- Authorization
- Decision Execution (DE)
- Java Format Adapter
- Integration
- User-Defined Functions
- Working with Multiple DM Repositories
- Minimized Input Optimization (MIO) API
- Parameter Management (PaM)
- ModelAI

If you are new to Sapiens Decision, read through the entire document once to benefit from all the best practices. Then use this document as a reference as you work in Sapiens Decision.

If you are already familiar with Sapiens Decision, then use the Table of Contents to navigate directly to the section of interest or use a keyword search to find best practices around a specific topic.

2 Workspace

For the best user experience when working in Decision Manager (DM), Sapiens recommends using the following settings:

- Screen resolution: at least 1920 x 1080 (full HD)
- Size of text, apps, and other items: 100%
- Browser zoom: 100%

Using these settings enables full functionality and provides sufficient screen space to view larger models.

3 Glossary

3.1 Getting Started

One of the best ways to enable decision modeling to get started quickly is to seed the Glossary by importing it prior to the start of modeling. With little effort, it is often possible to import the Fact Type definitions corresponding to the persistent data that will be used; however, this Glossary must already be in business-friendly terms to enable decision modeling using those business terms.

Glossary import is also a handy technique to import many Allowed Values for one or more existing Fact Types.

3.2 Fact Type Naming

The naming of Fact Types is the secret sauce that leads to shared understanding of the business logic being modeled. When done well, it is the single largest enabler of ease of understanding of the decision models and aids in identifying opportunities for reuse. Done poorly, it is an impediment to understandability and reusability. To facilitate this, the Glossary in DM is intended to be expressed in business-friendly terms, not technical terms or abbreviations.

A consistent naming scheme should be used for all Fact Types. It should be compact yet sufficiently descriptive to clearly inform the viewer what the Fact Type represents. As a best practice, the following naming scheme is used (in the order shown):

1. **Business Concept** – one or two words (typically) that identify the business object or data entity to which the Fact Type belongs, sometimes referred to as the organizing factor. Examples: Person, Insured Location.

   When using a Business Information Model (BIM) Entity other than Root, the *business concept* for all the Fact Types associated with that BIM Entity – and only the Fact Types associated with that BIM Entity – should match the name of the BIM Entity. For example, if your BIM represents a pizza order and has a BIM Entity called Pizza, the names of all the Fact Types associated with the Pizza entity would have a *business concept* of Pizza.

2. **Descriptive Words** – one or more words that describe what the Fact Type represents. These might include a description of the state (e.g., Initial, Approved), an adjective (e.g., First, Current, Previous), or a unit of measure (e.g., In Years), in addition to the subject itself.

3. **Data Type** – most Fact Types should include the *data type* as part of the name as this brings clarity to what the Fact Type represents, how it is used, and what values it can take.

   Consider a Rule Family with two eligibility Fact Types where one is an *Indicator*, and one is a *Code* with three values. Without the *data type*, the viewer could not easily discern that these are defined differently, resulting in confusion.

The *data type* can be omitted when it is redundant. For example, there is no need to include "Quantity" in a Fact Type named Patient Age In Years or in a Fact Type named Borrower Credit Score.

4. **List** – when a Fact Type can hold multiple values, add the List suffix to its name. This is particularly important when a Decision refers to both a non-List variant and a List variant of a Fact Type. This is common in some Rule Families that process BIM Entities.

5. **[Business Context]** – when the same Fact Type needs to represent two or more different values within the same Decision or the same Flow (for example when one value comes from a datastore and another comes from a user interface), these need to be discriminated by context. This is achieved by creating two Fact Types and adding a *business context* suffix to their names, surrounded by square brackets to separate it from the rest of their name.

The concatenated name components are written in title case to express the Fact Type name.

Fact Type naming examples:

Original Name	Business Concept	Descriptive Words	Data Type	List	Business Context	Final Name
First name of patient on a claim	Patient	First	Name			Patient First Name
Term of a vehicle lease	Vehicle	Lease term in months	Quantity			Vehicle Lease Term In Months
Loan program eligibility for a borrower	Borrower	Loan program eligibility	Indicator			Borrower Loan Program Eligibility Indicator
List of borrower credit scores	Borrower	Credit score	Quantity	List		Borrower Credit Score List
Transaction data for a borrower's monthly income	Borrower	Monthly income	Amount		Transaction	Borrower Monthly Income Amount [Transaction]
Size of pizza in order	Pizza	Size	Code			Pizza Size Code

Fact Type names should not include abbreviations or acronyms unless they are readily understood by all the stakeholders.

3.3 Description

All Fact Types should have a clear description that:

- Aids in the understanding of what the Fact Type and its values represent
- Differentiates the Fact Type from other similar sounding or related Fact Types

The description should not use the words in the Fact Type name to define it as that would result in a circular definition. The description should also not include any business logic from Rule Families as part of the definition as that logic may change over time.

3.4 Data Types

3.4.1 Overview

The use of the proper Fact Type *data type* is essential to effective decision models.

The data type determines the available operators (see section 4.3 *Operators* below) and drives appropriate validation of the logic. Each data type also has one or more *display formats* associated with it. These formats do not impact the values or how they are saved, only how they are displayed.

This section of the document will describe the various data types available within Sapiens Decision and best practices as to when to use each.

3.4.2 Boolean Data Types

Sapiens Decision has one Boolean data type: *Indicator*. Unlike traditional Booleans which represent the values *true* and *false*, the *Indicator* data type in Sapiens Decision can represent any two textual values. As a best practice, the values should have appropriate business meaning. Some common examples include:

- Compliant / Noncompliant – appropriate when there is something to be compliant with
- Eligible / Ineligible – appropriate when there is something to be eligible for
- Valid / Invalid – appropriate when there is a validity test

Common examples that lack business meaning:

- True / False
- Yes / No

Single-word values are preferred. When using multi-word values, it is advisable to avoid values containing the word “Not” (e.g., Not Compliant) as these are difficult to read and understand in cells with negative operators (e.g., Is Not Not Compliant).

3.4.3 Text Data Types

Sapiens Decision has multiple data types that are used for text values: *Text*, *Code*, *Enumerator*, *Identifier*, and *Name*.

3.4.3.1 Text

The *Text* data type is used to represent free text with no Allowed Values. It is the default data type. Although Allowed Values can be defined for the *Text* data type, it is intended to represent unrestricted text.

3.4.3.2 Code

The Code data type is used to represent text with Allowed Values. If there are exactly two Allowed Values, the *Indicator* data type should be used instead of the *Code* data type.

Domain values for Fact Types with the *Code* data type should be explicitly listed as a *Regular set* of Allowed Values. For a more detailed explanation, please see section 3.5 *Allowed Values* below.

3.4.3.3 Enumerator

The *Enumerator* data type is used to represent an ordered set of values, for example: Sunday, Monday, Tuesday…. When using any of the other text data types, Allowed Values are ordered alphabetically. When using an *Enumerator* data type, Allowed Values are ordered in the sequence in which they were added to the Fact Type definition. The order can be used in the business logic such that “Monday” is considered greater than “Sunday” (even though “Sunday” comes after “Monday” alphabetically). Note that the underlying value associated with each value is its position in the Allowed Value list, starting from 0, and these values cannot be mapped via a *Model Mapping*.

3.4.3.4 Identifier

The *Identifier* data type is used to represent a key value, such as Account ID. Although Allowed Values can be defined for the *Identifier* data type, it is typically used without Allowed Values. The *Identifier* data type is rarely used: most implementations use the *Text* data type instead.

3.4.3.5 Name

The *Name* data type is used to represent a name value, such as Account Name. Although Allowed Values can be defined for the *Name* data type, it is typically used without Allowed Values. The *Name* data type is rarely used: most implementations use the *Text* data type instead.

3.4.4 Date Data Types

Sapiens Decision has multiple data types that are used for date and date-time values: *Date*, *Date & time, Day*, *Month*, *Month &year*, and *Year*.

3.4.4.1 Date

The *Date* data type is used to represent a full date.

3.4.4.2 Date & time

The *Date & time* data type is used to represent a full date and time (without time zone). See section 4.20 *Dealing with Time Zones* below for more information about time zone support.

3.4.4.3 Day

The *Day* data type is used to represent either the day of the month (1–31) or the day of the week (1–7 with Sunday as 1), depending on the selected display format.

Even though the underlying data type is numeric, the day of the week values are entered and displayed in the corresponding textual form (e.g., Sunday or Sun) in the Rule Family according to the selected display format.

3.4.4.4 Month

The *Month* data type is used to represent the month (1–12) portion of a date. Values are entered in their numerical form. Depending on the selected display format, the values are displayed either as a number (e.g., 1) or text (e.g., January).

3.4.4.5 Month & year

The *Month & year* data type is used to represent the month (1–12) and year portions of a date. Within DM, values are entered as a month and year per the selected display format, or by choosing a date from the date picker. At runtime, values are entered as full dates, but must have the day portion set to 1.

3.4.4.6 Year

The *Year* data type is used to represent the year portion of a date.

3.4.5 Numeric Data Types

Sapiens Decision has multiple data types that are used for numeric values: *Numeric*, *Quantity, Amount*, *Percent*, and *Basis points*.

3.4.5.1 Numeric

The *Numeric* data type is used to represent a general numeric value.

It is possible to configure an *Integer* display format for use with *Numeric* Fact Types. It is important to understand that this is merely a display format—*Numeric* Fact Types are always treated as having decimal values. This can cause issues with validation and with the generation of Test Values. Therefore,

it is not recommended to use an *Integer* display format for a *Numeric* Fact Type. In the example below, the Rule Family is incomplete if *Credit Score* is defined as *Numeric* – regardless of display format – since values such as 699.5 are not covered:

Credit Score	Credit Score Assessment Code
Is Greater Than or Equal To ⁝ 700	Is ⁝ High
Is Between ⁝ {550,699}	Is ⁝ Medium
Is Less Than ⁝ 550	Is ⁝ Low

Integer values should be defined with data type *Quantity*. In the example above, if *Credit Score* is defined as *Quantity*, then all values are covered, and the Rule Family is valid as shown.

3.4.5.2 Quantity

The *Quantity* data type is used to represent an integer value.

3.4.5.3 Amount

The *Amount* data type is used to represent a currency amount. The selected display format specifies the currency symbol to be added on the left and the number of decimal positions to be used.

3.4.5.4 Percent

The *Percent* data type is used to represent a percentage value. The "%" symbol is added on the right. In the testing facility, a value of 80 in a Fact Type with the *Percent* data type represents the value 80% or .8. At execution time, the decimal equivalent, .8, is expected as the input.

3.4.5.5 Basis points

The *Basis points* data type is used to represent a basis point (a unit equal to one hundredth of a percentage point) value. The "BP" symbol is added on the right. In the testing facility, a value of 80 in a Fact Type with the *Basis points* data type represents the value 80‰ or .008. At execution time, the decimal equivalent, .008, is expected as the input.

3.5 Allowed Values

As a best practice, you should define Allowed Values for all applicable Fact Types. Generally speaking, this includes all *Indicator* Fact Types, most *Code* Fact Types, and some Fact Types with a numeric data type.

The specified Allowed Values are used only for validation; the Allowed Values are not checked at execution time unless explicitly tested in data acceptance Decisions (see section 4.15 *Modeling for Untrusted Data* below).

Completeness validation is done only for Fact Types with Allowed Values. For example, the completeness of the *Credit Score* values below is not checked unless *Credit Score* is defined with Allowed Values (say 300 to 850):

Credit Score	Credit Score Assessment Code
Is Greater Than or Equal To ⁝ 700	Is ⁝ High
Is Between ⁝ {550,699}	Is ⁝ Medium
Is Less Than ⁝ 550	Is ⁝ Low

Allowed Values can be defined as either one or more *Ranges* of values or a *Set* (either *Regular set* or *Dynamic set*) of values. Since the Glossary in DM does not restrict the length of textual Fact Type values, Allowed Values for Fact Types with one of the text data types should be explicitly listed as a *Set* of Allowed Values rather than using *Ranges* of Allowed Values. The following—admittedly egregious—example illustrates the problem: if the valid values are A, B, C and are defined as the range A to C, "AAAA" would be incorrectly accepted as an allowed value. Conversely, *Ranges* should be used only with numeric or date Fact Type values.

Like Fact Type names, title case is used for textual allowed values.

As a best practice, all Allowed Values for a Fact Type should be defined in DM. This may not be practical when the list of Allowed Values is large (as defined by project standards, say 100) and/or changes frequently. Defining and managing the Allowed Values within the DM Glossary becomes cumbersome in these cases because of the frequent need to version such a Fact Type and all the Decisions and Flows that use it. If only a few of the values are referenced in Rule Families, it may be helpful to define only those values within DM. This will help reduce the number of versions for that Fact Type.

3.5.1 Dynamic Set

Another option to reduce the number of versions of a Fact Type is to define the Allowed Values as a *Dynamic Set*. When using the *Dynamic Set* option, values may be added at any time (requires the ALLOW_DYN_SET_UPDATE permission) to the list of Allowed Values for a candidate or approved Fact Type without versioning the Fact Type. This is useful when the set of values changes frequently and most of those changes are the addition of new values that do not impact existing business logic.

When using a *Dynamic Set*, the business logic would have to be written in such a way that it would be insensitive to additional values appearing in the input at execution time. This is typically accomplished by having at least one cell in the column with the *Is Not In* operator followed by all the values that were referenced within the column. In this manner, the business logic could be complete, even if all the run-time values were not known at modeling time. However, validation of a *Dynamic Set* condition column does not test for completeness.

3.5.2 Handling Different Sets of Allowed Values

A Fact Type may have one set of values in one context and another set of values in a different context. This can be handled in several ways:

- One Fact Type version with a superset of the Allowed Values
- Two Fact Type versions, each with its own set of Allowed Values
- Two Fact Types, each with its own set of Allowed Values

While the management of the Fact Type is simplest in the first approach with only one Fact Type version to deal with, completeness validation will not be effective as it will look for the handling of the full set of Allowed Values in each context.

The second approach, Fact Type versioning, falls apart if the lists of Allowed Values ever need to be modified. This could result in a flip-flopping of the list of Allowed Values across Fact Type versions making the management of the Fact Type difficult and error prone.

Therefore, the third approach, distinct Fact Types for each context, is the best practice. Although you need to maintain multiple Fact Types, each Fact Type has its own easily managed and complete set of Allowed Values (for its context) providing effective completeness validation.

3.6 Domain Templates

Domain Templates enable a Glossary Administrator to predefine a domain consisting of the Data Type, Display Format, and Allowed Values. This is very useful for ensuring consistency and improving productivity, particularly when there are large sets of Allowed Values.

Domain Templates are best used for domains that do not change, such as various *Indicator* domains like the ones below:

Domain Name	Data Type	Display Format	Allowed Values
Compliance	Indicator	Text	Compliant / Noncompliant
Eligibility	Indicator	Text	Eligible / Ineligible
Validity	Indicator	Text	Valid / Invalid

Updating a *Domain Template* does not automatically update any Fact Type versions that use that *Domain Template*; however, you do get a warning when viewing a Fact Type version that is out of sync with its *Domain Template*. You can then re-select the *Domain Template* to re-sync it.

Currently the only way to see where a *Domain Template* is used is by means of a Glossary Export in XML format.

3.7 Fact Type Versions

Fact Types in DM can have multiple *versions* where each version can have different attributes such as Name, Data Type, Allowed Values, etc. When should a new Fact Type be created as opposed to creating a new Fact Type version?

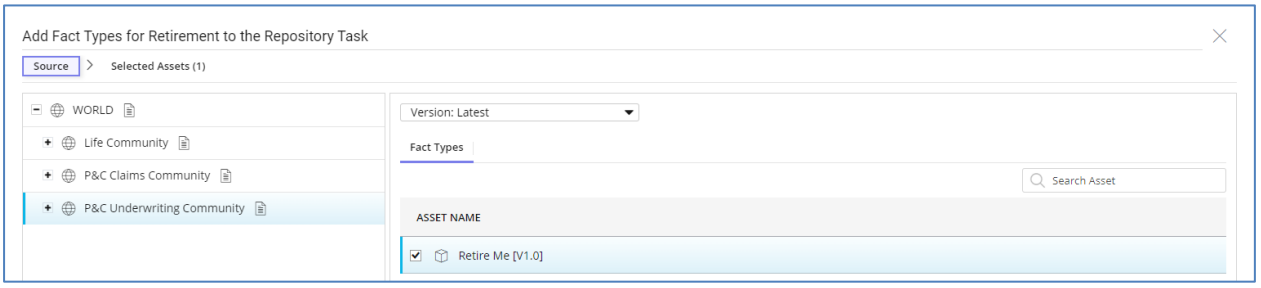
Versions should be used when a newer Fact Type version replaces the previous version. If all the places where the old Fact Type version was being used can be updated to use the new Fact Type version, then a new version is the way to go. If, however, some current versions of Decisions need to continue using the old Fact Type version, as a best practice, it is advised to create a new Fact Type to avoid deployment conflicts and simplify management.

Note that renaming a Fact Type – creating a new version of the Fact Type – does not release the old Fact Type name to be used in any other Fact Type. This is to ensure Fact Type name uniqueness and audit history.

3.7.1 Retiring an Unused Fact Type Version

A Fact Type version that is not used anywhere or one that is used only in retired Decision versions may be retired. This is useful to declutter the Glossary and is the only way to remove Fact Type versions that are no longer needed.

To retire a Fact Type version, create and submit a *Repository Task* with Task Type *Retire Fact Types*. Select the Fact Type versions to be retired:



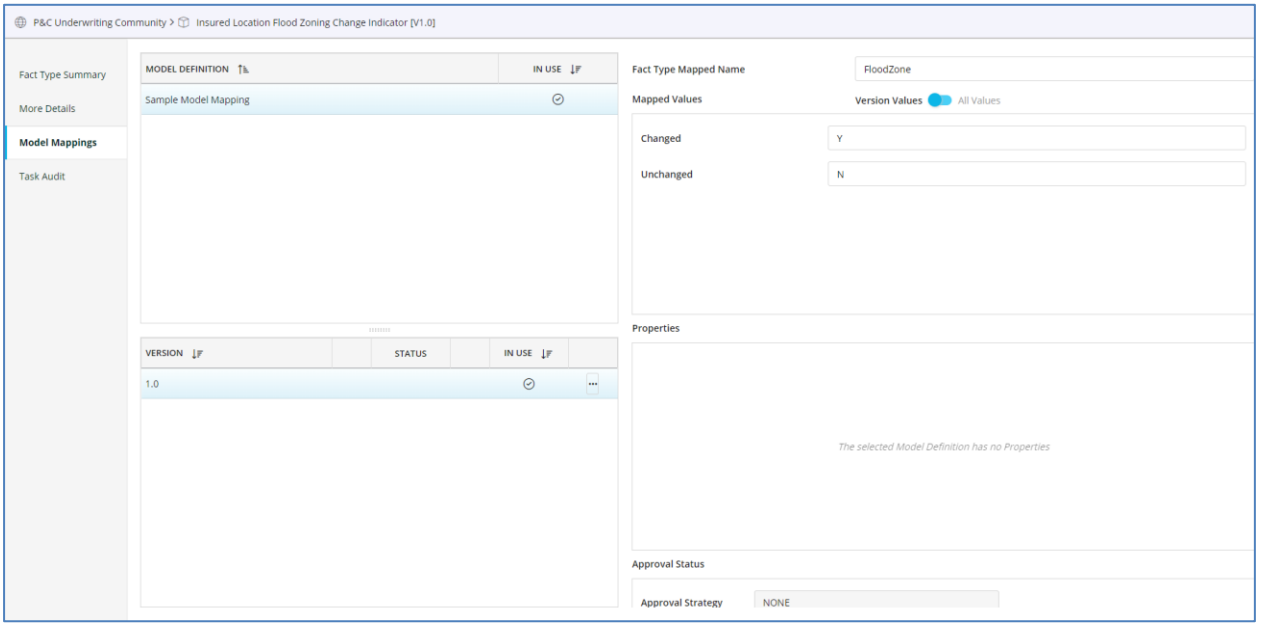
Retired Fact Type versions are generally not displayed anywhere in DM. They can be viewed via an *Advanced Search* that uses *Status is Retired* as one of its conditions.

To restore a retired Fact Type version, create and submit a *Repository Task* with Task Type *Unretire Fact Types*, then select the Fact Type version to be restored.

A user can create a new Fact Type with the same Name as a Fact Type whose versions have all been retired. In this situation, the system will create a new version of the retired Fact Type rather than creating a new Fact Type. This means that the user does not have to be aware of the existence of a retired Fact Type before creating a new one with the same Name.

3.8 Model Mappings

Fact Types in DM can be mapped via one or more *Model Mappings* which are used to specify the name, values, and properties that a Fact Type has in logical, physical, and/or canonical data models. Using the Model Mapping feature, the business-friendly names and business-friendly values that are defined in the Glossary are mapped to technical names and technical values for use at execution time or to document data lineage.

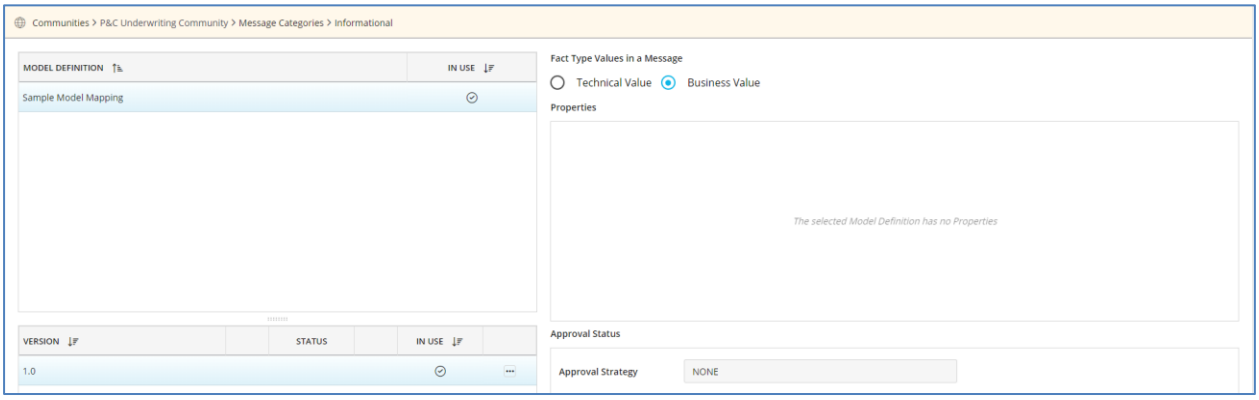


The Model Mapping information is shared across all versions of a Fact Type. You can use the Mapped Values toggle as shown above to filter the display to see the Fact Type's values from only the version currently being viewed or from all versions.

The Model Mapping information is used in four scenarios:

1. Deployment – if the target Environment for the deployment has a model version defined, the business-friendly names and business-friendly values of any Fact Type that has a Model Mapping in the specified model version are translated to their corresponding technical names and technical values per the specified model version.
2. Test Group export – if an Environment is selected during Test Group export and the Environment has a model version defined, the business-friendly names and business-friendly values of any Fact Type that has a Model Mapping in the specified model version are translated to their corresponding technical names and technical values per the specified model version.
3. Test Group import – if an Environment is selected during Test Group import and the Environment has a model version defined, the technical names and technical values of any Fact Type that has a Model Mapping in the specified model version are translated to their corresponding business-friendly names and business-friendly values per the specified model version.
4. Fact Type values in messages – by default when running outside the testing facility in DM, Fact Type values in messages are the technical values. If you wish to see business-friendly Fact Type values in

messages, set *Business Value* for the *Fact Type Values in a Message* setting for the model version(s) for each applicable Message Category and deploy to an Environment that has one of those model versions. This will result in translation of the technical values to their corresponding business-friendly values per the specified model version.



Note that the only translation that happens at run-time is the translation of Fact Type values in messages from technical values to business-friendly values as described in scenario 4 above.

In all four scenarios, if the source value is not found in the Model Mapping, it is passed through without translation.

Model Mappings are not used in the testing facility in DM; test execution is done using business-friendly names and business-friendly values.

The best practice is to map only one Fact Type to any given *Fact Type Mapped Name* for a particular model version. DM does not prevent defining more than one, although a warning is displayed when you create a duplicate *Fact Type Mapped Name* within a particular model version. However, you will get an error when you attempt to export a Revision containing two or more Fact Types with the same *Fact Type Mapped Name*. The best way to find these duplicate mappings is by exporting the Glossary and looking at the mapping information there. Note that duplicate technical names can also occur due to the normalization that DM does for business-friendly names when a *Fact Type Mapped Name* is not provided, removing spaces and special characters, prior to code generation.

It is not recommended to provide a *Fact Type Mapped Name* for Fact Types that are used as Decision conclusions, so that all parties refer to a decision by the same name (although as mentioned above, in the absence of *Fact Type Mapped Name*, the technical name will be a normalized version of the business-friendly name).

Since value translation can occur in either direction, there should be a one-to-one correspondence between the business-friendly values and the technical values within a model version. DM does not prevent having the same technical value for more than one business-friendly value. This may cause **unexpected results** in the scenarios (see scenarios 3 and 4 above) where a technical value is translated to it corresponding business-friendly value.

When debugging a case of unexpected results, a good place to start is by verifying that the execution environment was provided the names and values it expected: business-friendly names and values in the testing facility in DM, technical names and values for all environments that use Model Mapping.

Mapping of values is typically done for input Fact Types and is often also used for output Fact Types. There are situations where you will need to map the values of an interim Fact Type, for example, when the value of a Fact Type with a *Model Mapping* is being compared against the value of a Fact Type without a *Model Mapping*.

Consider the following logic:

…	Policy Hurricane Zoning Change Indicator	…	…
…	Is ⋮ Policy Flood Zoning Change Indicator	…	…

…	…	…	Policy Hurricane Zoning Change Indicator
…	…	…	Is ⋮ Changed
…	…	…	Is ⋮ Unchanged

Policy Hurricane Zoning Change Indicator has a supporting Rule Family, so it wouldn't typically have a Model Mapping. Its business-friendly values are Changed and Unchanged. *Policy Flood Zoning Change Indicator* is a persistent Fact Type and has the Model Mapping shown above. When testing in DM, only business-friendly values are used, so the domain for both Fact Types is the same and the logic works as expected. However, in the run-time environment, the domain for *Policy Flood Zoning Change Indicator* is the technical values Y and N and these do not match the business-friendly domain being used for *Policy Hurricane Zoning Change Indicator*, yielding **incorrect results**. The solution is to add the same Model Mapping to *Policy Hurricane Zoning Change Indicator*.

3.9 Glossary Hierarchy

The DM repository is structured into a hierarchy of *Communities*. Each Community may have its own *Glossary* or inherit its Glossary from the Community hierarchy in which it is based. Since Glossaries may be defined at multiple levels in the Community hierarchy and Fact Types may be defined in any of the existing Glossaries (a given Fact Type name can be defined in only one Glossary but may be moved to another Glossary at any time), guidelines are needed as to how to structure the Glossary hierarchy and where each Fact Type should reside.

Before we get to the guidelines, it is important to understand that all Fact Types, regardless of where they are defined, are viewable by everyone with *FT_READ* permission (it is a global permission). Furthermore, any user with *FT_WRITE* permission (also a global permission) can create a draft version of any Fact Type, even if they do not have access to the Community associated with the Glossary in which that Fact Type is defined, and then submit it for approval per the Fact Type Approval Process being used in their Modeling Task.

So why use multiple Glossaries at all? There are several benefits to having Community Glossaries:

- Grouping of related Fact Types together in the same Community Glossary supports easier browsing for Fact Types within a small set of Fact Types.
- The auto-complete mechanism uses a proximity-based search to prioritize best matching Fact Type name based on proximity of the Glossary to the Modeling Task Community, specifically, searching first in the Modeling Task Community Glossary if it exists, then up the Community hierarchy, then all other Glossaries until it fulfills its quota of matches (currently 25).
- Custom extensibility allows for different sets of Custom Metadata and Domain Templates per Community Glossary.

For the first project it is often tempting to simply use the WORLD Glossary. As additional projects are added, it becomes more and more difficult to manage the ownership of the various Fact Types in the WORLD Glossary. Eventually, it is decided to add some Community Glossaries. By this time, it can be tedious to determine which Fact Types are used in each Community so that they could be moved to the appropriate Community Glossary.

In the long run, it is easier to build the Glossary hierarchy bottom-up, starting with the lowest-level Community, and moving Fact Types up the Glossary hierarchy as needed for visibility across Communities. This approach results in Fact Types being placed in the lowest Community Glossary that offers visibility to all the Communities where the Fact Type is used. Of course, the Fact Type could be placed higher if desired.

3.10 Glossary Export and Import

DM offers two formats for Glossary import and export: CSV and XML.

The XML format is richer in metadata content and is the recommend import format. Always use XML format for interchange between DM repositories. As XSLT is ideal for transforming XML schemas, also use the XML format for import from third-party metadata systems that can produce an XML extract.

The CSV format is much more human-readable and human-maintainable. It is, therefore, more appropriate for export for human consumption and for import when the file is created manually.

3.11 Glossary Naming

For ease of identification, Glossary names should be unique.

The recommended naming convention is to use the Community name (dropping the “Community” suffix) with a “Glossary” suffix. For example, if the Community is named *Sample Community*, the Glossary should be named *Sample Glossary*.

4 Modeling

4.1 What Should Be Modeled in DM?

Not all logic is a good fit for decision modeling. In this section we will explore the key considerations that should be weighed in determining whether to model particular logic in a decision model.

Any logic that the business is interested in managing is an excellent candidate to be modeled in a decision model. To achieve manageability of the logic, the logic must be visible to the business. Historically, logic has been buried deep in code, invisible and incomprehensible to the business. Decision models are expressed in business terminology in easy-to-learn graphical and tabular models and are easily accessible to all interested parties by means of the robust navigation and search tools in DM. Logic that is primarily technical in nature, such as data transformation logic, is probably not of concern to the business, and while it could be modeled in decision models, there is likely little advantage to doing so.

Logic that has business variations for different contexts (such as jurisdictions, products, points in time, etc.) is another candidate to be modeled in a decision model. The use of views (see section 4.2 *Views and Versions* below) can simplify the logic compared to traditional approaches, making the variations more manageable.

Complex logic that is comprised of multiple factors, with interplay among them, is also an excellent candidate for a decision model, particularly where the logic is multi-staged/stepped and there are interim conclusions. Eligibility rules typically fall into this category. Conversely, simple logic that is comprised of one factor or maybe two factors with minimal interplay might best be modeled using traditional approaches. At the other end of the spectrum, algorithmic logic or complex calculations, especially non-trivial date calculations or complicated mathematical derivations such as those often found in valuations or rating, are usually not a good fit for a decision model since the details of the calculations are not of interest to the business and the calculations do not change often. If these complex calculations have a single result that is of interest, incorporating the calculations into a decision model via a User-Defined Function (for more information, see chapter 15 *User-Defined Functions (UDFs)* below) provides the best of both worlds.

Traceability requirements are another dimension where decision models are a good fit. In DM it is easy to attach Decisions and Rule Families to Knowledge Sources (see section 9.1 *Knowledge Sources* below) to provide traceability from a source document (or section of the source document) to the models that implement the logic requirements expressed in the source document. DM also offers a deep linking capability where direct links to specific assets can be embedded in external tools and documents providing traceability to the implemented logic. This level of bi-directional traceability is difficult to achieve with traditional solutions.

Decision modeling is a great fit for logic that changes frequently as well as logic that requires easy and quick modification. Decision models are typically created and maintained by those who understand the business requirements. They can immediately validate and test models to see that they function as expected and, where the organizational polices and infrastructure allow, are able to deploy the models all the way to production. In some Sapiens Decision projects, this cycle is completed in less than a day.

Rule Families may not be a good fit for the determination of parameter values. Tables of parameter values are often very large, far exceeding best practices for Rule Family size (see section 4.12.1 *Rule Family Size* below). Maintaining the parameter values separately from the business logic (i.e., in parameter tables) allows different users to manage each and enables separate governance regimes. Use of parameter tables allows replacement of code deployments with data updates, which may require fewer regulatory/procedural compliance steps than code deployments, facilitating even more frequent change.

Decision model execution provides rich output that can be used to capture the details of what happened, including inputs, outputs, values of interim Fact Types, messages, and which rows fired. This can easily be consumed and used for analytics and business intelligence. Traditional solutions are typically not instrumented to provide this level of detail; with decision modeling, it is always available.

Some examples of decisions that are good candidates to be modeled in a decision model are ones that…

- Change frequently and/or on short notice
- Have multiple business variations
- Have been difficult to implement within rigid deadlines
- Have been error prone
- Have a significant negative impact on reputation, customer satisfaction, or economics
- Are shared across and/or beyond the organization
- Are subject to regulatory governance
- Need highest quality of input data
- Are better governed by businesspeople
- Are targets of analytical and business improvements

4.2 Views and Versions

DM supports both *views* and *versions* and either can be used to create business logic variations. By default, if you copy an asset to a Modeling Task, DM will create a new version of the asset upon submission of the Modeling Task. This section of the document aims to describe when creation of a new view would be appropriate instead of creating another version.

Before we can describe when to create a new view, we must define what we mean by a view. Views are business logic variations of the same conclusion for specific circumstances such as different geographies (states, for example) or jurisdictions, different products or line of business, different customers or contracts, different statuses or points in time (at loan origination, for example), or any combination of these factors. The view notion can be applied at the Decision level and at the Rule Family level. A Decision may include Rule Families that are shared with other Decisions as well as Rule Families that are unique to the particular Decision, resulting in the ability to handle specific business logic variations while reusing everything else.

Versions should be used when a newer version in some sense replaces the previous version; bug fixes are a common example. If all the places where the old version was being used can be updated to use the new version, then a new version is the way to go. We sometimes see different Rule Family versions being used to create business logic variations that will coexist in perpetuity, in effect creating branches. Some current versions of Decisions will need to continue to use previous versions of the Rule Family. If these variations undergo further changes, the branches become intertwined making the maintenance of the different variations more difficult. Instead, as a best practice, it is advised to simplify management by using views to handle this situation.

Another common scenario in which the use of views would be appropriate is when you find the same condition(s) being tested in multiple Rule Families in the same Decision. Some common examples of this include different jurisdictions or lines of business. These conditions would be candidates to become one or more views. Pulling these conditions out simplifies the remaining business logic.

4.2.1 View Naming

The naming of views can itself be challenging. Generally, you will want to name each view based on the context it represents. However, in some cases this could be quite confusing.

Suppose we are authoring logic for eligibility for product ABC. The eligibility could vary by state. Suppose we start with Alaska and create a Rule Family called *Product ABC Eligibility Indicator (View: Alaska) V1.0*. We then model Florida and find out that its logic is identical to that of Alaska. We might want to create a *Product ABC Eligibility Indicator (View: Florida) V1.0* Rule Family, but DM does not allow us to do that as it is a duplicate of the Alaska view and DM tries to force reuse where possible. So, we attach the existing Alaska view of the Rule Family to our Florida view of the Decision. Everything works fine, but some users might find it confusing, or even incorrect, to have an Alaska Rule Family in a Florida Decision.

Later, the rules in Alaska change and so V1.1 of the Alaska view of the Decision uses the Nevada view of the Rule Family which happens to already have the logic we now need in Alaska. Looking at the Florida view of the Decision, it still has the *Product ABC Eligibility Indicator (View: Alaska) V1.0* Rule Family, and Alaska no longer uses that logic! Even more confusing!

There are two approaches to handling the naming of the views in these scenarios:

1. If there is some stable characteristic that defines the variations in the logic, use that as your context. So, instead of Alaska and Florida, perhaps use States With No Income Tax.

2. In the absence of a stable characteristic, de-contextualize the view names and use something like Variant 1, Variant 2, … or Option 1, Option 2, …. See chapter 8 *Decision Selection* below for a discussion of how to select the view and version to run.

4.3 Operators

4.3.1 Overview

To fully describe business logic using The Decision Model (TDM) it is essential to have a comprehensive understanding of the various *operators* available for use. Operators are used to connect a Fact Type and an operand so that the logical relationship of the two is fully described. TDM provides a wide variety of operators. DM aids in the process of operator selection by automatically providing a filtered list of relevant operators based on the data type of the Fact Type found in the column header of the cell being edited and whether the Fact Type holds a single value or multiple values (List Fact Type). This section of the document aims to describe the most effective operators within the context of specific scenarios which may arise during business logic modeling.

4.3.2 Available Operators

The following table contains the list of condition cell operators within DM*:

Standard Decision Operator	Description	Symbol
Is	Equal To	=
Is Not	Not Equal To	!=
Is Less Than	Less Than	<
Is Less Than or Equal To	Less Than or Equal To	<=
Is Greater Than	Greater Than	>
Is Greater Than or Equal To	Greater Than or Equal To	>=
Is Between	Greater Than or Equal To X AND Less Than Or Equal To Y	[]
Is Not Between	Less Than X OR Greater Than Y	![]
Is In the Range	Greater Than X AND Less Than Y	][
Is Not In the Range	Less Than Or Equal To X OR Greater Than Or Equal to Y	![][
Is In	IN {}	In
Is Not In	NOT IN {}	!In
Is On the Date	= DD/MM/YYYY	=
Is Not On the Date	Not DD/MM/YYYY	!=
Is Before the Date	< DD/MM/YYYY	<
Is On or Before the Date	≤ DD/MM/YYYY	<=
Is After the Date	> DD/MM/YYYY	>
Is On or After the Date	≥ DD/MM/YYYY	>=

Standard Decision Operator	Description	Symbol
Is Between the Dates	≥ and ≤ DD/MM/YYYYY	[]
Is Not Between the Dates	< DD/MM/YYYY OR > DD/MM/YYYY	![]
Is On the Time	= HH.MM.SS	=
Is Not On the Time	Not HH.MM.SS	!=
Is Before the Time	< HH.MM.SS	<
Is On or Before the Time	≤ HH.MM.SS	<=
Is After the Time	> HH.MM.SS	>
Is On or After the Time	≥ HH.MM.SS	>=
Is Between the Times	≥ and ≤ HH.MM.SS	[]
Is Not Between the Times	< HH.MM.SS OR > HH.MM.SS	![]
Is On the Date and Time	= DD/MM/YYYY, HH.MM.SS	=
Is Not On the Date and Time	Not DD/MM/YYYY, HH.MM.SS	!=
Is Before the Date and Time	< DD/MM/YYYY, HH.MM.SS	<
Is On or Before the Date and Time	≤ DD/MM/YYYY, HH.MM.SS	<=
Is After the Date and Time	> DD/MM/YYYY, HH.MM.SS	>
Is On or After the Date and Time	≥ DD/MM/YYYY, HH.MM.SS	>=
Contains	Superset Of or Equal To (cardinality ignored)	~
Does Not Contain	Not a Superset Of or Equal To (cardinality ignored)	!~
Contains All	Superset Of or Equal To (cardinality ignored)	~ All
Does Not Contain All	Not a Superset Of or Equal To (cardinality ignored)	!~ All
Contains Any	Overlaps With	~ Any
Does Not Contain Any	Does Not Overlap With	!~ Any
Contains Exactly	Superset Of or Equal To (cardinality considered)	~ Exactly
Does Not Contain Exactly	Not a Superset Of or Equal To (cardinality considered)	!~ Exactly
Contains Only	Subset Of or Equal To (cardinality ignored)	~ Only
Does Not Contain Only	Not a Subset Of or Equal To (cardinality ignored)	!~ Only
Contains Same As	Equal To (cardinality ignored)	~ Same As
Does Not Contain Same As	Not Equal To (cardinality ignored)	!~ Same As
Is Exactly	Equal To (cardinality considered)	= Exactly

Standard Decision Operator	Description	Symbol
Is Not Exactly	Not Equal To (cardinality considered)	!= Exactly

* Not all operators are available in all situations. Available condition cell operators are determined by the condition Fact Type data type and whether the Fact Type holds a single value or multiple values (List Fact Type).

The above operators are case-sensitive when comparing textual values. The operators that deal with text values also have a case-insensitive variant, represented in DM with an (IC) suffix which stands for Ignore Case, for example *Is (IC)*.

The following table contains the list of conclusion cell operators within DM*:

Standard Conclusion Operator	Description	Symbol
Is	Assignment	=
Is Incremented By	Used with numeric Conclusion Fact Types in a multi-hit Rule Family or with BIMs (see section 4.12.1 below)	+=
Is Appended With	Used when building a List Fact Type (see section 4.3.5.1 below)	+=

* Not all operators are available in all situations. Available conclusion cell operators are determined by the conclusion Fact Type data type, whether the Fact Type holds a single value or multiple values (List Fact Type), and whether the conclusion Fact Type resides in the same BIM Entity as the one attached to the Rule Family.

4.3.3 Operator Display Mode

Within DM a user may select to view either the full name of an operator or its symbol (see the tables in section 4.3.2 *Available Operators* above) by selecting the *Set operator display mode to symbol/name* option accessible by right-clicking the top-left corner of a Rule Family table.

Symbol display mode is used to display a Rule Family more compactly, allowing a user to see more of the Rule Family at once.

The example below shows the same cell in both display modes:

Name display mode:

Is Greater Than or Equal To : $20.00

Symbol display mode:

>= : $20.00

4.3.4 Operators Used with a Range Operand

Relevant Operators: Is Between, Is Not Between, Is In the Range, Is Not In the Range, Is Greater Than, Is Greater Than or Equal To, Is Less Than, Is Less Than or Equal To, Is Between the Dates, Is Not Between the Dates, Is Between the Times, Is Not Between the Times, Is After the Date, Is After the Date and Time, Is After the Time, Is Before the Date, Is Before the Date and Time, Is Before the Time, Is On or After the Date ,Is On or After the Date and Time, Is On or After the Time, Is On or Before the Date, Is On or Before the Date and Time, Is On or Before the Time

When modeling logic that uses a range as an operand, there are three basic operator options each producing a different logical statement. The first is the use of the variants of the operators *Is Between* and *Is Not Between*. Second is the use of *Is In the Range* and its counterpart *Is Not In the Range*. Third is the use of a combination of Boolean operators to include one end of the range while excluding the other end.

4.3.4.1 Is Between

Is Between and its variants are used to express a range operand that includes the specified endpoints. The simple example below indicates that Loan Amount *Is Between* $100,000 and $200,000 with both endpoints being inclusive. For instance, if a Loan Amount of either $100,000 or $200,000 were used as the input this cell would evaluate to true. However, if a Loan Amount of $99,999.99 or $200,000.01 or any other amount outside of the range were used as the input this cell would evaluate to false.

Loan Amount
Is Between : {$100,000.00,$200,000.00}

The same principle applies for the use of *Is Between the Dates* and *Is Between the Times*, only the operands and inputs used are either dates or times depending on which is relevant.

The negative or opposite of *Is Between* is the operator *Is Not Between*. This abides by the same rule for including endpoints as *Is Between* but produces the opposite effect. In the example below we use the same endpoints for our range but now use the operator *Is Not Between*. This time if a Loan Amount of either $100,000 or $200,000 is used as the input this cell would evaluate to false because either value is between the range selected. Conversely, if a Loan Amount of $99,999.99 or $200,000.01 or any other amount outside of the range is used as the input this cell would evaluate to true.

Loan Amount
Is Not Between : {$100,000.00,$200,000.00}

4.3.4.2 Is In the Range

The operators *Is In the Range* and *Is Not In the Range* differ from the variants of *Is Between* in that their endpoints are excluded from the range. In the example shown below, if a Loan Amount of $100,000 or $200,000 were used as the input this cell would evaluate to false because the endpoints are not included in the range. As with the *Is Between* operators, if any amount outside of the range were used as the input, this cell would evaluate to false. If any value greater than $100,000 and less than $200,000 were used as the input this cell would evaluate to true.

Loan Amount
Is In the Range : {$100,000.00,$200,000.00}

If *Is Not In the Range* is used we still see that the endpoints are not included in the range specification but the implied result is the opposite such that a value of $100,000 or $200,000 or any other amount outside of the range were used as the input this cell would evaluate to true. If any value greater than $100,000 and less than $200,000 were used as the input this cell would evaluate to false.

Loan Amount
Is Not In the Range : {$100,000.00,$200,000.00}

4.3.4.3 Left Excluded and Right Excluded Ranges

To achieve a combination of the two previous uses of ranges such that one endpoint is included in the range while the other is excluded from the range we are required to use a combination of Boolean logic statements. To accomplish this, two columns containing the same column header must be used. On a single row of the Rule Family Table each endpoint of the range must be defined in a separate column using the Boolean operators *Is Less Than, Is Greater Than, Is Less Than or Equal To, Is Greater Than or Equal To* (or the equivalent date and time operators). As a best practice, you should use the first instance of the Condition Column (leftmost) to define the lower boundary of the range and use the second instance of the Condition Column (rightmost) to define the upper boundary of the range. This will increase clarity and readability of the logic.

To create a *Left Excluded Range*, simply use the operator *Is Greater Than* when defining the lower boundary and the operator *Is Less Than or Equal To* when defining the upper boundary. In the example shown below an input value of $100,000 would evaluate to false while an input value of $200,000 would evaluate to true for this row.

Loan Amount	Loan Amount
Is Greater Than : $100,000.00	Is Less Than or Equal To : $200,000.00

To create a *Right Excluded Range*, simply use the operator *Is Greater Than or Equal To* when defining the lower boundary and the operator *Is Less Than* when defining the upper boundary. In the example shown below an input value of $100,000 would evaluate to true while an input value of $200,000 would evaluate to false for this row.

Loan Amount	Loan Amount
Is Greater Than or Equal To : $100,000.00	Is Less Than : $200,000.00

Be aware that the operators *Is Less Than* or *Is Less Than or Equal To* cannot be used to define the lower boundary and the operators *Is Greater Than* or *Is Greater Than or Equal To* cannot be used to define the upper boundary as both situations result in conflicting logic.

The table that follows shows an example of how to handle multiple ranges:

Loan Amount	Loan Amount
	Is Less Than : $100,000.00
Is Greater Than or Equal To : $100,000.00	Is Less Than : $200,000.00
Is Greater Than or Equal To : $200,000.00	Is Less Than : $300,000.00
Is Greater Than or Equal To : $300,000.00	

Note that for readability and understandability, all the lower boundaries (*Is Greater Than or Equal To*) conditions are in the left-hand column, all the upper boundaries (*Is Less Than*) are in the right-hand column, and the rows are in ascending order.

4.3.5 Operators Used with a List Fact Type

Relevant Operators: Is Appended With, Contains, Does Not Contain, Contains All, Does Not Contain All, Contains Any, Does Not Contain Any, Contains Only, Does Not Contain Only, Contains Same As, Does Not Contain Same As, Contains Exactly, Does Not Contain Exactly, Is Exactly, Is Not Exactly

A List Fact Type is a Fact Type that can hold multiple values. It is defined by setting its List Indicator to *Multiple Values*. List Fact Types have their own operators. We'll explore these operators in this section.

4.3.5.1 Building a List

To build a list, you must designate a List Fact Type as the Conclusion Fact Type for a Rule Family. The only conclusion operator available for a List Fact Type is *Is Appended With*. This operator will concatenate the list with the provided operand value. In the example below, if the first row of the Rule Family evaluates to true then *Order Applicable Discount Code List* will be concatenated with the value *Early Payment Discount*.

Order Early Payment Discount Eligibility Indicator	Order Quality Credit Discount Eligibility Indicator	Order Student Discount Eligibility Indicator	Order Applicable Discount Code List
Is : Eligible	Is : Eligible		Is Appended With : Early Payment Discount
		Is : Eligible	Is Appended With : Student Discount
…	…	…	…

When building a list, Sapiens Decision allows for multiple conclusion values to result from a single Rule Family execution. Returning to our previous example, if both rows evaluated to true then the Rule Family will result in two distinct conclusions: *Is Appended With Early Payment Discount* and *Is Appended With Student Discount*. This will result in the construction of a single list: *{Early Payment Discount,Student Discount}.*

Note that there is currently no mechanism in Decision to remove values from a list.

4.3.5.2 Lists as Condition Fact Types

When a List Fact Type is used as a Condition Fact Type, Sapiens Decision offers a variety of operators to compare its values with those in the operand. Most of Sapiens Decision's list operators do not consider the cardinality of their operand values (i.e., multiple instances of the same value are ignored). The operators that do consider cardinality are indicated by *Exactly* in their names. All the list operators ignore the order of the values in the list and in the operand. A description of the proper use for each operator is provided below.

4.3.5.3 Contains/Contains All

The *Contains* and *Contains All* operators are identical. As a best practice, you should choose just one (along with the corresponding *Does Not Contain* operator) for use in your decision models and disable the other one via the DM administration options. We will use the *Contains* operator in this document.

The *Contains* operator assesses whether all the operand values are found within the Condition Fact Type list. The order and cardinality of the values are ignored in making this evaluation.

Assumption: The input for *Order Applicable Discount Code List* is the list *{Student Discount,Early Payment Discount}.*

Order Applicable Discount Code List
Contains : {Quality Credit Discount,Early Payment Discount}
Contains : {Early Payment Discount}

Only the second row evaluates to true. The list *{Student Discount,Early Payment Discount}* contains all the values in the operand *{Early Payment Discount}*, however it does not contain all the values in the operand *{Quality Credit Discount,Early Payment Discount}* which is why the first row did not evaluate to true.

4.3.5.4 Contains Any

The *Contains Any* operator assesses whether any of the operand values are found within the Condition Fact Type List. In other words, is there any overlap between the operand values and the Condition Fact Type list? The order and cardinality of the values are ignored in making this evaluation.

Assumption: The input for *Order Applicable Discount Code List* is the list *{Early Payment Discount,Quality Credit Discount}*.

Order Applicable Discount Code List
Contains Any : {Quality Credit Discount,Early Payment Discount}
Contains Any : {Student Discount}

Only the first row evaluates to true. The list *{Early Payment Discount,Quality Credit Discount}* contains one or more of the values in the operand *{Quality Credit Discount,Early Payment Discount}*, however it does not contain any of the values in the operand *{Senior Discount}* which is why the second row evaluates to false.

Note: when the operand is just a single value, *Contains* and *Contains Any* return the same result.

4.3.5.5 Contains Exactly

The *Contains Exactly* operator assesses whether all the operand values are found within the Condition Fact Type list. The cardinality of the values is considered when making this evaluation, however the order of the values is ignored. Stated differently, we are checking the Condition Fact Type list for at least as many instances of each distinct value in the operand values.

Assumption: The input for *Pizza Topping Code List* is the list *{Spinach,Tomatoes}*.

Pizza Topping Code List
Contains Exactly : {Spinach,Tomatoes,Spinach}

The row evaluates to false since the list *{Spinach,Tomatoes}* does not have the same number of *Spinach* values as in the operand *{Spinach,Tomatoes,Spinach}*.

New assumption: The input for *Pizza Topping Code List* is the list *{Spinach,Spinach,Tomatoes,Tomatoes,Onions}*.

We are looking for at least two instances of *Spinach* and at least one of *Tomatoes*. With our new assumption the Condition Fact Type list has two instances of *Spinach*, two of *Tomatoes*, and one of *Onions*, so the row evaluates to true.

4.3.5.6 Contains Only

The *Contains Only* operator assesses whether all the Condition Fact Type values are found within the operand values. The order and cardinality of the values are ignored in making this evaluation.

Assumption: The input for *Pizza Topping Code List* is the list *{Spinach,Tomatoes}*.

Pizza Topping Code List
Contains Only : {Tomatoes,Onions,Green Peppers}
Contains Only : {Tomatoes,Spinach}

The first row evaluates to false since the value *Spinach* that is found in the Condition Fact Type list is not in the operand values. The second row evaluates to true since all the values in the Condition Fact Type list (*Spinach* and *Tomatoes*) are found in the operand values.

4.3.5.7 Contains Same As

The *Contains Same As* operator assesses whether all the operand values are found within the Condition Fact Type list and vice versa. The order and cardinality of the values are ignored in making this evaluation.

Assumption: The input for *Pizza Topping Code List* is the list *{Spinach,Tomatoes}*.

Pizza Topping Code List
Contains Same As : {Tomatoes,Onions,Green Peppers}
Contains Same As : {Tomatoes,Spinach}

The first row evaluates to false since both the Condition Fact Type list and the operand contain values that are not in the other (*Spinach*, *Onions*, *Green Peppers*). The second row evaluates to true since all the values in the Condition Fact Type list (*Spinach* and *Tomatoes*) are found in the operand values and all the values in the operand are found in the Condition Fact Type list.

4.3.5.8 Is Exactly

The *Is Exactly* operator assesses whether all the operand values are found within the Condition Fact Type list and vice versa. The cardinality of the values is considered when making this evaluation, however the order of the values is ignored. Stated differently, we are checking that the Condition Fact

Type list and the operand have the same distinct values and the same number of instances of each distinct value.

Assumption: The input for *Pizza Topping Code List* is the list *{Spinach,Tomatoes}*.

Pizza Topping Code List
Is Exactly : {Spinach,Tomatoes,Spinach}

The row evaluates to false since the value *Spinach* appears twice in the operand value but only once in the Condition Fact Type list.

New Assumption: The input for *Pizza Topping Code List* is the list *{Spinach,Spinach,Tomatoes}*.

The row now evaluates to true since the value *Spinach* appears twice in both the operand value and in the Condition Fact Type list, the value *Tomatoes* appears once in both, and neither contains any extraneous values that are not in the other.

4.3.5.9 Best Practices for List Fact Type Operators

When dealing with the List Fact Type operators, less is usually more. There are 14 operators (not including the case-sensitive variations). The operators have similar names and functionality, and it is difficult to remember exactly what each operator does. Therefore, it is best to minimize the set of List Fact Type operators used within a single Rule Family, ideally down to only two – one operator and its negative. This will assist with understandability and testability.

A common pattern with List Fact Type conditions is to test whether the Fact Type contains some values but does not contain others. This will require two columns with the same Condition Fact Type. For readability, it is best to do all the *Contains* tests in one column and all the *Does Not Contain* tests in the second column, as shown below:

Customer Priority Code List	Customer Priority Code List	Customer Priority Code
Contains Any : {High}		Is : High
Contains Any : {Medium}	Does Not Contain Any : {High}	Is : Medium
	Does Not Contain Any : {High,Medium}	Is : Low

4.3.6 Case Sensitivity

By default, the operators for textual Fact Types are case-sensitive. However, all the operators used with textual Fact Types also have a case-insensitive variant. These are identified by an (IC) suffix, which stands for Ignore Case. When should these be used?

Use of the (IC) variants adds extra processing to convert both the Condition Fact Type value and the operand value to all uppercase before comparing them. This has a small performance impact for each processed (IC) operator. This impact adds up when done repeatedly for the same Condition Fact Type value (e.g., when the Condition Fact Type appears in multiple Rule Families in a Decision or Flow) or for the same operand value (e.g., when the Rule Family is attached to a BIM Entity with many instances).

For best performance, convert each value at most once. This would typically mean normalizing the input Fact Type values prior to invoking the Decision or Flow and if necessary, using a Model Mapping to convert any technical values to properly cased business-friendly values. The decision models would then use the case-sensitive operators, improving readability of the models.

4.4 Negation

TDM principles require Rule Families to cover all condition Fact Type domain values that are within scope and, in particular, to cover all supporting Rule Family conclusion values. This can be accomplished by explicitly specifying all the relevant operand values. Alternatively, negation can be used.

Consider the following Rule Family:

Customer Priority Code	Customer Program X Eligibility Indicator
Is ⋮ High	Is ⋮ Eligible
???	Is ⋮ Ineligible

If the Allowed Values for *Customer Priority Code* are *High*, *Medium*, and *Low*, how should we replace the question marks? One could use *Is In {Medium,Low}* or alternatively *Is Not High*. These two options are similar, but not identical, as they yield different results for values outside the domain of the Fact Type: the *Is In {Medium,Low}* option will be false in this case while the *Is Not High* option will be true. If we assume that we are dealing only with trusted data (see section 4.14 *Trusted Data* below), the two options will yield the same results. Which option is preferred?

As a best practice, in most cases it is recommended to use the *Is Not High* paradigm for easier maintainability. The justification for this is the analysis work that needs to be done for changes is essentially the same for both paradigms. However, consider a scenario where an additional *Ineligible* value is added. In the above example, we would need to change the *Is In {Medium,Low}* cell while we would not need to change the *Is Not High* cell. So, using the *Is Not* paradigm could reduce the number of Rule Families that need to be changed when adding an Allowed Value.

This approach also works in cases where explicitly listing all the values is not feasible. One scenario in which this occurs is when the list of domain values for the Fact Type within DM is intentionally incomplete or omitted altogether, either due to the size of the list or its dynamic nature. In the latter case, the Fact Type may be defined as a *Dynamic Set* (see section 3.5.1 *Dynamic Set* above) and completeness validation checks are disabled for the column. Since the values presented at runtime may include values that were not known when the Rule Family was modeled, the negation option (i.e., *Is Not High*) must be used.

An exception to the above best practice of using the *Is Not High* paradigm is Indicators since these can have only two values. For an Indicator Fact Type, it is recommended to use *Is* for both the positive (e.g., *Is Eligible*) and the negative (e.g., *Is Ineligible*) rows. This results in easily understandable business logic. As an added benefit, there won't be any double negatives, even if some of the values do not follow best practice (see section 3.4.2 *Boolean Data Types* above) and begin with the word "Not".

The best practices presented in this section apply to all Rule Families, but in particular to row-diagonal Rule Families (see section 4.8.1.1 *Row-Diagonal Pattern* below) and row priority Rule Families (see section 4.8.1.2 *Row Priority Pattern* below). The *Create Diagonal* functionality in DM implements these best practices.

4.5 Formulas

Similar to cells in spreadsheets (such as Microsoft Excel), Rule Family cells can contain formulas comprised of basic arithmetic operations and functions, both built-in and user-defined (see chapter 15 *User-Defined Functions (UDFs)* below).

Consider a Rule Family such as the following:

Customer Available Monthly Funds Amount	Customer Loan Eligibility Indicator
Is Less Than Or Equal To ⁝ *formula*	Is ⁝ Ineligible
Is Greater Than ⁝ *formula*	Is ⁝ Eligible

where *formula* contains the actual formula being used. There are several downsides to this approach. DM cannot fully validate Rule Families with formulas since that requires execution to determine the results of the formula execution. You will see a validation warning about "partially overlapping rows". There is a risk that the formula used in both cells is inadvertently different in which case the Rule Family could yield incorrect results and, depending on the complexity of the formula, could be very difficult to spot. There is also a performance impact: at run-time, each row of the Rule Family will be executed, meaning that the formula will be invoked multiple times with the same data. Lastly, there is a maintainability aspect: if the formula needs to be changed, it will need to be changed everywhere it appears.

A better approach is shown below:

Customer Available Monthly Funds Amount	Customer Loan Eligibility Indicator
Is Less Than Or Equal To ⁝ Monthly Payment Amount [Calculated]	Is ⁝ Ineligible
Is Greater Than ⁝ Monthly Payment Amount [Calculated]	Is ⁝ Eligible

Monthly Payment Amount [Calculated]
Is ⋮ *formula*

This approach addresses all the downsides of the first approach and adds another advantage – the supporting Rule Family is reusable and can be reused anywhere the formula is required (assuming the same Fact Types are being used).

For anything other than a trivial formula, the best practice is to use the second approach. For our purposes, we'll define a trivial formula as one that contains no functions and no more than one operator.

4.6 Lookup Tables

It is often desired to perform a lookup from within a Rule Family at run-time. Perhaps the most common scenario in which this arises is the retrieval of threshold values (e.g., annual retirement plan contribution limits). While small lookup tables could be implemented as supporting Rule Families, a change to a Rule Family requires a code deployment, extensive testing, and working through all the controls organizations have put in place to govern such code deployments. It is usually easier and faster to roll out data table updates without a code deployment and this can frequently be done by the business independently. So, the preference is to manage lookup tables in a data store (such as a database) and pass the data to the decision logic.

There are two approaches to performing a lookup with Sapiens Decision:

- Performing the lookup externally prior to invoking the Decision or Flow
- Using a User-Defined Function (also referred to as a UDF, see chapter 15 *User-Defined Functions (UDFs*) below) as part of the processing of a Rule Family

In some cases, the pre-invocation option is not feasible since the value being looked up is determined within the Decision/Flow itself. You will need to split the Decision/Flow into two (or more) parts, determine the values necessary for the lookups in the first part, perform the lookups in between the two parts, and pass the results of the lookups as inputs to the second part. External orchestration will be required.

A User-Defined Function is often the preferred approach, although it could have significant performance implications, particularly if the function is relatively expensive from a resource point of view. Also, keep in mind that the function must handle exceptions such as when the data store is not available.

One concern with User-Defined Functions is that they could make the decision models *non-deterministic* (see section 4.16 *Modeling for Deterministic Execution* below).

As a best practice, the number of lookups done within a Decision should be minimized. Use unconditional supporting Rule Families as shown in section 4.5 *Formulas* above to ensure that a lookup is executed only once for each Decision invocation.

Similarly, for a Flow, you will also want to minimize the number of lookups. Instead of performing the same lookup in multiple Decisions, do it once and store the result in an interim Fact Type which you refer to later in the Flow as needed.

The Sapiens Decision Suite includes the Parameter Management (PaM) module (see chapter 17 *Minimized Input Optimization (*MIO) API

The MIO API is an alternative execution algorithm for Decisions. It provides an API for an interactive user interface to request the next question(s) – each representing an input Fact Type – to be displayed to the user. The MIO API selects which question(s) to ask next to provide the optimal user experience, that is, ask the user the fewest possible number of questions to reach a Decision conclusion. It does this by examining the declarative business logic expressed in a Decision and determining which question(s), given particular responses to each, would yield a Decision conclusion. If the responses received are not the ones that yield a conclusion right away, the process iterates using all the received inputs from previous iterations to determine the next set of questions to ask.

The MIO API enables automated UI generation, replacing scripted questionnaire logic with dynamic scripting based on a Decision. It is a great fit for interactive questionnaires such as insurance claims, loan applications, product selection, etc.

4.7 Using the MIO API with a Decision

When executing a Decision with the MIO API, the algorithm attempts to execute the Decision with the provided data, if any. Rule Family rows are skipped if data was not provided for one or more of its Fact Types. This differs from the standard execution algorithm which always executes all rows and produces all applicable messages. As a result, you may get fewer messages when executing with the MIO API than with the standard execution algorithm.

If a conclusion is not reachable with the provided data, the MIO API algorithm examines each row of the Decision Rule Family to see how many additional Fact Types are required to be able to execute that row and reach a conclusion, assuming values that enable that row to fire are provided. If any of the identified Fact Types have supporting Rule Families, they are not counted. Instead, the algorithm recurses into the supporting Rule Family, looking at each row, counting required Fact Types for each, and recursing into additional supporting Rule Families as needed. Within each Rule Family, the algorithm selects the row that requires the fewest additional Fact Types (including those from the supporting Rule Families). In case of a tie, the row with the lower sequence number – per the saved Perspective at deployment time – is selected. The set of additional Fact Types (including those from the supporting Rule Families) for the selected Decision Rule Family row is then provided as the output for this MIO API execution.

The process iterates by invoking the MIO API again, with a new set of data, ideally including the additional Fact Types requested. Note that the algorithm is stateless and will run with any set of input data – it knows only what was provided to it within the current execution.

As a best practice, model your Decisions as you normally would. Let the Decision logic drive the order of the questions, rather than the other way around. This will result in easier to understand Decisions.

To refine the order of the questions, you have the following options:

- Where there are ties, the order of the questions can be modified by changing the order of the rows in the various Rule Families so that rows to be fired first are above other rows in the Rule Family, saving the Perspective for each Rule Family, and regenerating the deployed code.
- Where there are certain questions that must be asked first, separate them into their own Decision, and use a Flow with a gateway to ensure that they are answered first, before proceeding. See below for more information about using the MIO API with a Flow.

Decisions with child BIM Entities are not currently supported with the MIO API.

4.8 Using the MIO API with a Flow

The MIO API optimizes individual Decisions as described above. When used with a Flow, the MIO API is applied to each Decision independently. It does not optimize the inputs for the Flow as a whole, nor does it request values for gateway Fact Types.

The MIO API algorithm processes the Decision Tasks within the Flow sequentially until it encounters a gateway where it cannot determine the branch to be taken or it reaches the end of the Flow. The required Fact Types to request next are provided for each processed Decision separately. These should be consolidated by the calling application. Note that some Decisions might have conclusion values while others still request additional data, and some might not have been evaluated at all because the Flow execution stopped at a gateway.

Since the Flow is not optimized as a whole, the best practice is to ensure that the Flow requests Fact Types from only one Decision at a time. This is achieved by placing a gateway after each Decision, with the conclusion Fact Type of that Decision as the gateway Fact Type. The Flow execution will not proceed unless that Decision resulted in a conclusion value. The MIO API will list the required Fact Types for just that Decision, as any prior Decisions will already have conclusion values.

4.9 UI Approaches with the MIO API

A UI built with the MIO API needs to translate the Fact Type names and values expected by Decision into questions and answers. This is typically done through lookup tables (or similar means) to enable speed and ease of change and to easily support multiple languages.

The following approaches are commonly used to interact with the MIO API:

1. Display only the question(s) currently requested by the MIO API

   This is the simplest approach, taking the user down a specific path driven by the MIO API. If the user wants to change a prior response, they either need to back up (which may result in different questions being asked than what was asked previously) or start over.

2. Display the cumulative list of questions requested by MIO API

   The currently requested question(s) are appended to the list of questions already asked. This allows a user to change previous responses, which may result in different questions being asked than what was asked previously.

3. Display all questions, highlighting the questions currently requested by the MIO API

   This approach allows a sophisticated user to answer the questions in any order and to easily change previous responses. It is better suited for an internal user, such as an agent, rather than an end-user.

The same Decision or Flow can be used with any combination of the above approaches simultaneously, allowing you to mix and match as needed to support different UI needs. Which approach works best depends on your requirements.

4.10 Omitted Fact Types

Decision's standard execution algorithms handle omitted Fact Types, null values, and empty values as NotPopulated. The MIO API handles omitted Fact Types as questions to be asked. If you want to pass a null or empty value, you must do so explicitly by including the Fact Type in the input message.

4.11 Testing

Testing with the MIO API is not supported currently in DM. Testing with the MIO API must be done externally. As a best practice, deploy your assets as candidates (see section 7.6 *Candidate Deployment* above) for external testing with the MIO API prior to approval.

Parameter Management (PaM) below) to provide a user interface for managing parameter tables externally from DM and provide the ability to perform lookups against them at runtime via a lookup function, which as of this writing, is implemented as a Sapiens-provided *LOOKUP* UDF.

When using PaM's *LOOKUP* function, the return data type is *Text*. Use a *VALUE* function to convert the lookup result to a numeric format. Note that the *VALUE* function will result in an exception if the *LOOKUP* function returns null. As a best practice, make sure the *LOOKUP* always returns a value. Alternatively, use the *VALUE* function with a default value, e.g., VALUE(LOOKUP(….),0), but note that this requires DM version 7.5 or later.

4.12 Dealing With Null Values

It is often necessary to deal with null values in a decision model. If done properly, the resulting decision model is technology-independent and will run in any technology. However, if this is done improperly, either the testing facility within DM or the execution environment, or both, may fail to execute the decision model or may produce unexpected results. The best practice is to test Rule Families with all reasonable input values, including null values if appropriate.

Problems can arise when a value of null is compared to another value. The results of this sort of comparison are unpredictable across execution technologies and may even vary for different data types, working for some, and causing runtime exceptions for others. Fortunately, Decision's standard operators are null-safe—meaning that null input values do not cause runtime exceptions—when used with Decision's standard execution adapters (the testing facility within DM, DE, or code generated via the Java Format Adapter). If you are using a different adapter to generate your own executable code, you may encounter this issue and may need to refactor the Rule Family to avoid comparing a condition

column with a value of null to another value. This can be accomplished by setting an interim Fact Type to a specific "flag" value (such as 0 or -1) when the original Fact Type is null and then using the interim Fact Type instead of the actual Fact Type in the original Rule Family.

Another problematic area is null values within formulas. While DE and the testing facility in DM generate null-safe code for formulas, the Java Format Adapter may not. See section 4.12.2 *Null-Safe Execution* below for a detailed discussion.

There are some recurring scenarios where null values may occur and must be explicitly dealt with. These include data acceptance logic (e.g., testing for conditionally required values), use of the Else Pattern (see section 4.13.2.1 *Else Pattern* below), and BIM Entities with zero instances.

Sapiens Decision has several features to assist with the handling of null values:

- Populated Characteristic columns
- Null-safe execution
- Null message Fact Type setting
- Setting a Fact Type to a null value

4.12.1 Populated Characteristic Columns

To test a Fact Type for a null value, use a separate condition column, and toggle it to be a *Populated Characteristic* column. The Populated Characteristic check treats both null values and empty values as *NotPopulated*.

The following example shows a violation of TDM normalization principles that is frequently seen in Rule Families with Populated Characteristic columns:

Tier Code	Test Score PC	Test Score	Category Code
Is ⁝ Tier 1	Is ⁝ Populated	Is Greater Than or Equal To ⁝ 70	Is ⁝ Category A
Is ⁝ Tier 1	Is ⁝ Populated	Is Less Than ⁝ 70	Is ⁝ Category B
Is ⁝ Tier 1	Is ⁝ NotPopulated		Is ⁝ Category B
Is ⁝ Tier 2	Is ⁝ Populated	Is Greater Than or Equal To ⁝ 70	Is ⁝ Category C
Is ⁝ Tier 2	Is ⁝ Populated	Is Less Than ⁝ 70	Is ⁝ Category D
Is ⁝ Tier 2	Is ⁝ NotPopulated		Is ⁝ Category D
Is Not In ⁝ {Tier 1,Tier 2}			Is ⁝ Category E

In this example, the *Test Score*, if provided, is being compared against a threshold value. The normalization issue is that both *Test Score* columns together represent a hidden conclusion, a Pass/Fail

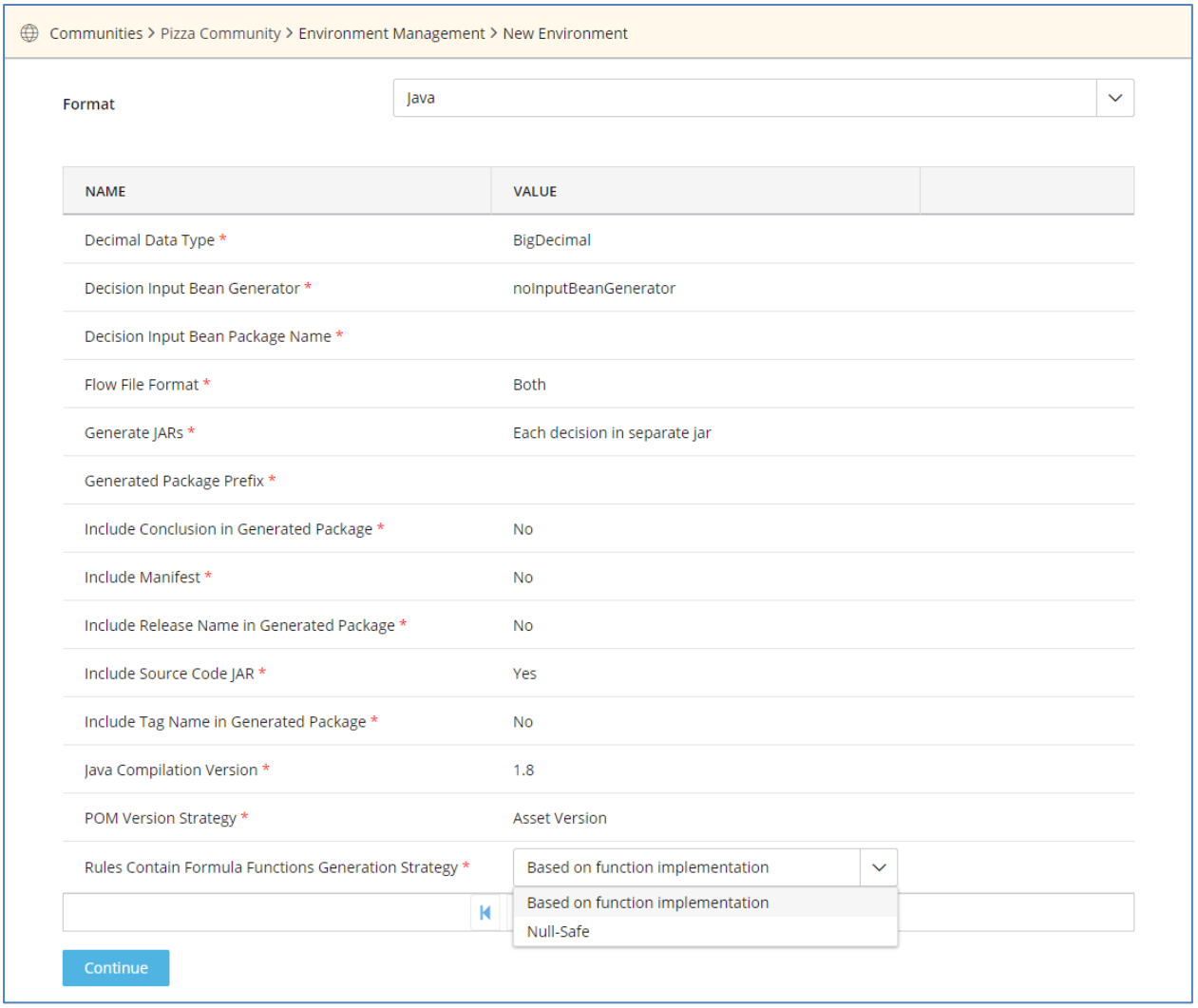
finding. As is typical with normalization issues, we can resolve this by adding a supporting Rule Family as shown below:

Tier Code	Test Assessment Indicator	Category Code
Is ⁝ Tier 1	Is ⁝ Pass	Is ⁝ Category A
Is ⁝ Tier 1	Is ⁝ Fail	Is ⁝ Category B
Is ⁝ Tier 2	Is ⁝ Pass	Is ⁝ Category C
Is ⁝ Tier 2	Is ⁝ Fail	Is ⁝ Category D
Is Not In ⁝ {Tier 1,Tier 2}		Is ⁝ Category E

Test Score PC	Test Score	Test Assessment Indicator
Is ⁝ Populated	Is Greater Than or Equal To ⁝ 70	Is ⁝ Pass
Is ⁝ Populated	Is Less Than ⁝ 70	Is ⁝ Fail
Is ⁝ NotPopulated		Is ⁝ Fail

4.12.2 Null-Safe Execution

When using Decision's standard execution adapters, the code generated for Decision's operators is null-safe, meaning that it will handle nulls gracefully. DE and the testing facility in DM also generate null-safe code for formulas. However, the same may not be true for formulas generated via the Java Format Adapter. To generate executable Java code that implements null-safe behavior, select *Null-Safe* in the "Rules Contain Formula Functions Generation Strategy" Java Format Adapter options:



When *Null-Safe* is selected, any Fact Types participating in a formula (in either a condition cell or the conclusion cell) are checked for null prior to execution of the formula. If any are found to be null, the formula execution is bypassed, and the entire row is skipped.

The default setting of *Based on function implementation* attempts to execute all formulas and could result in null pointer exceptions if passed null values. This means that you may get different results when running in Java compared to what was tested within DM, unless you change the default setting.

The best practice is to use the *Null-Safe* setting to match the way the testing facility runs within DM.

One common misconception is that the use of *Is Populated* is sufficient to guarantee proper execution. This is not the case because the order of the columns as displayed in the Rule Family does not control the order in which the logic evaluations are generated. In other words, there is no way to force the *Is Populated* test to execute before the use of the Fact Type. The result could be a Null Pointer Exception when used with the *Based on function implementation* setting.

Consider the following example:

Customer Non-Recurring Funds Amount PC	Loan Amount	
Is ⋮ Populated	Is Less Than or Equal To ⋮	Customer Available Monthly Funds Amount + Customer Non-Recurring Funds Amount
Is ⋮ Populated	Is Greater Than ⋮	Customer Available Monthly Funds Amount + Customer Non-Recurring Funds Amount
Is ⋮ NotPopulated	Is Less Than or Equal To ⋮	Customer Available Monthly Funds Amount
Is ⋮ NotPopulated	Is Greater Than ⋮	Customer Available Monthly Funds Amount

In the two rows that check for *Is Populated*, what would happen if the Loan Amount column was processed first? If *Customer Non-Recurring Funds Amount* is null, then the formulas would be trying to add a null value to an amount. Depending on how this was implemented in the execution technology, this could result in an exception. The *Null-Safe* setting in the Java Format Adapter averts this exception. DE and the testing facility in DM always generate null-safe code.

Fortunately, the above example is easy to correct so that it always processes correctly. Use a supporting Rule Family like the one below to set *Customer Non-Recurring Funds Amount [Calculated]* to 0 when *Customer Non-Recurring Funds Amount* is null, or to the value of *Customer Non-Recurring Funds Amount* otherwise. Then use *Customer Non-Recurring Funds Amount [Calculated]* in the calculation instead of *Customer Non-Recurring Funds Amount*.

Customer Non-Recurring Funds Amount PC	Customer Non-Recurring Funds Amount [Calculated]
Is ⋮ Populated	Is ⋮ Customer Non-Recurring Funds Amount
Is ⋮ NotPopulated	Is ⋮ 0

4.12.3 Handling of Null Message Fact Type Values

DM has a System Option called “Fact Type null in Message Element placeholder” to establish the handling of null values in Message Fact Types.

The default for this setting is “null”, which means the message will display “null” in place of the value if the value is null.

The System Option may be set to any other string (including empty). At execution time, if there is a null Message Fact Type value, the system substitutes the value specified for the System Option for the null value.

In addition, the variable %Fact Type Name% can be placed in the System Option to include the name of the Fact Type in the substitution.

For example,

Rule Family Message Text: "The Loan " + %Loan Id% + " is ineligible."

Fact Type null in Message Element placeholder: <Unknown %Fact Type Name%>

If the Loan ID is null at execution, the Rule Family Message text will be:

"The Loan <Unknown LoanId> is ineligible."

4.12.4 Setting a Fact Type to a Null Value

Although not recommended, it is sometimes necessary to set a conclusion value to a null value. This is done by using the NO_VALUE() built-in function with either the *Is* or *Is Appended With* operator as appropriate. The best practice would be to use a value that has meaning to the business, rather than a null value. This will improve the readability and understandability of the decision model.

Note that the NO_VALUE() function should be used only in conclusion cells, not in condition cells. To test whether a Fact Type is null, use a Populated Characteristic column.

Also, make sure to test for null conclusion in the supported Rule Family by using a Populated Characteristic column.

4.13 Rule Family Patterns

In this section we will explore some frequently occurring Rule Family patterns to understand when to use each. We'll start with patterns involving only one Rule Family as these tend to be simple and easy to understand. We'll then look at patterns involving multiple Rule Families.

4.13.1 Singe Rule Family Patterns

4.13.1.1 Row-Diagonal Pattern

The Row-Diagonal Pattern is the most common and simplest Rule Family pattern. Row-diagonal Rule Families are common in eligibility and validity determinations, such as data acceptance. This pattern is a good fit anywhere there is just one positive rule to evaluate, and additional rows are needed for completeness per the TDM principles.

The pattern name comes from the shape in which it most frequently appears: one row with all *n* condition columns populated and *n* rows, each with a different condition populated, arranged as a diagonal for visual clarity. The diagonal visual pattern is just a visual aid; because of TDM's declarative principles, the order of the columns and the order of the rows does not matter.

The Row-Diagonal Pattern is a direct result of De Morgan's law in Boolean algebra: $\neg(A \wedge B) \Leftrightarrow \neg B \vee \neg A$. In plain English, the opposite of A AND B is (NOT A) OR (NOT B). In a Rule Family, A AND B is expressed as a full row with 2 condition columns. For completeness we also need the inverse of the row, which per

De Morgan's law, is an OR expression with the inverse conditions. In a Rule Family, OR's are expressed as separate rows—the diagonal.

Structurally, a row-diagonal Rule Family has the following characteristics:

1. *n* condition columns
2. *n* + 1 rows
3. Conclusion Fact Type with a data type of *Indicator*. We refer to its 2 values as C1 and C2.
4. A row in which every condition cell is populated (i.e., has a logic condition) and the conclusion value is C1. We refer to this row as the *full row*.
5. *n* rows, arranged as a diagonal for visual clarity. Each row has a different condition column populated, the condition is the inverse condition of the one in the full row, and the conclusion value is C2.

The following example shows a row-diagonal Rule Family:

Policy Pricing Within Bounds Indicator	Policy Underwriting Risk Score	Policy Type Code	Policy Renewal Method Indicator	Informational
Is ⋮ Within Bounds	Is Less Than or Equal To ⋮ 9	Is In ⋮ {Commercial,Life}	Is ⋮ Automatic Renewal Process	
Is ⋮ Out Of Bounds			Is ⋮ Manual Renewal Process	"Policy Pricing out of bounds"
	Is Greater Than ⋮ 9		Is ⋮ Manual Renewal Process	"Policy Underwriting Risk is unacceptable"
		Is Not In ⋮ {Commercial,Life}	Is ⋮ Manual Renewal Process	"Automatic renewal is not available for the Policy Type"

A key feature of row-diagonal Rule-Families is that they are multi-hit. In other words, multiple rows can fire—if they share the same conclusion value. This is to return all the messages related to the particular conclusion value. In the above example, if *Policy Pricing Within Bounds Indicator* is *Out Of Bounds*, *Policy Underwriting Risk Score* is *11*, and *Policy Type Code* is *Auto*, the result would include all 3 messages. If you need to prioritize the messages and produce just one, you will need to use the row priority Rule Family pattern (see section 4.13.1.2 *Row Priority Pattern* below).

Row-diagonal Rule Families are so common that DM has a built-in feature to create the diagonal. However tempting it is to use this hammer, not every Rule Family is a nail.

Some consideration should be given to what to use for the inverse conditions in the diagonal. Consider a Fact Type with the following allowed values: A, B, C, D. If the positive condition is *Is In {A,B}*, what should the inverse condition be? There are 2 options:

- *Is In {C,D}*
- *Is Not In {A,B}*

While both yield the same result for values within the allowed values, they yield different results for values outside the allowed values (including nulls). If we assume the data is trusted (see section 4.19 *Trusted Data* below), then this should not be a concern.

What happens when we add another allowed value, say E, to the domain of the Fact Type? All Rule Families in which the Fact Type participates should be reviewed for impact. Let's assume that in our Rule Family, the E value should be part of the inverse condition. If we use the *Is In* paradigm, we will have to update the Rule Family. If we use the *Is Not In* approach, the Rule Family will not need to be changed, saving effort.

Therefore, as a best practice, it is generally recommended to use the *Is Not In* approach to inverse conditions for lists of values. One notable exception is Indicator Fact Types since these have only two values. This is how the *Create Diagonal* feature is implemented in DM. The sample Rule Family above demonstrates the best practice and the exception.

4.13.1.2 Row Priority Pattern

The Row Priority Pattern is a commonly used single hit pattern. The Rule Family logic is expressed in such a way that only one row can ever fire. It is typically used to prioritize conclusion values or messages when multiple values or messages apply.

For visual clarity, a row priority Rule Family is arranged as a full row (either first or last) and the prioritized rows in descending priority as a wedge consisting of the diagonal (same as in the Row-Diagonal Pattern) and all the cells under the diagonal. The visual pattern is just a visual aid; because of TDM's declarative principles, the order of the columns and the order of the rows does not matter.

Structurally, a row priority Rule Family is similar to a row-diagonal Rule Family and has the following characteristics:

1. *n* condition columns
2. *n* + 1 rows
3. A row in which every condition cell is populated (i.e., has a logic condition). We refer to this row as the *full row*.
4. *n* rows, arranged in descending priority for visual clarity
    a. The conditions on the diagonal have the inverse condition of the one in the full row
    b. The higher priority conditions (left of the diagonal) have the same condition as the one in the full row

Due to the similarities with the Row-Diagonal Pattern, the same best practice in regard to using the *Is Not In* approach to inverse conditions for lists of values applies here as well.

Like the Row-Diagonal Pattern, the Row Priority Pattern is simple and easy to follow.

One disadvantage of the Row Priority Pattern is the effort required to reprioritize or add a new item with a priority in the middle. The effort increases as the table grows.

4.13.1.2.1 Row Priority Pattern – Messages

When used to prioritize messages, a row priority Rule Family is similar to a row-diagonal Rule Family. The key differences are:

- The cells to the left of the diagonal are populated with the same condition as the one in the full row – this is what forces the single hit
- At least one of the *n* rows has a message – otherwise should use the row-diagonal pattern to reduce visual clutter

The following example shows a row priority Rule Family that prioritizes messages:

Policy Pricing Within Bounds Indicator	Policy Underwriting Risk Score	Policy Type Code	Policy Renewal Method Indicator	Informational
Is ⋮ Within Bounds	Is Less Than or Equal To ⋮ 9	Is In ⋮ {Commercial,Life}	Is ⋮ Automatic Renewal Process	
Is ⋮ Out Of Bounds			Is ⋮ Manual Renewal Process	"Policy Pricing out of bounds"
Is ⋮ Within Bounds	Is Greater Than ⋮ 9		Is ⋮ Manual Renewal Process	"Policy Underwriting Risk is unacceptable"
Is ⋮ Within Bounds	Is Less Than or Equal To ⋮ 9	Is Not In ⋮ {Commercial,Life}	Is ⋮ Manual Renewal Process	"Automatic renewal is not available for the Policy Type"

Regardless of the valid input values provided, only one row will fire in the above Rule Family, and at most one message will be returned. The messages are prioritized so only the highest priority one is returned. Consider, for example, the message in the last row. It will be returned only if the other rows do not apply, effectively making it the lowest priority.

This pattern is commonly used in data acceptance logic (see 4.20 *Modeling for Untrusted Data* below).

4.13.1.2.2 Row Priority Pattern – Conclusion Values

The following example shows a row priority Rule Family that prioritizes the conclusion value. Assume a person wants a vegan pizza, but if one is unavailable, they would prefer vegetarian. And if that too is unavailable, they will have a pizza with meat.

Pizza Vegan Type Availability Indicator	Pizza Vegetarian Type Availability Indicator	Pizza Meat Type Availability Indicator	Person Prioritized Pizza Dietary Type Code
Is ⁝ Available			Is ⁝ Vegan
Is ⁝ Unavailable	Is ⁝ Available		Is ⁝ Vegetarian
Is ⁝ Unavailable	Is ⁝ Unavailable	Is ⁝ Available	Is ⁝ Meat
Is ⁝ Unavailable	Is ⁝ Unavailable	Is ⁝ Unavailable	Is ⁝ NA

Regardless of the valid input values provided, only one row will fire in the above Rule Family, correctly prioritizing the pizza selection per the person's preferences. For example, a vegetarian pizza will be selected only if a vegan pizza is not available.

4.13.1.3 List Priority Pattern

Another common prioritization pattern evaluates a List Fact Type, testing for values of interest, in priority order. The list may come from anywhere – from a supporting Rule Family, from an earlier Decision in a Flow, or provided as input. This pattern is frequently used in the processing of BIM Entities (see section 4.18 *Business Information Model (BIM)* below).

A list priority Rule Family has the following characteristics:

1. Two condition columns, which per best practice, we'll use as a *Contains* column and a *Does Not Contain Any* column
2. *n* rows, arranged in descending priority for visual clarity, where row *i*
    a. Tests that the list contains the item in *i*-th priority
    b. Test that the list does not contain any items higher than *i*-th priority

The visual pattern is just an aid; because of TDM's declarative principles, the order of the columns and the order of the rows does not matter.

The following Rule Family shows a list priority solution to the pizza dietary requirements presented in the previous section:

Pizza Available Dietary Type Code List	Pizza Available Dietary Type Code List	Person Prioritized Pizza Dietary Type Code
Contains : {Vegan}		Is : Vegan
Contains : {Vegetarian}	Does Not Contain Any : {Vegan}	Is : Vegetarian
Contains : {Meat}	Does Not Contain Any : {Vegan,Vegetarian}	Is : Meat
	Does Not Contain Any : {Vegan,Vegetarian,Meat}	Is : NA

Comparing the List Priority Pattern with the Row Priority Pattern, both have simple to follow visual patterns. However, the List Priority Pattern has only two condition columns, perhaps making it easier to scale than the Row Priority Pattern. Like the Row Priority Pattern, the List Priority Pattern requires effort to reprioritize or add a new item with a priority in the middle. The effort increases as the table grows.

The List Priority Pattern may become difficult to read as the list of values in the *Does Not Contain Any* cells grows. It may also become increasingly difficult to ensure the correctness of these lists of values.

4.13.2 Multi-Rule Family Patterns

4.13.2.1 Else Pattern

TDM principles require Rule Families to be complete, that is, a Rule Family is required to produce a conclusion value for all combinations of condition Fact Type domain values that are within scope. However, it may be impossible or impractical to do this in large or complex Rule Families. Even if it was successfully done for a Rule Family table when initially authored, subsequent additions/changes may require remodeling the negative rows, a task that becomes increasingly more difficult as the positive logic gets more complex.

A practical solution to this challenge is to use two Rule Families. The supporting Rule Family is left intentionally incomplete, dealing only with the positive logic. The supported Rule Family, then, “traps the resulting null” and returns a negative (or default) conclusion value. As an inseparable unit, the two Rule Families satisfy the TDM principles and return a conclusion value for all combinations of condition Fact Type domain values that are within scope.

Consider the different combinations of toppings, cheese, and crust that make for an *Acceptable* pizza and those that make for an *Unacceptable* pizza. Rather than list out all the combinations that are *Unacceptable*, you can more simply maintain only the logic that makes for *Acceptable* combinations.

The supported Rule Family will look like this:

Pizza Acceptability Indicator [Interim] PC	Pizza Acceptability Indicator
Is : Populated	Is : Pizza Acceptability Indicator [Interim]
Is : NotPopulated	Is : Unacceptable

As a best practice, the Else Pattern should be used only when Rule Families cannot practically be made complete. In other words, the Else Pattern should be the exception, not the rule.

4.13.2.2 List Scoring Pattern

The List Scoring Pattern is another prioritization pattern. Like the List Priority Pattern, the List Scoring Pattern evaluates a List Fact Type, testing for values of interest, in priority order. The list may come from anywhere. It requires less effort than the Row Priority and List Priority Patterns to reprioritize or add a new item with a priority in the middle. The required effort is constant even as the table grows.

The idea behind the List Scoring Pattern is to create another list containing a numerical score for each item in the original list and then look for lowest score. This requires three Rule Families:

1. Bottom Rule Family: creates the score list
2. Middle Rule Family: selects the best (minimum) score in the score list
3. Top Rule Family: sets the conclusion value based on the best score

The scores represent the priority order. As a best practice, it is recommended to leave plenty of space between the scores for additions and changes.

The following example shows a list scoring solution to the pizza dietary requirements presented in section 4.13.1.3 *List Priority Pattern* above.

Pizza Preference Priority Score	Person Prioritized Pizza Dietary Type Code
Is : 100	Is : Vegan
Is : 200	Is : Vegetarian
Is : 300	Is : Meat
Is : 999	Is : NA

Pizza Preference Priority Score
Is : MIN(Pizza Preference Priority Score List)

Pizza Available Dietary Type Code List	Pizza Preference Priority Score List
Contains : {Vegan}	Is Appended With : 100
Contains : {Vegetarian}	Is Appended With : 200
Contains : {Meat}	Is Appended With : 300
Does Not Contain Any : {Vegan,Vegetarian,Meat}	Is Appended With : 999

As shown in the example above, each row in the List Scoring Pattern is simple – deals with only one item. This makes it very easy to understand the prioritization and to add new items or change the prioritization of existing items.

One challenge with the List Scoring Pattern is that every change must be done in two places, both the top and the bottom Rule Families. This may feel redundant. Furthermore, these changes need to be kept synchronized – this could be error-prone.

The List Scoring Patten is a great fit for frequently changing priorities. The benefits of this pattern make it worth the hassle of having to change in two places.

4.14 Row and Column Order

While row and column order are ignored for execution (per the declarative nature of TDM), the row and column order do impact readability and understandability of a Rule Family. The optimal row and column order for analysis may be different than the optimal order for business consumption.

In general, for business consumability, the positive row(s) – the row(s) that represent the rule being modeled in the Rule Family, sometimes referred to as the "happy path" – should come first, followed by the negation rows. In many Rule Families, this has the effect of sorting the rows by the conclusion value.

Similarly, for business consumability, the column order should reflect the way the business logic is described.

4.15 Messages

Messages are a feature of Sapiens Decision that can be utilized to output additional information upon the execution of a Decision or Flow. The content of messages can be utilized for informational purposes, informing of downstream systems, analytics and metrics, and/or for auditing purposes.

4.15.1 Message Notation

Messages are comprised of the concatenation one or more elements of the following types:

Notation	Description
"Text"	Free text
%Fact Type%	Value of specified Fact Type at execution time
<Supporting Rule Family name>	Message(s) in the same Message Category returned from the specified Supporting Rule Family
(Value: condition column)	Cell operand value of the specified condition column
(Operator: condition column)	Cell operator of the specified condition column
@Message Category@	Same message as the specified Message Category (within the same Rule Family row)
#RF name#	Rule Family name, including view name
!Row ID!	row ID

The *<Supporting Rule Family name>* notation is used to "bubble up" messages in the same Message Category from supporting Rule Families. You can "bubble up" messages from any direct or indirect supporting Rule Families. This can help minimize redundant entry of message content across your Decision Models and will result in fewer changes when messages need to be edited. If there are multiple messages being "bubbled up" with this notation, they will be concatenated with semi-colons. This happens when multiple rows fire with messages in the Message Category or when messages from multiple BIM instances are being aggregated (see section 4.18 *Business Information Model (BIM)* below).

The *(Value: …)* and *(Operator: …)* notations are a great away to avoid duplicating logic within the message, by using the logic itself as the message source, grabbing the contents of the cell at the intersection of the row and specified condition column. This simplifies maintenance by providing just one place to change. Note that these notations are not usable when the Rule Family has more than one condition column for the specified Fact Type name, for example a Populated Characteristic column and a regular condition column for the same Fact Type.

The *@Message Category@* notation is used to avoid duplicating message notation when multiple categories in the same row have the same message. One Message Category is the source, and the others simply point to it. Like the *(Value: …)* and *(Operator: …)* notations, the *@Message Category@* notation simplifies maintenance by providing just one place to change.

The *#RF name#* notation exposes the full name of the Rule Family, including the view name, at execution time. This is the only way to get the Rule Family view name at execution time.

The *!Row ID!* notation has limited usefulness as this row ID information is readily available in the execution output. It can be helpful in conjunction with *<Supporting Rule Family name>* notation to "bubble up" the IDs of the rows that fired, rather than having to look through the execution trace.

4.15.2 Fact Type Values in Messages

The DM testing facility does not use Model Mappings. It shows all Fact Type values in messages (either the *%Fact Type%* or *(Value: …)* notation) using their business-friendly representations.

By default, when running outside the testing facility in DM, Fact Type values in messages are the technical values. There is an option to show the business-friendly values for particular Message Categories. See section 3.8 *Model Mappings* above for more information.

4.15.3 Message Output at Execution

There are two approaches to collecting output messages from the execution of a Decision:

- Collecting only the messages from the Decision Rule Family
- Collecting all messages from all Rule Families

The first approach provides a consistent and simplified integration pattern as only the output from the Decision Rule Family needs to be handled. The various execution adapters provide easy means to directly get the Decision Rule Family conclusion value and messages. When using DE with this approach, the settings to produce the minimal output (see section 12.1.2 *Output Message Size* below) can be leveraged to improve performance. Collecting only the messages from the Decision Rule Family provides an opportunity to use logic to select messages (see example later in this section). These benefits, however, come with the cost of the additional modeling effort required to "bubble up" all the messages to the Decision Rule Family.

The second approach requires less modeling effort; however, it could result in more complex integration as messages need to be collected from throughout the output of each Decision, a structure that varies from one Decision to the next. When using DE with this approach, the minimal output option will not work as the messages from the supporting Rule Families will be omitted. Instead, you will need to use at least the following options:

- *DecisionViewFactTypeFetchStrategy=CONCLUSION_VALUES*
- *FactTypeMessageFetchStrategy=ALL*

This will produce larger output messages and will worsen performance. Lastly, when collecting all messages, there is limited opportunity to inject logic to select the messages to be externalized.

As an example of how logic can be used to select messages, consider the following (incomplete) *Pizza Acceptability Indicator* logic:

Diner Dietary Type Code	Pizza Acceptability Indicator	Informational
Is ⋮ Vegan	Is ⋮ Pizza Vegan Diet Acceptability Indicator	<Pizza Vegan Diet Acceptability Indicator>
Is ⋮ Vegetarian	Is ⋮ Pizza Vegetarian Diet Acceptability Indicator	<Pizza Vegetarian Diet Acceptability Indicator>
…	…	…

Pizza Topping Code List	Pizza Vegan Diet Acceptability Indicator	Informational
Contains ⋮ {Cheese}	Is ⋮ Unacceptable	“Non-vegan cheese is not an acceptable topping in a vegan diet”
Contains ⋮ {Chicken}	Is ⋮ Unacceptable	“Chicken is not an acceptable topping in a vegan diet”
…	…	…

Pizza Topping Code List	Pizza Vegetarian Diet Acceptability Indicator	Informational
Contains ⋮ {Cheese}	Is ⋮ Acceptable	
Contains ⋮ {Chicken}	Is ⋮ Unacceptable	“Chicken is not an acceptable topping in a vegetarian diet”
…	…	…

This example is intentionally simple; it could easily be done in one Rule Family. Yet it demonstrates the technique. It scales well to handle many different diet types, which would easily overwhelm a single Rule Family in size and complexity.

We recommend modeling your Decision Models so that all relevant messages are included in the output of the Decision Rule Family. Avoid the alternate approach of outputting all messages unless your

requirements dictate otherwise. Whichever approach you select, it is best practice to use that approach across a project for consistency of modeling and interface.

4.15.4 Message Categories

Sapiens Decision supports the use of any number of Message Categories within a Rule Family. At execution time, when a row fires, all the messages for that row are produced along with the conclusion value for that row. This provides the ability to generate separate messages for any number of purposes, such as diverse audiences or multiple consuming systems, or for capturing granular metrics.

The best practice for Message Categories is to define your categories so that they are independent of one another; one Message Category should not be used to describe the content of another. Instead, the Message Category itself should convey that information.

For example, you would want to avoid setting up the following Message Categories:

- **Error Severity** – the content within this category might include values such as Severe, Warning, and Informational
- **Error Message** – the message content within this category would include the text of the Severe, Warning, or Informational message

In the above scenario, the Message Categories are interdependent; the content of *Error Severity* drives the content of *Error Message* and the content of both needs to "travel" together. The dependency complicates the definition and maintenance of the messages since the user will need to verify the contents of each Message Category and that they align with one another. As a manually entered value, the *Error Severity* could easily be mistyped, causing integration issues. When using "bubble up", the connection across the Message Categories must be manually maintained by ensuring that both Message Categories are "bubbled up" together in each supported Rule Family.

Alternatively, the best practice would be to define your Message Categories as follows:

- **Severe Message** – content would include descriptive text about the severe error
- **Warning Message** – content would include descriptive text about the warning
- **Informational Message** – content would include descriptive text for informational purposes only

In this preferred scenario, the Message Category name indicates the error severity, and each Message Category can have its own independent content. This approach simplifies the definition and maintenance of your message content since you don't need to manually maintain dependency across multiple Message Categories. Manual entry of the severity classifications is eliminated. It simplifies "bubbling up" of messages from supporting Rule Families as there is no dependency across Message Categories that needs to be maintained.

While there is no upper limit enforced on the number of Message Categories in a Rule Family, a large number of Message Categories is difficult to manage as the rows become very wide. Ideally, for ease of understanding, the logic and messages should all fit in one page. That is usually too restrictive, so as a best practice, we recommend no more than five Message Categories per Rule Family. A need for more

Message Categories (to support multiple languages, for example) is better handled by externalizing the messages as described in the section 4.15.6 *External Message Management Systems* below.

4.15.5 Message Content Guidelines

Messages should be maximally informative. In general, a message should answer the following questions:

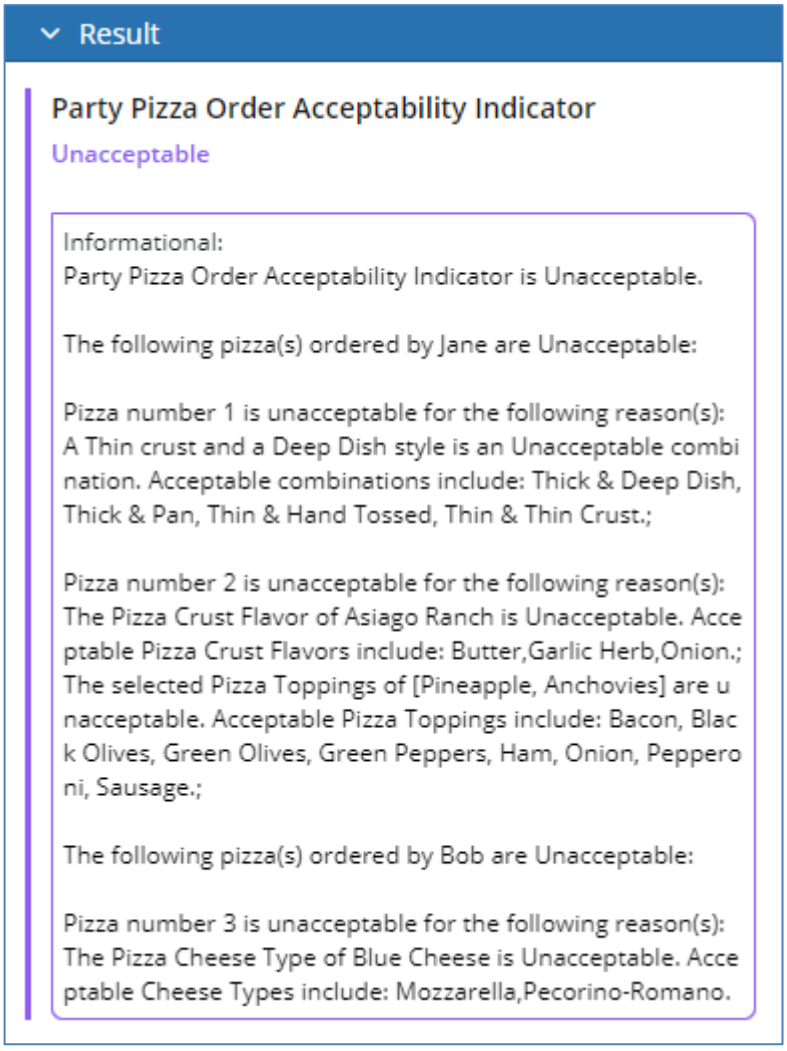
- What the values are?
- What the values should be?
- Why?

The message should include all the Fact Types applicable to a particular row of logic as well as any Fact Type values needed for identification of the impacted BIM Entity instances.

To minimize maintenance of the message content, use message elements to include Fact Type values, cell operators, and cell operand values to dynamically generate your messages from the logic of the row.

Format the messages with line breaks to improve readability of user-facing messages. This is particularly useful to separate messages from different BIM Entity instances in the "bubbled-up" message.

See the example on the right as a sample of what is possible.



4.15.6 External Message Management Systems

As an alternative to managing messages in DM, consider managing your message content in an external system. When the content is managed outside of DM, each message is assigned a unique identifier (MsgID) to be used throughout your Decision Models. This best practice has several advantages:

- It enables updating message content at any time without going through a code deployment of a new version of a Decision or Flow.
- It provides support for language-specific messages for each MsgID.
- It enables sending different message content to different target audiences using a single output from a Rule Family. For example, an insurance company may want to send one message to the

insured and an alternate message to the agent. In this case, the Decision output would include the unique MsgID, which is translated via the external message system to the relevant targeted messages.

To pass values to the external message system, name the required parameters for substitution within the external message and use key-value pairs along with a delimiter, typically the '|' character, in your message:

MsgID | parm1=val1 | parm2=val2 ….

For ease of understanding, use descriptive names for the parameters.

4.16 Flows

Flows enable the orchestration of multiple Decisions in a single service. Flows are not considered processes as they do not support external tasks, manual steps, nor iteration. Flows have a limited palette, so they cannot replace full-blown processes. A Flow is conceptually a black box: it is called with all its input, it executes Decisions in the specified sequence, then it returns to the caller with the conclusion values and messages from all the executed Decisions.

Flows are based on a small subset of BPMN consisting of only five symbols:

- Start Event
- Decision Task
- Exclusive Gateway
- Sub-Flow
- End Event

4.16.1 Flow Asset Versions

4.16.1.1 Fact Type Versions

DM allows multiple versions of a Fact Type to be used within the same Flow, in Decisions and as gateway Fact Types. However, there are some restrictions. All the Fact Type versions used in the Flow must have the same name, same single/multiple value setting, and the same underlying basic data type (such as text or numeric).

That leaves differences in allowed values as the primary driver for the use of the multiple Fact Type versions. A Flow is invoked with a single input message, so all Decisions and gateways within the Flow are exposed to the same values and each component within the Flow must accommodate the superset of allowed values. It does not make much sense to validate each Decision and the Flow itself against different sets of allowed values. The best practice, then, is to use a single Fact Type version across the Flow, in all Decisions and gateways, for consistency of data type and values.

When following the best practice, the creation of a new Fact Type version results in a new Decision version for every Decision that uses that Fact Type and a new version of the Flow to call the new Decision versions and to update any gateways that refer to that Fact Type. To reduce the effort, consider changing the Fact Type to be a dynamic set (see section 3.5.1 *Dynamic Set* above). While this will not reduce the required analysis work, it will greatly reduce the required change work when adding allowed values.

Export and import of Flows adds another complexity. Since the target Modeling Task can hold only one draft version of a Fact Type at a time, if multiple versions of the same Fact Type are being used, the overall Flow import will fail, however some assets will be imported. You will need to approve the imported assets and then import the Flow again, repeating as necessary until the Flow has been successfully imported.

4.16.1.2 Decision and Rule Family Versions

DM also allows multiple versions of a Decision or of a Rule Family to be used within the same Flow. This is common when using a Flow to select which Decision version to execute based on some criteria, often date driven (see chapter 8 *Decision Selection* below).

Notwithstanding the above, for consistency and maintainability, it is recommended to use only one version of a Decision or Rule Family in a Flow. If there is a need for multiple business logic variations, consider using views instead of versions (see section 4.2 *Views and Versions* above).

As with Fact Types, export and import of Flows with multiple versions of an asset is problematic. Since the target Modeling Task can hold only one draft version of an asset at a time, the overall Flow import will fail, however some assets will be imported. You will need to approve the imported assets and then import the Flow again, repeating as necessary until the Flow has been successfully imported.

4.16.2 Primary Conclusion for a Flow

Flows are sometimes considered to be one large decision with one conclusion value, typically that of the last Decision in the Flow, being of primary interest. Execution output does not distinguish between any Decisions, so an alternate way to identify the Decision (or more precisely, the conclusion Fact Type) of interest in the output is needed. One way to achieve this is by means of a Flow-level custom property to specify the primary conclusion Fact Type. The custom property would be externalized via the Flow's manifest file (see section 14.2 *Reading the Manifest Dynamically* below). A simpler approach is to name the Flow the same as the primary conclusion Fact Type.

4.17 Asset Size

4.17.1 Rule Family Size

A frequent question that arises during modeling is when is a Rule Family too large? The act of asking that very question is a strong indication that the Rule Family is probably already too large. While there are no firm rules for what is considered too large, we can offer some guidelines.

A Rule Family should not become too large such that it cannot be truly understood all at once. This generally means that the Rule Family should fit into a single screen (with no scrolling, other than to see any Message Categories) and usually translates into roughly 5 to 7 Condition Columns and up to 10 rows.

Short of using a higher resolution monitor, you can "squeeze" more real estate out of your existing screen by:

- Switching to full screen mode in your browser by using the appropriate browser feature (F11 in Google Chrome or Microsoft Edge)
- Adjusting the browser zoom level
- Adjusting the Rule Family zoom level. Note that editing the Rule Family is possible only when the Rule Family zoom level is set to 100%.
- Enabling Wrap Text Mode for the Rule Family (via the Rule Family menu available by clicking on the box at the top-left corner of the Rule Family) and then adjusting column widths. Users with the *RFV_PERSPECTIVE_DEFAULT_WRITE* permission can save the adjusted column widths as the default perspective in candidate and approved Rule Families.
- Hiding rows and/or columns to focus on the logic of interest

To reduce the size and complexity of a Rule Family, look for ways in which the business logic can be refactored. One common scenario is when two or more columns have values that vary in unison as compared to the other columns. This is often indicative of a "hidden conclusion" which should be normalized into a supporting Rule Family. When considering this scenario, focus solely on column combinations that have true business meaning as opposed to arbitrary sets of columns. Once the now extraneous rows and columns are removed, the resulting logic will be much simpler to follow and ensure completeness and correctness.

Since the business logic in row-diagonal Rule Families (see section 4.8.1.1 *Row-Diagonal Pattern* above) is quite simple, the above guidelines do not apply, and you will likely encounter such tables with many Condition Columns and rows.

4.17.2 Decision Size

When is a Decision too large? While there are no firm rules for what is considered too large, we can offer a guideline.

A Decision should not become too large such that it cannot be easily understood or edited. Large Decisions also take longer to load and to validate and may have a detrimental impact on runtime performance. This generally means that a Decision should have up to 30 or so Rule Families.

To reduce the size of a Decision, refactor it into two or more smaller Decisions and use a Flow or a business process to link them together. While the refactoring can technically be done anywhere within the Decision, select branches that have business meaning to become their own Decisions.

4.17.3 Flow Size

When is a Flow too large? There is no upper limit on the size of a Flow, but there are best practices as to how to structure Flows to make them more maintainable and more understandable.

Since Flows are a subset of BPMN, we'll look to Bruce Silver's BPMN style rules[1] for guidance. Per the style rules, process models should be hierarchical (nested sub-processes, each in its own diagram), and each level should fit on one page. In our context, this means that Flows should be organized using Sub-Flows and that each Flow/Sub-Flow level should have no more than 10 Decision Tasks/Sub-Flow shapes.

Note that the above sizing recommendations do not mention gateways. For gateways, the BPMN style guidelines recommend avoiding having a gateway follow another gateway. In Decision, if the gateway Fact Type in the two gateways is the same, simply use one gateway with the branches from both gateways. DM does not impose a maximum on the number of branches out of a gateway. If the gateway Fact Types are not the same, the BPMN style guidelines suggest using a Decision to combine all the branch options from the two gateways into a single gateway Fact Type, but this is likely more trouble than it is worth and is not considered a Decision best practice.

4.18 Business Information Model (BIM)

Business Information Models (BIMs) enable a Decision or a Flow to consume hierarchical data structures of arbitrary complexity.

BIM examples:

- A pizza order comprised of pizzas and their toppings
- An insurance policy comprised of insureds, their vehicles, and coverages

4.18.1 BIM Structure

A BIM is made up of BIM Entities (e.g., Pizza, Topping). The top entity in the BIM is known as Root. The Root entity always has exactly one instance, while there may be any number (including zero) of instances of other BIM Entities. See section 4.18.3 *BIM Instances* below for a discussion of best practices related to the number of instances.

When architecting the BIM structure, any one-to-one structures should be flattened into the same BIM Entity. When doing this, if it is important to maintain the grouping of Fact Types of the entity that got subsumed, you can use the entity name as the Business Concept portion in the names of the related Fact Types. This is an exception to the normal BIM Entity Fact Type naming scheme.

While there is no product limitation on the number of levels supported in a BIM structure, having many levels will push the limits of understandability and will likely exceed Decision size best practices (see

[1] Bruce Silver, *BPMN Method and Style, Second Edition, with BPMN Implementer's Guide* (Aptos, CA: Cody-Cassidy Press, 2011), pp.71 – 82.

section 4.17.2 *Decision Size* above). The practical limit is 6 or 7 levels. You also need to be sensitive to the total number of instances in the BIM structure – see section 4.18.3 *BIM Instances* below.

DM imposes some limitations on the structure. A Fact Type can exist only in one place in the entire BIM structure, whether for a Decision or for a Flow. This often leads to duplication of Fact Types, one for each BIM Entity where the Fact Type is needed, for example *Shipping Address City* and *Billing Address City*. Another significant limitation is that the BIM structure used in all the Decisions within a Flow must be consistent. All Decisions in the Flow must have the same Root entity and their BIM structures must align with each other.

4.18.2 BIM Entity Naming

4.18.2.1 BIM Entity Name Length

How long is too long for a BIM Entity name?

Since the recommended Fact Type naming scheme (see section 3.2 *Fact Type Naming* above) utilizes the BIM Entity Name for the *Business Concept* portion for all Fact Types associated with that BIM entity, it is best to keep the BIM Entity Name short, a word or two.

4.18.2.2 Singular or Plural?

Should BIM Entities be named as singular (e.g., Topping) or plural names (e.g., Toppings)?

Use of the singular form aligns with the recommended Fact Type naming scheme (see section 3.2 *Fact Type Naming* above) where the BIM Entity name is used for the *Business Concept* portion of a Fact Type's name. Since it is easy to see from the Fact Type name which BIM Entity a Fact Type is associated with, this promotes understandability of the models and makes the creation and maintenance of the BIM easier.

The best practice is to use the singular form for BIM Entity names.

4.18.3 BIM Instances

The number of instances in the BIM has a large impact on the performance of Decisions/Flows invoked with that input structure. Each Rule Family that is attached to a BIM Entity will iterate over **all** instances of that BIM Entity. There is currently no mechanism to stop that iteration when some condition is satisfied. The sizes of the input message and its resulting output message are directly related to the number of instances in the BIM. This impacts the memory requirements and the processing time on both ends to marshal/unmarshal the messages.

As a rule of thumb, we recommend limiting your BIM structures to no more than 1,000 instances in total.

Why is this a rule of thumb and not a best practice? It is a rule of thumb because it is intended to stimulate thinking about performance and the appropriate decision architecture to ensure acceptable performance for the specific use case. Furthermore, what is an instance? Since a BIM Entity may be

comprised of any number of Fact Types, the sizes of instances may vary widely, as will their performance impact.

4.18.4 Approaches to Modeling With BIM Entities

Decision models are ultimately about a single conclusion. When multiple instances are involved, any findings about those instances needs to be aggregated. There are four approaches to doing this. In this section we will explore all four approaches, using the same example for each, and then make recommendations at the end.

We'll use the following intentionally simple example to explore the approaches to modeling with a BIM: a pizza order must have at least one large pizza to be eligible for the large pizza promotion.

In general, an aggregate conclusion about a set of BIM instances (for example, the pizzas in a pizza order) can be reached in one of two ways:

- Counting the relevant instances
- Appending the relevance finding for each instance into a List Fact Type and then searching the list for relevant values

We call these the *Count* and *List* methods respectively.

The aggregation can be done separately from the determination for each instance or at the same time. The former will require three Rule Families to make the final determination while the latter will require only two. Either aggregation method can be combined with both the Count and List methods, yielding four approaches.

4.18.4.1 Count-3 Approach

The first approach we'll look at is the simplest. As its name suggests, the Count-3 approach uses the count method and three Rule Families.

Order Large Pizza Promotion Eligible Pizza Count	Order Large Pizza Promotion Eligibility Indicator
Is Greater Than or Equal To : 1	Is : Eligible
Is : 0	Is : Ineligible

Pizza Large Pizza Promotion Eligibility Indicator	Order Large Pizza Promotion Eligible Pizza Count
Is : Eligible	Is Incremented By : 1
Is : Ineligible	Is Incremented By : 0

Pizza Size Code	Pizza Large Pizza Promotion Eligibility Indicator
Is : Large	Is : Eligible
Is Not : Large	Is : Ineligible

The *Pizza Large Pizza Promotion Eligibility Indicator* Rule Family makes a determination for each pizza whether it is eligible for the promotion. The *Order Large Pizza Promotion Eligible Pizza Count* Rule Family then counts the eligible pizzas. Finally, the *Order Large Pizza Promotion Eligibility Indicator* Rule Family compares the eligible pizza count against the threshold.

4.18.4.2 Count-2 Approach

In the Count-2 approach, the Decision Rule Family is the same as in the Count-3 approach. The other two Rule Families are combined into one.

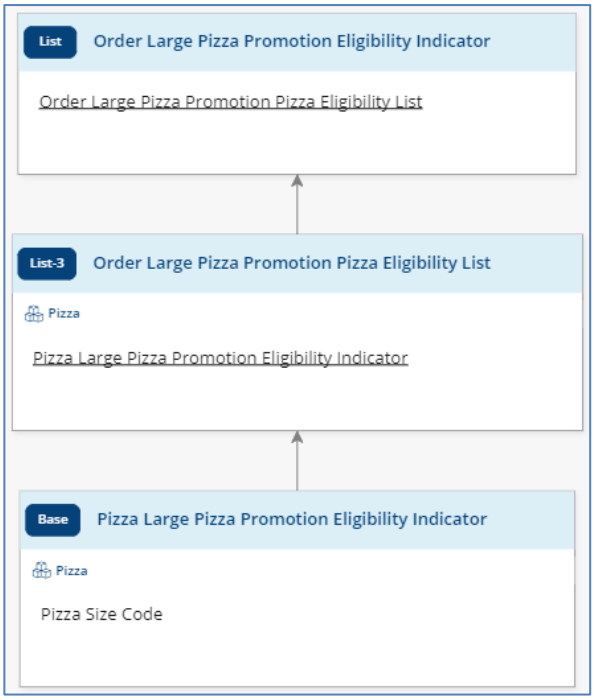
Order Large Pizza Promotion Eligible Pizza Count	Order Large Pizza Promotion Eligibility Indicator
Is Greater Than or Equal To ⁝ 1	Is ⁝ Eligible
Is ⁝ 0	Is ⁝ Ineligible

Pizza Size Code	Order Large Pizza Promotion Eligible Pizza Count
Is ⁝ Large	Is Incremented By ⁝ 1
Is Not ⁝ Large	Is Incremented By ⁝ 0

Here, the *Order Large Pizza Promotion Eligible Pizza Count* Rule Family counts pizzas that are eligible without making an explicit eligibility determination. Then, the *Order Large Pizza Promotion Eligibility Indicator* Rule Family compares the eligible pizza count against the threshold.

4.18.4.3 List-3 Approach

As expected, the List-3 first approach uses the list method and three Rule Families.



Order Large Pizza Promotion Pizza Eligibility List	Order Large Pizza Promotion Eligibility Indicator
Contains : {Eligible}	Is : Eligible
Does Not Contain : {Eligible}	Is : Ineligible

Order Large Pizza Promotion Pizza Eligibility List
Is Appended With : Pizza Large Pizza Promotion Eligibility Indicator

Pizza Size Code	Pizza Large Pizza Promotion Eligibility Indicator
Is : Large	Is : Eligible
Is Not : Large	Is : Ineligible

As in the Count-3 approach, the *Pizza Large Pizza Promotion Eligibility Indicator* Rule Family makes a determination for each pizza whether it is eligible for the promotion. The *Order Large Pizza Promotion Pizza Eligibility List* Rule Family then appends the eligibility finding to a list. Finally, the *Order Large Pizza Promotion Eligibility Indicator* Rule Family checks the list to see if any of the findings are eligible.

4.18.4.4 List-2 Approach

In the List-2 approach, the Decision Rule Family is the same as in the List-3 approach. The other two Rule Families are combined into one.

Order Large Pizza Promotion Pizza Eligibility List	Order Large Pizza Promotion Eligibility Indicator
Contains : {Eligible}	Is : Eligible
Does Not Contain : {Eligible}	Is : Ineligible

Pizza Size Code	Order Large Pizza Promotion Pizza Eligibility List
Is : Large	Is Appended With : Eligible
Is Not : Large	Is Appended With : Ineligible

Here, the *Order Large Pizza Promotion Pizza Eligibility List* Rule Family determines the eligibility of each pizza and appends it to a list. Then, the *Order Large Pizza Promotion Eligibility Indicator* Rule Family checks the list to see if any of the findings are eligible.

4.18.4.5 Approach Recommendations

The advantages of the three-Rule-Family methods over the two-Rule-Family methods are the reusability of the lowest Rule Family in Decisions with a flat BIM and the structural separation of the aggregation from the logic. These advantages come at the cost of an additional Rule Family.

The primary advantage of the two-Rule-Family methods is compactness of the solution. However, the two-Rule-Family methods, in particular in conjunction with the count method, suffer from a **risk** of incorrect results. To demonstrate this, let's change our example and say that an order is eligible for a

promotion only if it has no more than one pizza that is not small and round. The Rule Family would commonly be authored as a row-diagonal Rule Family:

Pizza Size Code	Pizza Shape Code	Order Promotion Ineligible Pizza Count
Is : Small	Is : Round	Is Incremented By : 0
Is Not : Small		Is Incremented By : 1
	Is Not : Round	Is Incremented By : 1

The problem with this Rule Family is that a large rectangular pizza (and others as well) would be counted twice as both *Is Incremented By 1* rows will fire. When used in this fashion, the Rule Family must be modeled so that only one row could ever fire:

Pizza Size Code	Pizza Shape Code	Order Promotion Ineligible Pizza Count
Is : Small	Is : Round	Is Incremented By : 0
Is Not : Small		Is Incremented By : 1
Is : Small	Is Not : Round	Is Incremented By : 1

Because of this risk of incorrect results and the reusability of the eligibility Rule Family, the recommendation is to use the three-Rule-Family methods.

Comparing the list method with the count method, most users find the list method easier to understand as counting can be unnatural in those use cases where the list method is applicable. If the condition being tested is one of the following, there is no need to count the instances of interest and the list method should be used:

- At least one
- All
- None

Putting it all together, the recommend approach from a business perspective, when it fits (at least one, all, or none), is the List-3 approach. When it is not applicable, use the Count-3 approach. Where there are high performance requirements, the Count-2 approach, with fewer Rule Families and simple counting instead of processing lists of strings, will yield the fastest performance.

4.18.5 BIM Entities With Zero Instances

When a BIM Entity does not have any instances, all Rule Families attached to that BIM Entity will be skipped during execution and their conclusion Fact Type values are set to null (if the BIM Entity instances in which the conclusion Fact Types reside have instances). If zero instances are expected, then the null must be handled in the supported Rule Families as described in section 4.12 *Dealing With Null*

Values above. If zero instances are not expected, then this is considered untrusted data and should be handled as described in section 4.20 *Modeling for Untrusted Data* below.

4.18.6 Skipping BIM Levels

DM supports skipping of BIM levels in a Decision. This reduces the size of the Decision, making it more understandable and improving performance.

For example, consider a pizza order promotion where the eligibility for the promotion depends only on the selected toppings. If the BIM structure is

- Root (Order)
  - Pizza
    - Topping

there is no need to have a Rule Family for the Pizza BIM Entity as there is no business logic at that level.

You can skip any number of levels as long as at least one Fact Type from each BIM Entity is referenced somewhere in the model (could be in a message).

4.19 Trusted Data

Rule Families that implement business logic should be modeled under an assumption of trusted data, meaning that the Rule Family only handles values that are within the expected domains of the participating Fact Types. The same concept also applies to BIM Entity instances, meaning that a Rule Family should not handle the zero instances case if at least one instance of the BIM Entity is required.

The assumption of trusted data enables separation of the business logic from any logic dealing with the validity of the input data, which we refer to as data acceptance (see section 4.20 *Modeling for Untrusted Data* below). The result is simpler Rule Families, and two sets of logic that can be independently modeled, governed, and reused.

4.20 Modeling for Untrusted Data

What happens if the data being used for a Decision cannot be trusted? In other words, how do we ensure that our business logic is executed with data that is of sufficient acceptability[2] to make a high-quality decision based on that data?

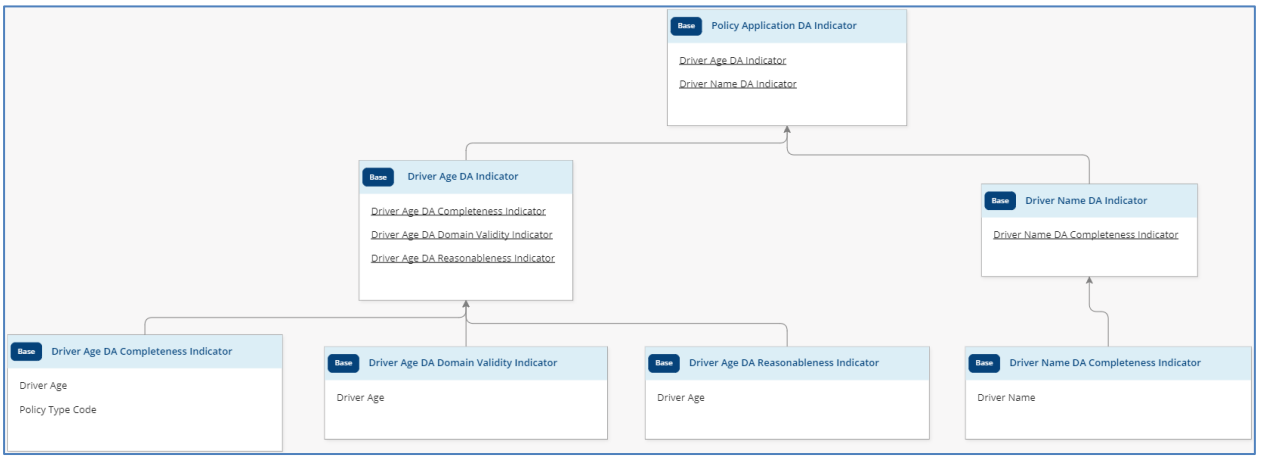
Per the best practices discussed in the previous section, Rule Families that implement business logic are modeled under an assumption of trusted data, so the data acceptability would need to be assessed prior to invoking the business logic, typically via one or more data acceptance (DA) Decisions earlier in the

[2] We prefer the term *data acceptance* over the term *data quality* as we are assessing only the usability of the data for the purpose of making business decisions based on that data.

Flow or programmed in the calling application. For the remainder of this section, we will assume that the data acceptance logic is being done in Decisions.

4.20.1 Data Acceptance Framework

The Sapiens Decision Data Acceptance Framework provides a structured template for how to model data acceptance Decisions. The core concept of the Data Acceptance Framework is to create a branch of Rule Families for each input Fact Type (two input Fact Types, *Driver Age* and *Driver Name*, are being assessed in the example below), assessing each data acceptance dimension in its own Rule Family and rolling all the dimensions up to make an acceptability determination for the input Fact Type.



The Fact Type branches are then grouped by BIM Entity (or Root) and rolled up to make a determination for each BIM Entity and finally for the overall input data.

The Data Acceptance Framework makes extensive use of messages to provide a detailed explanation of all the reasons why a set of input Fact Types was deemed unacceptable.

Data acceptance models are structured and simple to understand. Therefore, the sizing best practices for both Rule Families and Decisions can be relaxed when dealing with data acceptance models. When the models become too large, refactor them into multiple Decisions, grouping by BIM Entities or sets of related Fact Types.

For more information, please refer to the examples in the training material for the Data Acceptance Framework.

Note that the Data Acceptance Framework deals with data acceptance for Fact Type values. It does not deal with ensuring the expected cardinalities of BIM Entities.

4.20.2 Data Acceptance Dimensions

The Sapiens Decision Data Acceptance Framework covers the following dimensions of data acceptability:

Data Acceptance Dimension	Definition
1. Completeness	The determination that no additional data is needed, and irrelevant data is eliminated.
2. Domain Validity	The determination that a provided value falls within the Fact Type's valid range.
3. Consistency	The determination that a provided value makes business sense in the context of related Fact Type(s) fact values.
4. Accuracy	The determination that a provided value (regardless of the source) approaches its true value.
5. Reasonableness	The determination that a provided value conforms to predefined reasonability limits.
6. Format Validity	The determination that a Fact Type value's format conforms to a predefined format.

4.20.3 Naming of Data Acceptance Fact Types

Like the data acceptance logic itself, the Fact Type names used in data acceptance are also highly structured:

Item	Naming Scheme	Example
Conclusion Fact Type for evaluating a data acceptance dimension for an input Fact Type	{Input Fact Type name} DA {dimension} Indicator	Driver Age DA Completeness Indicator
Conclusion Fact Type for evaluating an input Fact Type	{Input Fact Type name} DA Indicator	Driver Age DA Indicator
Conclusion Fact Type for evaluating a BIM Entity instance	{BIM Entity name} DA Indicator	Topping DA Indicator
Conclusion Fact Type for evaluating all instances of a BIM Entity	{Parent BIM Entity name} {BIM Entity Name} DA Indicator List	Pizza Topping DA Indicator List
Decision Rule Family conclusion Fact Type for a data acceptance Decision	{Input data structure name} DA Indicator	Pizza Order DA Indicator

Note that the first two examples show the data acceptance naming as suffixes to the name of the input Fact Type being evaluated. This is intended to group the input Fact Type along with its data acceptance.

4.21 Modeling for Deterministic Execution

As a best practice, decision models should be *deterministic*. Given the same input, the same decision model should consistently return the same results.

Why is this beneficial? This simplifies testing and aids in ensuring the correctness of the logic. Using deterministic models also greatly reduces on-going Test Case maintenance.

Decision models built in DM generally do not have the ability to access data other than that provided with the input. However, external data can be brought into decision models through the use of functions. Two common examples of this are the TODAY() and NOW() functions. Similarly, some decision models incorporate a randomization function (e.g., A/B test routing) as a User-Defined Function. User-Defined Functions may also be used to retrieve data from a data source (see section 4.6 *Lookup Tables* above). In all these cases the models are not deterministic and running the same logic again, perhaps on a different day or time, with the same input data, may yield different results.

To get back to deterministic execution:

- Replace the TODAY() and NOW() functions with appropriate input values.
- Replace the randomization function with a random number provided in the input.
- Perform any data lookups prior to invoking a Decision/Flow and provide the values in the input. These external lookups are not possible if any of the values to be used for the lookups are determined within the Decision/Flow itself.

The above does not apply to the lookups performed by the Sapiens Decision Parameter Management (PaM) lookup function as its data table versions are immutable (via the PaM app and services) and therefore deterministic.

4.22 Duplicate Logic

DM is designed to encourage reuse of existing assets. To that end, DM prevents the creation of duplicate logic. This is enforced as part of the checks that are done when a Modeling Task is submitted, or when manually initiated. These checks compare draft assets with the latest version of approved assets.

Two Rule Families are considered duplicate if they have the exact same business logic: the same Conclusion Fact Type, the same Condition Fact Types, the same cells (i.e., logic), and the same messages. The order of the columns and rows does not matter. The Fact Type versions do not matter since that relationship is maintained at the Decision level.

Two Decisions are considered duplicate if they have the same Rule Families and the same Fact Type versions.

By default, the check for duplicates is done only against the latest version of other views of the Decision or Rule Family being checked. DM does allow the undoing of a change in a subsequent version. For example, if version 1 of a Rule Family checks against a credit score of 650 and the threshold is changed in version 1.1 to 640, the threshold can be changed in version 1.2 back to 650 without it being

considered a duplicate. The duplicate logic check may be turned off via a system-wide configuration setting.

From a best practice point of view, Sapiens recommends starting with the default settings and then turning the duplicate check off if needed. Going in the reverse direction is not recommended since duplicate assets may have already been created.

There are situations which may require temporarily turning the duplicate check off. The most common one occurs when you want to change the view name of some approved assets while leaving the logic the same.

4.23 External Rule ID

It is often desirable to associate a fixed ID with a particular rule (the word *rule* is being intentionally used here rather than *row*). Common use cases include publishing of the rule and tracking how many times the rule fired (or didn't fire) over time.

DM assigns a unique row ID to each row in a Rule Family, however all rows in a Rule Family get a new row ID when a draft version of the Rule Family is created. DM also calculates an external hash for each row based on the contents of the row (excluding messages). While the hash will remain constant for unchanged rows in a new Rule Family version, changed rows will get a new hash. Both the row ID and the row hash are available in execution output, but as we've seen, they are too fine grained to track a rule as it changes over time.

When enabled, the External Rule ID mechanism assigns a unique ID to each row and that ID stays with the row across changes and versions. This ID is a fixed identifier for the rule. If a row has changed significantly so that it is in essence a new rule, it is possible to generate a new ID for that row.

By default, the External Rule ID excludes conclusion Fact Types. This means that a row with the same condition columns, condition cells, and conclusion cells in two unrelated Rule Families (different conclusion Fact Type) would get the same External Rule ID. Note that messages are excluded. Most implementations consider the conclusion Fact Type to be part of the logic, so as a best practice, when using the External Rule ID feature, we recommend turning on the option to include the conclusion Fact Type in the calculation of the hash that drives the External Rule ID. See the *Decision Manager (DM) Configuration Guide* for more information.

4.24 Handling Dates and Times

Date and *Date & time* data types are represented with the same internal data type; a *Date* value is merely a *Date & time* value with the time set to midnight.

A *Date & time* value can be assigned to a *Date* Fact Type. When this happens, the time portion is not removed and the *Date* Fact Type value is processed with the time as provided. This can lead to **incorrect results**. Furthermore, it is difficult to spot this in the testing facility within DM as the display formats for a *Date* Fact Type omit the time portion, so the extraneous time data is not visible.

Let's look at an example to make this clearer. When checking for lapses in insurance coverage, we may want to check *DAY_DIFF(Policy Term Start Date,Policy Prior Term End Date)*. If *Policy Term Start Date* is January 5, 2023 and *Policy Prior Term End Date* is inadvertently set to 10AM on January 4, 2023 instead of just January 4, 2023, the function will return 0 because the difference between the two dates is only 14 hours, not a full 24 hours.

As a best practice, when passing a *Date & time* value to a *Date* Fact Type, you should use the *DATE_FROM_DATETIME* function to set the time portion to midnight, effectively removing it. You should do this for any *Date* inputs that may have a time portion and for any other conversions from *Date & time* to *Date*.

In the above example, had *DATE_FROM_DATETIME* been applied to the *Policy Prior Term End Date* before calling the function, the function result would be 1 as expected.

4.25 Dealing with Time Zones

The time in a Fact Type of type *Date & time* does not have a time zone. Times are processed based on the system time of the server (e.g., UTC). All times with a time zone must be converted to the server time zone and provided as execution input without a time zone.

If your logic is dependent on the local time (for example, offering a discount for a pizza order placed before 4PM local time) then you may need to pass the time zone or its offset as a separate input and calculate the adjustments from the server's time zone. Since Sapiens Decision does not have a built-in *HOUR_ADD* function, this is best done with a UDF.

5 Testing

5.1 Testing Methodology and Best Practice

A successful testing strategy is one that provides assurance of a decision model's soundness and correctness. This strategy should also give confidence that when implemented, the model will execute as intended. A sound decision model is one that covers all situations within its scope and does so without contradiction. A sound model that returns correct conclusions is one that is certain to execute as intended. Therefore, our testing methodology will focus on 3 evaluation dimensions: completeness, consistency, and accuracy. In some cases, we will recommend a strategy for selecting values in the test module's generator tool. In others, we will give observational guidance that we recommend incorporating into your model review process.

Completeness: every reasonable scenario has a conclusion.

Consistency: every conclusion is concludable.

Accuracy: every conclusion is the right conclusion.

We'll look at testing of new models first, before moving on to maintenance.

5.1.1 Completeness

5.1.1.1 Definition

A complete model is one that accounts for every reasonable scenario within its scope. Given a valid combination of input values, a complete model will always return a conclusion.

5.1.1.2 Testing Theory/Strategy

The test should contain all reasonable scenarios, considering context, scope, and whether each Fact Type is expected to have a value or not. This test is considered passed when all its cases return a conclusion.

5.1.2 Consistency

5.1.2.1 Definition

A consistent system of logic is one that is free from contradiction and stands in agreement with facts. For a decision model, it makes sense to deal with each requirement separately:

- Internal consistency – agreement within itself (no contradictions)
- External consistency -- agreement within its context

A consistent decision model is one where every one of its conclusions can be derived from a reasonable scenario (external consistency) and no single reasonable scenario leads to a contradiction (internal consistency).

5.1.2.2 Testing Theory/Strategy

DM's validation process will ferret out contradictions of overlapping logic and conflicting conclusions within a model, which means, for the most part, testing for internal consistency is not needed. However, we will present guidance on what is needed to verify internal consistency, including how to handle any applicable validation warnings.

The test for external consistency should include one Test Case for each conclusion (as specified in the logic) and each of these Test Cases must contain only valid and/or reasonable input values.

Note: when needed, the consistency Test Group may be extended to fully cover a model's logic by representing all reasonable scenarios that lead to each conclusion value (see section 5.2.1.3 *Rule Family Accuracy* below).

5.1.3 Accuracy

5.1.3.1 Definition

An accurate model is one that returns the correct conclusion – as intended by the business – in every reasonable scenario.

5.1.3.2 Testing Theory/Strategy

Testing with expected values in every reasonable scenario may be prohibitively time consuming and considerably redundant. In such situations, our methodology will require testing for full coverage consistency in combination with a smaller – yet sufficiently representative – standard accuracy test.

5.1.3.3 Standard Accuracy Test

Since a decision model's accuracy measurement standards typically originate outside the model (i.e., from the business), the testing strategy should align with what business is accustomed to. For example, business owners may have already defined test scenarios and acceptance criteria for the system task that consumes the model. These should be converted into DM Test Cases and considered a baseline for accuracy. If no acceptance criteria or test scenarios exist, we suggest attempting to solicit them from the business owners. Whatever their source, accuracy test cases should always reflect real-world scenarios – particularly ones that business cares about – with expected results that match what the business intends.

5.2 Testing Sequence

The decision model's simple structure and rigid logical framework combined with the software's inbuilt validation algorithms (which enforce the principles of that framework) provide several assumptions that can significantly boost testing efficiency. For example, consider that Decisions are merely sets of inferentially related Rule Families, and Flows are just ordered sets of those same inferentially related Rule Families. Therefore, many of the assurances we gain from testing Rule Families can be directly applied to the Decisions and Flows they are a part of. This means that testing a single Decision or Flow is significantly less intensive when all its constituent Rule Families are already complete, consistent, and accurate. Thus, our methodology assumes that when a Decision or Flow is to be tested, the testing of all its constituent Rule Families has already been completed, unless otherwise noted. Similar efficiencies arise when accuracy is tested after consistency and consistency after completeness. For these reasons, it is strongly recommended that our methodology be followed in the order presented here.

5.2.1 Rule Family Testing

5.2.1.1 Rule Family Completeness

A test for a Rule Family's completeness is essentially a test of TDM Principle 12e which states:

> *"A Rule Family must result in at least one conclusion value for any set of valid input values for the condition fact types."*

Therefore, a Rule Family completeness test should include all possible combinations of valid input values – those that are reasonably expected and within the scope of the Rule Family's logic. Null values should only be included for inputs that are optional or conditionally required. A Rule Family may be considered logically complete when there is a Test Group constructed in this manner and every Test Case satisfies at least one row of logic. This result can be seen by filtering the executed Test Cases by *No Row Hit* and verifying that there are none.

5.2.1.1.1 Choosing/Creating Fact Type Test Values in the Test Case Generation Module

- Fact Types with discrete domain values: all valid values

  However, TDM Principle 12e only requires that the full scope of the Rule Family logic be covered. So, for logic like *FT1 Is In {a,b,c}*, only two Test Values are necessary: *a* and *d*, where *d* is any of the Fact Type's other domain values (i.e., not *b* or *c*).

- Fact Types without discrete domain values: 2-3 values for each unique condition statement

  - If condition is a boundary comparison (e.g., *FT1 Is Greater Than or Equal To 7*), three values – one for each side of the boundary and one for the boundary itself. Example Test Values: {2,7,12}
  - If condition is a matching argument (e.g., *FT1 Is In {a,b,c}*), two values – one matching and one non-matching. Example Test Values: {b,d}
  - If condition is a boundary comparison with a Fact Type in its operand (e.g., *FT1 Is Less Than FT2* or *FT1 Is On or Before The Date FT2*)

For each *unique* pair of Fact Types in the Rule Family use a single pair of Test Values for both Fact Types (the same two values for both). This will ensure that all combinations of *less than* (before), *greater than* (after), *is*, and *is not* will be generated.

- Example:

FT1	FT2
1/1/2000	1/1/2000
5/5/2000	5/5/2000

- Fact Types in conclusion cells only: 1 Test Value
  - 1 Test Value is sufficient for completeness testing, as formulae will be tested for accuracy using multiple values.
- Optional or conditionally required Fact Types: include *null* as a Test Value
- List Fact Types
  - Currently, the testing module does not generate multiple values for a List Fact Type. Therefore, to satisfy completeness, we must create its Test Cases manually using all reasonable combinations of the set elements given in the Rule Family operands.

    For example, given the following Rule Family,

Pizza Topping Code List PC	Pizza Topping Code List	Pizza Topping Code List	Pizza Acceptability Indicator
Is : NotPopulated			Is : Acceptable
Is : Populated	Contains : {Pineapple}	Does Not Contain : {Mushrooms}	Is : Acceptable
Is : Populated	Contains : {Onions}	Does Not Contain : {Pineapple}	Is : Unacceptable
Is : Populated	Contains : {Mushrooms}		Is : Unacceptable

the set of *Pizza Topping Code List* values for completeness testing should look like this:

NULL

{Mushrooms}

{Onions}

{Pineapple}

{Mushrooms,Onions}

{Mushrooms,Pineapple}

{Onions,Pineapple}

{Mushrooms,Onions,Pineapple}

5.2.1.1.2 Reasonableness Considerations

We may at times encounter persistent Fact Types whose value domains are conditionally restricted by context or by an implied relationship with another Fact Type. For example, Total Amount vs. Item Amount, or Previous Date vs. Current Date, where one Fact Type is always larger-than, smaller-than, before, or after the other. A completeness Test Group should preserve these kinds of conditional relationships and context restrictions so that no unreasonable scenarios are included.

5.2.1.1.3 Data Acceptance Considerations

When testing data acceptance (see section 4.20 *Modeling for Untrusted Data* above) Rule Families, you will want to also test invalid values and invalid combinations of values. Include nulls.

5.2.1.2 Rule Family Consistency

5.2.1.2.1 Internal Consistency

- If no "overlapping logic" warnings are returned by the validation process, no further action is needed.
- For any warnings of "partial" overlap, consider them carefully.

  A common scenario occurs in a row-diagonal (see section 4.13.1.1 *Row-Diagonal Pattern* above) Rule Family where the same formula appears in two cells, within one condition column, with opposite operators. For example:

Pizza Acceptability Indicator	Pizza Slice Quantity	Pizza Order Acceptability Indicator
Is ⁝ Acceptable	Is Greater Than Or Equal To ⁝ *formula*	Is ⁝ Acceptable
Is ⁝ Unacceptable		Is ⁝ Unacceptable
	Is Less Than ⁝ *formula*	Is ⁝ Unacceptable

  DM reports a "partial" overlap, but there is no overlap.

  After consideration, if any partial overlap warnings remain, ensure that the completeness Test Group is free from execution errors, as they can be a result of inconsistencies uncaught by validation (e.g., when formulas contain Fact Types).

- For any warnings of "full" overlap, refactor the Rule Family to remove them.
  - Any overlapped rows present the potential for a RowConflictException, wherein a single input set satisfies more than one row AND those rows have different conclusions.
  - Rule Families that use the conclusion aggregation operators (e.g., append/increment) are not subject to row conflicts.

5.2.1.2.2 External Consistency

- Consider any occurrence of the following validation warning: “Rule Family does not use all Domain values for Condition Fact Type...”. If the referenced Fact Type is a persistent one, ensure that any domain values omitted from the Rule Family’s logic cannot be reasonably expected as inputs. If the warning references an interim Fact Type, ensure that the supporting Rule Family (where that Fact Type is its conclusion) does not contain a conclusion cell yielding one of those omitted values.
- Write one test case that satisfies each row of logic – this ensures *full coverage*. If the Test Group has satisfied every row of logic, then every Rule Family conclusion value will be present in the Test Group.

5.2.1.3 Rule Family Accuracy

A Rule Family that is succinct, atomic, and complete is one whose behavior is self-evident. This is especially true for smaller Rule Families and those with a single simple pattern (e.g., a row-diagonal Rule Family). The accuracy testing of such Rule Families may appear so trivial as to be unmeaningful, yet we still recommend testing them. At the very least, accuracy tests can reveal modeling mistakes (e.g., omitting the *equal-to* boundary in a *less-than*/*greater-than* comparison) that have persisted through previous testing dimensions.

5.2.1.3.1 Primary Rule Family Accuracy Testing Approach: Enriching the Consistency Test Group

Add expected values to the consistency Test Group. Review each Test Case in the consistency Test Group and enter an expected result that matches the business intent or requirement. Should the consistency Test Group prove too large for this approach, use the secondary approach.

5.2.1.3.2 Secondary Rule Family Accuracy Testing Approach

Extend the consistency Test Group so that it fully covers the logic in each primary conclusion row, where the primary conclusion rows are those that contain business rule logic and are not an “otherwise” row.

The fully covered consistency Test Group contains one Test Case for each way of satisfying the logic in the primary conclusion row(s). For example, consider a primary conclusion row with the condition statement, *FT1 is in {**a**,**b**,**c**}*. Full coverage means having Test Cases for each implied statement within the condition, i.e., *FT1 is* **a**; *FT1 is* **b**; *FT1 is* **c**.

Add a representative accuracy Test Group that contains valid real-world scenarios with expected results that the business intends.

Rule Families that contain formulas should always have an accuracy Test Group – apart from their consistency and completeness Test Groups – in which all formulas are tested.

5.2.2 Decision Testing

5.2.2.1 Decision Completeness

The completeness of a Decision is assured by the completeness of all its Rule Families. When all the Rule Families in a Decision are complete, it follows that the Decision itself is complete. Therefore, no completeness testing is needed for a Decision if all its Rule Families have passed their respective completeness tests.

5.2.2.2 Decision Consistency

5.2.2.2.1 Internal Consistency

A Decision's *internal consistency* is essentially an aggregation of the *external* consistencies of every one of its Rule Families. An internally consistent Decision is one where all its Rule Families agree with one another and more specifically, that every inferential relationship is fully valid. This means that all the specified conclusions of each supporting Rule Family are fully handled by the logic of its supported Rule Families. DM's validation process will generate a Principle 14 warning when an explicit conclusion value in a supporting Rule Family is not consumed in each supported Rule Family. DM's validation will not generate a warning when the conclusion values come from Fact Types or formulas.

5.2.2.2.2 External Consistency

An *externally* consistent Decision is one whose conclusion values are all concludable. Specifically, every explicitly stated Rule Family conclusion is achievable (e.g., every row can be hit), given a set of input values that are reasonable for the Decision.

Even if all Rule Families in the Decision are themselves complete and externally consistent, and the Decision is internally consistent, the external consistency of the Decision is not ensured. Inconsistencies may arise, particularly when best practices are not followed in the modeling of the Rule Families whereby all conclusion Fact Types are clear, precise, and aligned with a single business concept.

One approach to testing for external consistency would be to do a test of all valid combinations of input values and then examine the conclusion values to ensure that all possible conclusion values were produced. This is only feasible for Decisions with few input Fact Types, as the number of Test Cases could easily be in the billions or more as the number of input Fact Types grows.

As a practical approach:

- Generate a large set of Test Cases that is reasonably broad with at least 2 values for each Fact Type.
- Run the Test Group and then export as CSV.
- Filter the Decision Rule Family's conclusion values for uniqueness and check that all its conclusion values are present.

5.2.2.3 Decision Accuracy

Create a Test Group that contains valid real-world scenarios with expected results that the business intends. If possible, solicit Test Cases directly from the business.

5.2.3 Flow Testing

5.2.3.1 Flow Completeness

A Flow is complete when all the reasonable scenarios within its scope will reach an endpoint – since the "conclusion" of a Flow is simply one of its endpoints.

If a Flow has no gateways, its completeness merely depends on the completeness of all its constituent Decisions. If all the Decisions are complete, then the Flow's completeness is trivial because all its Decisions will execute, thus the Flow's endpoint will always be reached.

If a Flow has gateways, in addition to the completeness of all its constituent Decisions, verify that every gateway is "exit-able":

- Persistent Fact Type gateways - ensure that all possible values of the gateway Fact Type are matched to one of its output pathways.
- Interim Fact Type gateways – ensure all the conclusion values assigned to the gateway Fact Type in a Rule Family are matched to one of the gateway's output pathways.

5.2.3.2 Flow Consistency

5.2.3.2.1 Internal Consistency

DM's validation process will uncover many of the most common ways a Flow is internally inconsistent, such as conflicting BIM structures and differing Fact Type versions.

However, since it is common practice to consume interim conclusions in later Decisions in a Flow (these are identified as Flow interim conclusions and shown with a double arrow icon in DM's test facility; they essentially extend Rule Family inferential relationships across Decisions within a Flow), DM's validation does not know whether a Decision's input is meant to be an input or the consumption of a conclusion of a previously executed Rule Family.

To assist with this, we present the following guidance:

- For each Flow interim conclusion, perform your analysis backwards from the endpoint(s), verifying that each is one of the following:
  - Input: expected to be available as data from a calling system
  - Interim: a conclusion Fact Type from an upstream Decision (one that occurs earlier in the Flow along ALL shared execution pathways)

- Hybrid: expected to be provided as data from a calling system along some Flow pathways and determined within one or more Rule Families in other Flow pathways

In the interim and hybrid cases, you will need to confirm for each Fact Type that all the values produced upstream are consumed. Construct a consistency Test Group with a Test Case to produce each upstream value and ensure that is consumed downstream properly.

Furthermore, in the hybrid case, you will need to verify that all the pathways that expect an upstream value are provided with one. This can be done by adding an additional Test Case for each such pathway, and excluding the Fact Type from the input in each of these Test Cases.

5.2.3.2.2 External Consistency

A Flow's input is the aggregate of all its Decision inputs and its persistent gateway Fact Types. If a Flow has no gateways, its *external consistency* is assured by the external consistency of its Decisions. If a Flow has gateways, construct a consistency Test Group with a Test Case for each pathway through the Flow to ensure that all pathways are traversable.

5.2.3.3 Flow Accuracy

Create a representative accuracy Test Group that contains valid real-world scenarios with expected results that the business intends.

5.3 BIM Testing

The DM testing facility currently generates only one instance for a BIM Entity per Test Case. To do completeness testing more easily for a Rule Family connected to a BIM instance, we recommend flattening the BIM structure by cloning the Rule Family to a Decision with a flat BIM. This enables you to use the test facility's generation functionality to generate the needed Test Cases for the Rule Family. You may want to keep this Decision for future testing needs (or create it as needed), but it should not be deployed.

5.4 Test Group Settings

The following settings are independent – all combinations are possible.

5.4.1 Persistent Test Groups

Each Test Group is marked as *Persistent* by default. This means that that Test Group is automatically copied to new draft versions of the asset. Any *Persistent* Test Group may then need to be updated to accommodate the changes made in that draft version.

As an alternative to the use and maintenance of *Persistent* Test Groups, consider managing your Test Cases externally from DM, in a canonical data model. This will reduce the Test Case maintenance in DM, particularly when the set of Fact Types referenced in the business logic changes – instead of updating the Test Cases, import the Test Groups.

5.4.2 Regression Test Groups

Test Groups may be marked as *Regression* Test Groups.

Deployment environments have an *Include Test Group* setting. When this is selected, the *Test Groups To Export* setting appears and can be set to *All Test Groups* or *Regression Only*. Together, these settings specify which Test Groups, if any, are included when deploying Flows or Decisions to an environment. This is useful for automated pipeline testing (see 5.7 *External Testing* below) to ensure that the external results are the same as the ones seen in the DM testing facility.

5.5 Testing Logic Changes

When the business logic is changed, the changes should be tested to ensure they work as intended and do not have any unintended consequences.

If the change is confined to one Rule Family, and there is no change to the Rule Family's message contract (meaning the inputs and outputs are not changed), it is sufficient to test only that Rule Family. Otherwise, you will need to test the Rule Family, Decision, and Flow (if applicable).

We also need a way to measure the effectiveness of the change; for example, did it improve business results? This is often evaluated via comparative testing or simulation.

5.5.1 Requirements Testing

This testing ensures that the changed logic does what it is supposed to do. Typically, this is done by constructing targeted Test Cases to test the specific requirements, similar to what was described for new business logic models in Sections 5.2 *Testing Sequence* above.

5.5.2 Regression Testing

This testing ensures that the updated logic changes only what was supposed to change due to the new requirements, and nothing more. Typically, this is done by running the same Test Cases (or same test data superset in case of data model changes) against the old version of the logic and the new version of the logic and analyzing any difference in results.

To facilitate this sort of testing within DM, execute your Test Group with the old version of the logic. Export the Test Group and using an external tool such as Excel, set the expected value to the actual value, and then import the Test Group back into DM. Lastly, run the same Test Group – after adjusting as needed for any structural changes – against the new version of the logic and examine any Test Cases that are reported as *Failed*, meaning that the actual conclusion value from execution did not match the expected value.

The above works extremely well for single Rule Family conclusion values and for Decision Rule Family conclusion values. However, it does not work for messages, nor for any Decision interim conclusions. These need to be compared manually within DM as they do not appear in any exports. It may also not work for data structure changes, depending on the extent of the changes.

5.5.3 Comparative Testing/Simulation

Comparative testing is used to assess the impact of a logic change. Two common approaches are Champion/Challenger testing and A/B testing. Results are usually looked at in aggregate, for example a pie chart showing the distribution of conclusion values.

5.5.3.1 Champion/Challenger Testing

In Champion/Challenger testing the same data is run through both the old version of the logic (the champion) and the new version of the logic (the challenger) to see if the challenger yields better results than the reigning champion. If so, it becomes the new champion.

To do Champion/Challenger simulation testing in DM, you would run the same Test Cases (or same test data superset in case of data model changes) against the champion (typically the latest approved version) and the challenger (typically a draft version). Then export the Test Groups and compare the results in an external tool such as Excel.

5.5.3.2 A/B Testing

In A/B testing, one set of data is run against one version of the logic, and another set of data is run against the other version of the logic. In DM, this may be simulated by running different Test Groups. Alternatively, it may be done in production, by randomly allocating some percentage of the total population to the new version.

5.6 Debugging Defects

The best way to debug defects in a Decision environment when the external execution does not produce the expected results is to use the DM testing facility with the same data (after conversion from technical names and values to business-friendly names and values) against the same asset version to visualize the execution and compare the interim results within the testing facility against what was seen externally.

Other than a few edge cases with Model Mapping (see example in section 3.8 *Model Mappings* above) and any expected execution differences due to different null-safe settings (see section 4.12.2 *Null-Safe Execution* above), running the same data against the same asset version will yield the same results in the DM testing facility as it does externally. The comparison of interim results is helpful even in the cases where the execution differs, to quickly home in on the source of the discrepancies.

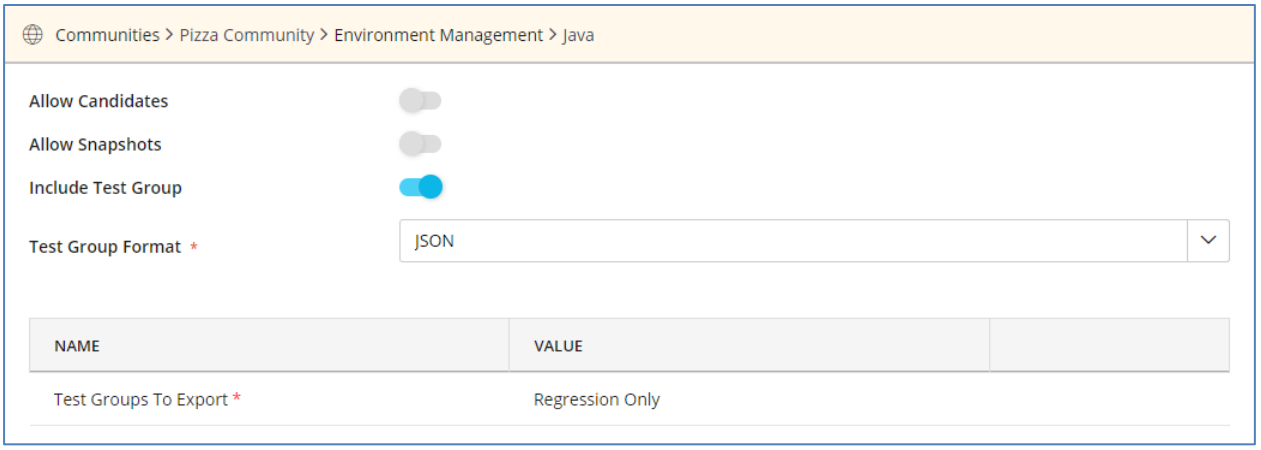
When using DE for execution, you may need to increase the output verbosity to get the required level of detail for comparison. See section 12.1.2 *Output Message Size* below for more information. By design, both the DE SOAP (XML) input messages and the DE RESTful (JSON) input messages can be easily imported into the DM testing facility with minimal manipulation. Selecting the appropriate deployment environment in the import dialog will apply the appropriate Model Mapping to convert technical values to business-friendly values for execution in the DM testing facility.

When using the Java Adapter for execution, see the approach to get the required level of detail for comparison in the *Operationalizing the Java Adapter - getting the results you expect* white paper.

5.7 External Testing

To test that external execution results match those seen in the DM testing facility, export appropriate Test Groups from DM and run them in a test harness after each deployment to compare the results.

When using the Java Adapter, this can be automated into your pipeline by including Test Groups in the deployment package:



Select the appropriate Test Group format and whether to include all Test Groups for the assets in the deployment package or only Test Groups marked as *Regression* Test Groups (see section 5.4.2 *Regression Test Groups* above).

Since DE does not provide a web service to retrieve Test Groups, you will need to extract the Test Groups from the deployment package generated for a deployment to a separate environment (e.g., Minimal CSV) that includes Test Groups as shown above for the Java Adapter and is configured with the same Model Mapping as the DE environment.

The exported Test Groups include only Decision Rule Family results so the external comparison will not be able to compare messages or interim results.

6 Governance

DM was designed from the ground up with extensive governance capabilities. The primary governance facilities are customizable *approval processes* for Fact Types and logic assets and optional approval strategies for Model Mappings, Revision Tasks, and deployments. DM also requires approvals for Repository Tasks. We will explore each of these capabilities.

6.1 Approval Processes

Approval processes have a dual purpose in DM: they govern the review and approval of assets, and they control a key part of the asset life cycle (draft to candidate to approved).

Each Modeling Task requires two approval processes to get approved assets into the repository: one for asset approval and one for Fact Type approval. Fact Types have their own approval processes due to their importance to the shared understandability of decision models. Having a separate approval process provides the means to enable a dedicated role, typically the *Glossary Administrator* role, for the review and approval of Fact Types.

Approval processes can be as simple as one-click approval or comprised of multiple review/approval steps. The best practice is to start out simple (with one-click approval or one review step) and add more complexity as the use of DM matures.

Each stage in an approval process is associated with a role. In general, only users who have that role in the Community associated with the Modeling Task can move the approval process forward when it is in a stage associated with that role. As an exception to the previous statement, the Modeling Project Manager can move the approval process forward for transitions that do not change the asset life cycle stage, those transitions that have *maintain status*. The Modeling Project Manager can also move the approval process to earlier stages, based on the available transitions in the current stage. In addition, prior to approval, a user that participated in the approval process can *recall* the approval process back to the previous stage from which the user submitted. Post approval, a participating user can *recall* only to the previous approved stage from which the user submitted.

One question that must be answered is what does “Approved” mean? In some organizations, “Approved” means that the assets in the Modeling Task are approved to be published into the repository and available for all (subject to permissions) to see and use. In other organizations, “Approved” means approved for deployment into production. Depending on the definition, the approval processes will be designed differently. In the latter case, deployment will need to be done while the assets are in candidate status, prior to approval. DM needs to be configured for this (see section 7.6 *Candidate Deployment* below) and the approval process must be set up to accommodate this.

6.1.1 Approval Processes and the Asset Life Cycle

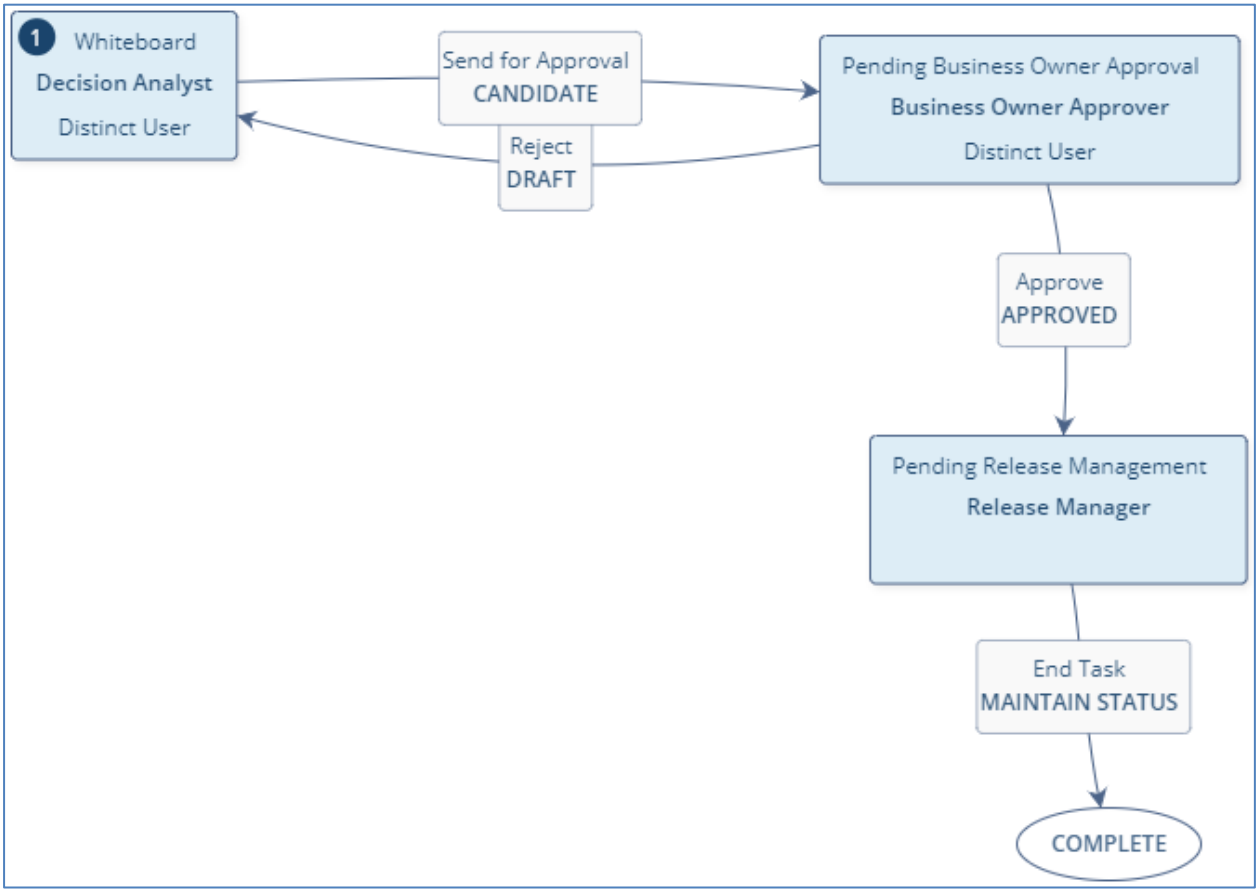
One of the purposes of the approval processes in DM is to control a key part of the asset life cycle. This is done through the state transitions in the approval process. Approval processes always start in draft state. Thereafter, the allowed state transitions are:

- Draft to candidate
- Draft to approved
- Candidate to draft
- Candidate to approved

As a best practice, use *maintain status* on any transition that does not change the asset life cycle stage.

All approval processes must end in approved state (or maintain status after approved).

The screenshot below shows a simple approval process, draft to candidate and then either back to draft or on to approved. The states can be seen in bold at the bottom of the boxes on each transition arrow.



6.1.2 Segregation of Duties

Many organizations have a requirement to segregate duties. In the context of DM, this typically means that a user that modeled something cannot also review or approve it. This requirement is handled by selecting *Distinct User* for all the stages in the approval process which require distinct users.

In the screenshot above, the *Whiteboard* and the *Pending Business Owner Approval* stages both have *Distinct User* set. This means that the Decision Analyst that clicked *Send for Approval* to submit the

Modeling Task from the *Whiteboard* stage cannot claim the Modeling Task when it is in the *Pending Business Owner Approval* stage, even if the user has the Business Owner Approver role and therefore, they will also not be able to click *Approve*. Since the *Pending Release Management* stage does not have *Distinct User* set, any user that has the Release Manager role in the Modeling Task Community can claim the Modeling Task.

As a best practice, it is recommended to have at least one additional person review a model prior to its approval into the repository, regardless of whether this is part of the formal governance enforced via the approval processes.

6.1.3 Group Tasks List or Assignment to Named Users

At any time prior to reaching a particular approval process stage, you can assign that stage to a named user. If this is not done, the approval process will go to the Group Tasks list and any user (other than *Distinct User* violations – see section 6.1.2 *Segregation of Duties* above) with the same role in the Community as the role specified for the stage in the approval process will be able to claim the Modeling Task. If a named user was specified, the Modeling Task will be assigned directly to the named user.

With the Group Tasks list option, users get to select what they will be working on (assuming more than one user has the relevant role in the Community) rather than the Modeling Tasks getting assigned to them.

Which option fits better in your organization will depend on your internal processes.

Note that you can use a hybrid approach, assigning named users only for some stages (e.g., Fact Type approval or deployment).

6.1.4 Changing an Approval Process

Once an approval process has been used in a Modeling Task, for auditability, it can no longer be changed.

Instead, clone the approval process to a new one, make the required changes, then inactivate the old approval process – this will not impact any Modeling Tasks already underway but will prevent the old approval process from being available for selection – and activate the new approval process.

6.1.5 Discarding a Modeling Task

A Modeling Task can be discarded when it is in the first stage of its asset approval process. Any Fact Types that have already been approved remain approved. All other assets are deleted.

As a best practice, you should not leave Modeling Tasks hanging in the repository (for example, when assigned to someone who has left the organization). This is especially relevant prior to approval, as this may prevent the creation of assets with the same name in other Modeling Tasks and may cause conflicts. Reassign the Modeling Task, use the Modeling Project Manager overrides or the ability to recall the Modeling Task, and then move the approval process forward to completion or back to the first stage to be discarded.

6.2 Approval Strategies

DM uses a simpler governance mechanism, known as an *approval strategy*, for the review and approval of the following items:

- Model Mappings – review of each Model Mapping request within a particular Model Definition
- Revision Tasks – review of each Revision Task request within a particular Release Project
- Deployments – review of each deployment request to a particular environment

The default approval strategy is *None* which means that each of the above requests is automatically approved.

The other approval strategies that are available are *Any* and *All*. With either of these, you must specify a named list of users (among those that have the necessary permissions). With the *Any* setting, the request is posted to the Group Tasks list for the named users and any one of them can claim the task and approve it. With the *All* setting, the request is posted to each user's My Tasks list and all must approve it in order to proceed.

As with approval processes, you may select the *Distinct User* option to segregate duties so that the user that submitted the request will not be able to approve it.

Like the recommendation for approval processes, the best practice is to have at least one additional person review the more sensitive or impactful of these request types. Since a Model Mapping change will directly impact any subsequent deployment that uses that Model Mapping, it is a good idea to review these requests. This is often done by a more technically oriented Glossary Administrator. If you are managing your deployments to different SDLC environments from within DM, you may want to set up approval for deployments to the higher environments (e.g., Production), but not for the lower environments (e.g., Development). On the other hand, Revision Task requests have little direct impact and therefore their review is often seen as too onerous.

6.3 Repository Tasks

Every Repository Task requires approval. Repository Tasks do not have a *Distinct User* feature, so a user can approve their own requests if they have the appropriate permissions for the Repository Task type. The following types of Repository tasks are available:

- Decision association/dissociation
- Retire/unretire assets
- Retire/unretire Fact Types

The approver can reject individual line items in the Repository Task, but then has to send the entire Repository Task back to the requestor to revise and resubmit.

7 Release Management

7.1 Revision Management Approaches

There are two primary approaches to managing Revisions in DM. In this section we will compare the approaches, but we will not be recommending a best practice as that is project specific.

7.1.1 Latest Version Approach

In the first approach, only the latest version of the relevant Decisions/Flows is included in a Revision. When a new version is ready for deployment, you add the new version and remove the previous one. You need to keep previous deployments to be able to execute prior versions.

When using DE, the latest version can be easily executed by running the *latestTagDetails* REST service or *findLatestTag* SOAP service to get the latest tag for the Release Project and then calling the Decision/Flow by any *effectiveDate*. To execute an earlier version, you will need to know the tag name and you will then be able to call the Decision/Flow by any *effectiveDate*.

When used with the Java Format Adapter, this approach yields a separate deployment package (as a result of separate deployments) for each asset version. Each deployment package will need to be integrated with the calling application.

7.1.2 All Versions Approach

In the second approach, all active versions of the relevant Decisions/Flows are included in one Revision. When a new version is ready for deployment, you only add the new version. With multiple versions in the Revision, the Revision size will be larger, the deployment time will be longer, and the executable will be larger than with the Latest Version approach. There is no need to keep prior deployments to execute prior versions, as all relevant versions are included in the same deployment; you can un-deploy prior DE deployments.

When using DE, any version (that is included in the latest Revision) can be easily executed by running the *latestTagDetails* REST service or *findLatestTag* SOAP service service to get the latest tag for the Release Project and then calling the Decision/Flow by the relevant version or *effectiveDate*.

When used with the Java Format Adapter, this approach produces all the relevant class files in one deployment package. The Non-typed approach (see section 13.1 *Typed vs. Non-typed* below) facilitates calling the Decision/Flow by the relevant version or *effectiveDate*.

As part of the validation done for a Revision, DM checks for overlapping versions of the same Decision/Flow. By default, omitting the effective/expiration dates sets them to 1970-01-01/9999-12-31 respectively. As a result, if you include multiple versions of a Decision/Flow in a Revision, you need to specify effective/expiration dates to avoid the overlap situation. However, if *dd_empty_dates_range= no-date-range* (see the *Decision Manager (DM) Configuration Guide*) is set in the *bdms.cust.properties*

file, omitted effective/expiration dates are ignored for the purposes of the overlap check. When using this option, execution must be done by explicitly specifying the version.

7.2 Revision Size

Removing unnecessary assets from a Revision reduces deployment time and the load on the DM server to generate and compile all the code. As a best practice, each Revision should include only those asset Views and versions that are needed for the selected Revision management approach (see section 7.1 *Revision Management Approaches* above).

When using Flows, there is no need to explicitly include the Decisions that are invoked by the Flow in the Revision unless you also need to invoke them individually by effective date – they can be invoked by version just by including the Flow in the Revision.

In addition, removing unnecessary views and versions reduces the size of the factory method for each asset and creates smaller executables, reducing load time, instantiation time, and memory footprint, improving performance (likely not significantly) for a Decision/Flow.

7.3 Software Development Life Cycle (SDLC)

Clicking the *Deploy* button to deploy a tag generates the source code for the selected environment per the current Model Mapping. If you deploy to an environment and then deploy again at some later time to the same environment or a compatible one (one that has the same format adapter, same format adapter properties, same model definition, and same model version), there is no guarantee that you will get the same code. This can occur because the Model Mapping changed, the environment settings changed, or the DM version changed. The code may also be generated in a different sequence each time it is generated.

For SDLC purposes it is desirable to promote the same code that was tested in a lower environment. To ensure deployment of the identical code assets to another compatible environment (such as the next stage in the SDLC), use one of the following approaches:

- External deployment

  Instead of using a repository adapter to synchronously push deployed artifacts all the way to the execution environment, grab the generated artifacts and then deploy them to each environment with pipeline tools (e.g., Jenkins). For DE, this will involve using its asynchronous deployment services.

- Redeployment

  Instead of regenerating the artifacts to deploy to the next environment, redeploy the artifacts from the original deployment. If the environments, as defined in DM, are compatible and are defined with a repository adapter, you can use the *Redeploy* option on the *Deployment History* screen instead of the *Deploy* option. This will use the previously generated package, updating only the deployment information in the *Release_Information.xml* file. This is the recommended approach if you are using synchronous deployment to DE.

7.4 Tags vs. Snapshots

Snapshots are intended for iterative testing within a branch, using the same deployment package name (with a suffix as configured in the environment settings or the default *-SNAPSHOT* suffix). Iterative deployment is best done with candidate assets (see section 7.6 *Candidate Deployment* below). Once finalized, the assets should be approved, and the branch should be deployed as a Tag. Tags are intended to be immutable.

By default, environments do not accept snapshots. You must enable snapshots for the relevant environments by selecting *Allow Snapshots* in the environment settings. Typically, this would only be set for development environments.

All environments accept Tags.

When using DE, it is important to note that prior to DE V7.1.2.3, deployment of Tags and snapshots did not un-deploy assets when deploying a previously used Tag/snapshot name, possibly resulting in a mix of assets from both deployments. If you wish to reuse a Tag/snapshot name prior to DE V7.1.2.3, it is best to manually un-deploy by calling the *deleteAllArtifactsInTag* DE web service prior to re-deployment. Alternatively, you can manually un-deploy individual DE artifacts by calling the *deleteArtifact* DE web service (as of this writing available in SOAP only) prior to redeployment.

7.5 Tag Naming

Tags names default to a timestamp. This provides a unique name for each tag within a Release Project. Best practice is to just use the default name, although any other naming convention that provides a unique tag name (e.g., sequential release numbers) would work just as well.

Note that the timestamps get normalized and need to be referenced at runtime as *dmmYYYY_MM_DD_HH_MM_SS*.

Also see section 12.4 *Reusing Tags* below on reusing tags in DE.

7.6 Candidate Deployment

For iterative deployment of assets for external testing, the best practice is to use snapshots along with candidate assets. Using candidate assets in your deployment will reduce the number of approved versions in the repository as you will be able to make changes without incrementing the version number.

To change a candidate asset, send the Modeling Task containing the candidate asset back to draft status. When the Modeling Task is re-submitted, its assets will be assigned the same version numbers as before. DM keeps track of these different revisions of the candidate assets. The old revisions will be shown as rejected in any Branch Revisions that included them. Adding the new revision of a candidate asset in a Revision Task will remove the rejected one. Note that you can add other assets (but not versions of rejected revisions) without removing any rejected revisions. You will get a validation warning, but you can proceed, and even deploy the Revision, even though it has rejected assets.

By default, DM is not configured for deployment of candidates. You must enable candidate deployments by selecting *Support candidate in revision* in the DM System Options. You must also enable candidates for the relevant environments by selecting *Allow Candidates* in the environment settings. Typically, this would only be set for development environments.

8 Decision Selection

How do you determine which view and version of a Decision to run? It might be as simple as always run the latest version of the Base view. It could be a little bit more complex, dependent on effective/expiration dates and perhaps a small set of views. Or it may require complex logic to determine the appropriate Decision based on a myriad of conditions including things like jurisdiction, line of business, dates, and much more. This is especially relevant when using de-contextualized view names (see section 4.2.1 *View Naming* above). In this section, we will explore several approaches for selecting a Decision to run.

Although this section focuses on Decisions, the same approaches could also be applied to Flows, with the caveat that Flows do not have views. The selection of a Flow version determines the views and versions that are invoked by that Flow.

8.1.1 Approach 1: Single Version per View

This approach is a good fit if you know the view name – either because you only use the Base view or you can determine the view name simply from the context, say by jurisdiction or line of business – and you only execute the latest version. See section 7.1.1 *Latest Version Approach* above for guidance on how to set up your Revision when using this approach.

8.1.2 Approach 2: Effective Dating

Effective dating is a good approach if you know the view name (as in approach 1), there are multiple versions deployed, and you need to select the version to be executed based on some date. Within the Revision, each asset will have effective and/or expiration dates. At runtime, you will invoke the Decision/Flow by passing the relevant date as the effective date, which will then be used to automatically select the corresponding asset version. See section 7.1.2 *All Versions Approach* above for guidance on how to set up your Revision when using this approach.

A potential downside of this approach is that the effective/expiration dates are only visible within Release Projects, not as part of the logic assets themselves.

8.1.3 Approach 3: Deployment Desciptors

Deployment Descriptors are appropriate when there are multiple criteria evaluated to select the view and version to be executed. The additional criteria are defined as custom metadata, known as *Deployment Descriptors*, added to a Community, and then used in a Revision Task to define the conditions under which an asset version is selected for execution. Revision Task setup is similar to that of approach 2, except you will use the Descriptor View option to specify the *Deployment Descriptor* conditions.

The *Deployment Descriptors* are only visible within Release Projects, not as part of the logic assets themselves. Since they are not subject to the TDM principles, there is limited validation done on the

Deployment Descriptors, specifically looking for overlaps and effectivity date range gaps. Furthermore, there is no way to test the *Deployment Descriptors* within DM.

While the *Deployment Descriptors* are managed in DM and they are included in an XML file within the export package, neither DE nor the Java Format Adapter currently operationalize them. In other words, your code will need to read the XML file, determine the view and version to execute, and then invoke the appropriate Decision.

8.1.4 Approach 4: Selector Decisions

The final approach, *selector decisions*, is appropriate when there are complex selection criteria or there is a requirement to have the selection logic visible to and managed by the business. This is achieved by creating a separate Decision, known as a *selector decision*, per Decision conclusion. One s*elector decision* answers the question of which view and version of the corresponding Decision to execute. The *selector decisions* need to be updated each time there is a new view and/or version of their corresponding Decision, or for any other selection logic change. The *selector decisions* need to be deployed, so they are typically included in the same Revision as the logic assets they select.

A simple convention can be used for naming *selector decisions*: simply append “Selector” to the end of the Decision name. *Selector decisions* always have just one view, typically Base.

Selector decisions return the view and version to execute based on their logic. This would appear to be two conclusions. Decisions can only return one conclusion. This challenge is resolved by realizing that the view and version are simply components of a single conclusion – the namespace of the selected Decision. The convention then is to concatenate the view and version to produce a single conclusion with all the required information. As a best practice, the form “view | version” is used.

As with *Deployment Descriptors*, neither DE nor the Java Format Adapter currently operationalize *selector decisions*. Your code will first need to invoke the applicable *selector decision* (as determined by the business process), then use its conclusion to invoke the appropriate Decision.

Unlike *Deployment Descriptors*, *selector decisions* are proper TDM assets. They are visible to and managed by the business. They can be fully validated and tested within DM.

As a best practice, use separate columns for testing conditions against effective date values and expiration date values. This makes it easy to see which views and versions are currently in effect as their expiration condition column will be blank.

9 Requirements Traceability

A frequently encountered requirement is to be able to trace the source of a particular set of business logic back to its source document, typically a policy or regulation. There are several features within DM that can be used to tie a piece of business logic to a particular section of a document. We will discuss Knowledge Sources in depth as the primary means to provide requirements traceability in DM. We will then review the additional features that can be used for requirements traceability along with the considerations around when to use each feature. Note that in some situations you may need to use a combination of features to provide the desired level of traceability.

9.1 Knowledge Sources

A Knowledge Source is a DM asset that captures the relationship of a Decision or Rule Family with an external artifact, such as a regulatory or policy document, a contract, or a business process diagram for informational purposes, primarily requirements traceability. DM is not intended to be the tool to manage these artifacts – that is best left to document management systems, business process management systems, etc. By defining these artifacts (which are in reality just pointers to the real artifacts) in DM along with their relationships in DM, we are able to answer impact analysis questions such as:

- Where did the logic for this Decision/Rule Family come from?
- The external artifact has been updated. Which DM assets are impacted?

9.1.1 Knowledge Source Types

Each Knowledge Source instance must have a Knowledge Source Type. DM is delivered with one Knowledge Source Type predefined at the WORLD Community level, appropriately called *Knowledge Source*. You may define additional Knowledge Source Types as needed to capture additional information about specific Knowledge Sources by means of custom properties. As a best practice, the set of custom properties should be kept only to what is needed in DM for informational and traceability purposes – the intent is not to manage the external artifact in DM.

If you are defining your own Knowledge Source Types, in which Community should they be defined? The best practice is to define them in the lowest Community that allows them to be accessed from all lower Communities that will be using that Knowledge Source Type (referred to below as *Knowledge Source Type scope*).

A key feature of Knowledge Sources is the ability to deep link directly to the Knowledge Source instance in the external tool. This is enabled by adding a *Location URI* custom property to the Knowledge Source Type. We recommend adding this custom property to all Knowledge Source Types that are used to represent external assets that support deep links. Note that the custom property name must be exactly *Location URI*.

Custom properties are displayed alphabetically. If you want them to appear in a particular order, you must number them as a prefix to their names (except for *Location URI*).

9.1.2 Knowledge Source Instances

A Knowledge Source instance represents a specific external artifact.

9.1.2.1 Knowledge Source Name

Use a business-friendly name for the Knowledge Source to aid with understandability, reusability, and traceability. Avoid technical terms and abbreviations in the name, unless well understood in your organization.

The name must be unique across the entire repository.

9.1.2.2 Version Identifier

The *Version identifier* is intended to represent the external version identification of the Knowledge Source. It can be any text, not just a number. DM maintains an internal version number, not visible in the UI, for its own versioning purposes.

Any updates to versioned Knowledge Source data (include the *Knowledge Source name*, *Version identifier*, and any Knowledge Source Type custom property that is marked as *versionable*) require a change to the Version Identifier.

9.1.2.3 Description

All Knowledge Sources should have a clear description. This aids in the understanding of what the Knowledge Source represents.

9.1.2.4 Community

Knowledge Sources reside directly in a Community (unlike Decisions which reside in folders within a Community) and may be defined in any Community on the branch from the Community where the Knowledge Source Type is defined to the Community of the Modeling Task in which the Knowledge Source is created.

As a best practice, create a Knowledge Source in the lowest Community that allows for the desired level of sharing of that Knowledge Source and is within the scope of the Knowledge Source Type. Note that there currently isn’t an option to move a Knowledge Source.

9.1.2.5 Location URI

When the tool hosting the external asset supports deep links, we recommend specifying the deep link in *Location URI* custom property to provide a convenient way to access the external asset directly from DM.

9.1.3 Knowledge Source Associations

The value of having a Knowledge Source in DM comes from its associations to Decisions and Rule Families. It is these associations that enable impact analysis.

There is no point to defining a Knowledge Source in DM if it will not have any associations.

Add only those Knowledge Source associations that represent true associations. Extraneous associations dilute the value of the traceability and of the impact analysis that can be done for the related Knowledge Source.

9.2 Additional Options for Requirements Traceability

One of the most critical considerations for requirements traceability is the desired level of granularity. Is it sufficient to provide traceability at the Flow level, Decision level, or the Rule Family table level? Or is traceability needed down to the Rule Family row level, column level, or even to the cell level?

How easily can a viewer see that there is requirements traceability information?

Another critical consideration is whether there is a need to search for all the items associated with a given source?

Is the requirements traceability information considered to be part of the business logic? If so, then perhaps the traceability information would need to be governed together with the business logic, exported along with the business logic, or included in the execution output such that conclusions will be presented along with references to the source that led to those conclusions.

The following table summarizes the key considerations and how they are supported by the various features that can be used for requirements traceability:

Feature	Applicable to						Search for Associated Assets	Visibility	Appear in Export	Appear in Execution Output	Governance
	Flow	Decision	RF Table	RF Row	RF Column	RF Cell					
Notes	✓	✓	✓	✓	✓	✓		RF Table, Additional Info			Along with Flow/Decision/RF
Messages				✓			Via Advanced Query	RF Table	✓	✓	Along with RF
Custom Properties	✓	✓	✓	✓			Via Advanced Query	Additional Info, manifest files			Along with Flow/Decision/RF
Knowledge Sources		✓	✓				✓	Decision diagram	JSON only		Independent

Notes are part of the versioned information of an asset. They are immutable, once approved. Coupled with the inability to search notes, this feature is usually less preferable than the others, unless column-level or cell-level traceability is required.

Unlike the other requirements traceability features, Knowledge Sources are independently governed objects.

10 Communities

DM's Communities are used to structure the repository, control access, provide terminology, and provide Community-level settings such as approval processes, Message Categories, Views, and deployment environments.

Different asset types reside in different places in the Community hierarchy:

- Fact Types reside in a Glossary associated with a Community. Upon creation, Fact Types are placed in the lowest Glossary on the branch from the WORLD Community to the Community of the Modeling Task in which the Fact Type is created.
- Decisions reside in a folder within a lowest level Community. Initially they are defined in the Community of the Modeling Task in which they are created, but they may be associated/dissociated with others.
- Flows reside directly in a lowest level Community, the same one as the Community of the Modeling Task in which they are defined.
- Knowledge Sources reside directly in a Community and may be defined in any Community on the branch from the Community where the Knowledge Source Type is defined to the Community of the Modeling Task in which the Knowledge Source is created.

In this chapter, we'll explore best practices for structuring the Community hierarchy and for naming Communities and the folders they contain.

10.1 Community Hierarchy

Establishing the Community hierarchy within DM is an important task that impacts the long-term maintainability of the repository. It is made more challenging by the fact that changing the Community structure is difficult once a Community has content within it. It is best to build a Community hierarchy that will remain relatively stable in the face of change.

When should you create a Community and when should you create a folder? The primary driver toward creating a Community is access privileges. If the content needs distinct access rights for read and/or write than other content, then it must be in a separate Community as that is the only vehicle in DM to control access. Conversely, fewer Communities increase shareability, but might make ownership less clear. In general, use folders, unless different access privileges are required.

Flows add an additional consideration in that Flows can only use Decisions that are attached to the same Community as the Flow. If there will be a need to have Flows in different Communities use the same Decisions, then combine the Communities if possible or associate the Decisions with both Communities.

By convention, the top-level Community is always WORLD. In general, nothing should be defined at the WORLD level.

Immediately beneath WORLD, the convention is to have a Community for the organization. This allows for the possibility of merging repositories from different organizations (currently supported only via export/import of assets).

The third level and beyond is where the Community hierarchy gets interesting. There are multiple approaches to structuring the hierarchy with the most common ones being by organizational structure, function, or product. The relative importance and stability of these various dimensions varies in each organization. In many enterprises, the organizational structure changes fairly frequently so in such enterprises, a more stable hierarchy approach is needed. A functional or product-based approach generally yields the desired stability. It is important to note that the approach does not have to be uniform across the entire Community hierarchy – branches can be structured by different dimensions, yielding a hybrid approach.

10.2 Community Naming

For ease of identification, Community names and the folder names within the Communities should be unique across the entire repository.

The recommended naming convention is to use a "Community" suffix for Community names, e.g., *Sample Community*, and a "Folder" suffix for folder names, e.g., *Sample Folder*.

Consider an insurance company with multiple lines of business. Each could have a Community called *Underwriting*. Since there are many places in DM where you see only the name of the lowest Community, there would not be enough information to easily understand the context of the asset being looked at. By adding the line of business name to the Community name (e.g., *Personal Auto Underwriting Community*), the name has full context and becomes unique in the repository.

10.3 Archiving Communities

DM does not offer a way to archive Communities that are no longer needed. Instead, perform the following steps:

1. Retire Knowledge Sources that are no longer needed.
2. Retire Flows that are no longer needed.
3. Retire Decisions that are no longer needed.
4. Retire unused (except in retired assets) Fact Types.
5. Move remaining Fact Types to other Glossaries as appropriate.
6. Associate Decisions with other Communities as appropriate.
7. Dissociate Decisions that were associated with another Community and are not used in Flows.
8. Export Flows and import into other Communities as appropriate.
9. Remove user access privileges to the Community.

a. Remove users directly associated with the Community.

b. Turn off the *Inherit User Access* setting for the Community.

10. Rename the Community to a name indicating that it should not be used. Best practice is to start the new name with “zz” so that it appears at the end of any alphabetized list of Communities for any users that still have access. You may also want to include “DO NOT USE” as part of the name.

11 Authorization

Authorization in DM is handled by a Role-Based Access Control (RBAC) mechanism. DM has about 150 Permissions which are assembled into Permission Groups, which are then assembled into Roles. Users are assigned one or more Roles globally (at the user-level) and then assigned a subset of their Roles in each Community in which they require access. It is important to note that some permissions are global in nature: if assigned to a user, they apply to all Communities regardless of assigned Roles. The most notable example of this is FT_WRITE which is normally assigned as part of the Decision Analyst Role for them to be able to create and edit Fact Types. Since this permission is global, within a Modeling Task in Community A, the Decision Analyst can create draft versions of Fact Types that are defined in any Community.

DM comes with several Permission Groups and Roles pre-configured. The Release Notes include a Permissions Matrix where you can review the pre-configured authorization setup. You can create your own Permission Groups and Roles as needed.

It is not recommended to use the provided Permission Groups and Roles directly – instead create and use your own. The reason for this is that with an upgrade to a new DM version, the provided Permission Groups and Roles may change. Using your own gives you full control over the provisioning of your users. If you do use the provided Permission Groups and Roles, we recommend using them as-is. This is to make sure you do not get locked out of being able to make a necessary change in the system.

It is good practice to always have at least one administrator user at the WORLD level to ensure you have full control over the application.

One very common practice is to assign a user multiple Roles. This typically results in a user getting the same permissions multiple times. In extreme cases, the user is assigned the *Super User* Role along with other Roles. Having the same permission multiple times adversely impacts login time and ongoing performance within DM as the permissions structure is much larger than necessary. You can use the *User Permission Summary* and *User Permission Detail* reports in DM to review how many times a user has each permission. The count should be as small as possible, ideally one. To reduce the count, eliminate any unnecessary Roles from the user definition. The only Roles that must appear in the user definition are Roles that add permissions the user needs but would not otherwise have and Roles that the user needs to be able to progress the Approval Processes (typically *Decision Analyst*, *Decision Peer Reviewer*, *Business Owner Approver*, and *Glossary Administrator*) – all the rest are redundant and should be removed. For example, if the user has the *Super User* Role, you can eliminate ALL other Roles, except those needed for Approval Processes in which the user participates.

As a wide-spread best practice, the principle of least privilege should be applied: a user should have the minimal permissions required to perform their duties. In general, assignment of the *Super User* Role should be avoided.

12 Decision Execution (DE)

12.1 Optimizing Run-time Performance

When looking at the roundtrip execution time for a Decision or a Flow, it is typical to see that a significant portion of that roundtrip is spent marshalling the data in and out and in the propagation delay to and from the DE server. The actual execution time – typically no more than a few milliseconds for a large Decision – is usually a small portion of the total roundtrip time. For this discussion, we will exclude any optimization to the Decisions and Flows themselves as that is not where the majority of the time is spent.

Since DE is web-service-based, to optimize run-time performance for a given Decision or Flow execution, less is more. In other words, the smaller the input message size and the smaller the output size, the better the throughput and scalability will be.

DE supports both SOAP (over XML) and RESTful (over JSON) calls. DE JSON messages are dramatically smaller (roughly 1/3 the size) than their equivalent DE XML messages. As a performance best practice, we recommend using DE with RESTful calls.

12.1.1 Input Message Size

To optimize run-time performance, remove anything extraneous from the input message. We'll discuss what can be done in the *executionInput* and *executionConfiguration* sections of the input message.

12.1.1.1 executionInput

To reduce the input size, only provide those input Fact Types and values and *executionKeyValues* that are needed for the execution of the specific Decision or Flow. While it is certainly easier from an integration point of view to use the same data structure for the execution of different Decisions and Flows (see section 14.1 *Message Contract* below), this will not yield the best performance.

For a Decision execution, include only persistent Fact Types.

For a Flow execution, include only persistent Fact Types. Note that some Fact Types may be marked as both interim and persistent and you will need to determine if they, in fact, need to be provided in the input message.

Each Fact Type should include only its name and value(s).

If your *executionInput* was created from a Test Case export in XML format, you should omit all the following:

- Conclusion Fact Types
- *isConclusionValues="false"* attributes
- *<resultComparison>* tags and everything they contain

- *<executionKeyValues>* tags and everything they contain

12.1.1.2 executionConfiguration

You can reduce the input message size by omitting any configuration options that are set to their default value. You can even omit the *executionConfiguration* section altogether if all configuration options are set to their default values.

You should also omit any configuration options that are not needed for production execution. One such configuration option that is commonly used with manually crafted test input messages is *InputMustBelongToFactTypesInManifest=TRUE*. Checking that each Fact Type that is provided in the input is found in the manifest for each production input message increases CPU consumption unnecessarily so this option should be omitted from production messages.

12.1.2 Output Message Size

Many of the *executionConfiguration* settings impact the verbosity of the output message size which in turn could have a large impact on the roundtrip execution time. In general, you should use the *executionConfiguration* options that return the minimal output that contains the execution results you need.

For a Decision, use these options to return the minimal output:

- *DecisionViewFactTypeFetchStrategy=NONE* (default is *ALL*)
- *FactTypeMessageFetchStrategy=NONE* (default is *ALL*)

For a Flow, use these options to return the minimal output:

- *DecisionViewFactTypeFetchStrategy=NONE* (default is *ALL*)
- *FactTypeMessageFetchStrategy=NONE* (default is *ALL*)
- *FlowFactTypeFetchStrategy=NONE* (default is *ALL*)

To quantify the impact of these settings, we've seen decreases of over 90% in the output size going from the default options to the minimal options. Even more importantly, this improves throughput. In a benchmark performed by Sapiens, the number of Decisions executed per second increased by over 66% when using the minimal options vs. the default options.

If the above options do not produce the desired output, find the combination of options that produces the desired output with the minimal output size. These options may need to be set separately for each Decision or Flow depending on the output requirements.

In addition, you can use the *CategoryName* option along with *CategoryNamesStrategy=EXCLUDE* or *INCLUDE* to further reduce the output size by filtering the Message Categories that are returned.

To reduce the output size even further, you might want to consider enabling HTTP response compression in Tomcat. While this increases CPU load on both ends, it decreases network traffic resulting in improved performance when the network is a bottleneck due to congestion and/or

propagation delay. In a benchmark performed by Sapiens, the throughput (measured by number of Decisions executed per second) increased by over 18% just by enabling HTTP response compression.

12.1.3 Version or Effective Date

When a Decision or Flow is called, DE calls a factory method to determine the class to invoke. The factory method complexity increases with the number of views and versions included in the same deployment package.

When calling by effective date, the factory method needs to determine the version to invoke, based on the provided date. This logic is a bit more involved than when calling by version directly rather than effective date. Calling by version may improve run-time performance by an insignificant amount for each call but will require external orchestration to select the appropriate version to execute.

As a best practice, minimize the size of the factory methods by removing unnecessary views and versions from the deployment package. This will decrease deployment time and execution time.

12.1.4 Number of Calls

Reducing the number of calls made to DE by combining multiple calls into one call improves the overall performance by authenticating just once, by performing data marshalling just once on each end, by incurring roundtrip propagation delays just once, and by reducing overall network traffic. This can be achieved by combining multiple Decisions into a single Flow and by batching multiple executions into a single service call.

12.1.4.1 Combining Decisions into Flows

If possible, combine multiple Decisions into a single Flow to do one web service call for the Flow rather than one for each Decision. This is possible if the data structures used by the Decisions are compatible – meaning they are all subsets of a larger common data structure – and if the input required for all the Decisions is available up front.

12.1.4.2 Batching Calls

DE provides batch execution services where you can group multiple sets of input Fact Types into a single service call.

The question then becomes how big to make each chunk?

If you are using synchronous web services, you need to ensure that the entire input message is processed and returns the response message before the web service call times out. The timeout duration is typically set systemwide but often defaults to 300 seconds. You will want to leave a cushion for variations in the input messages, network congestion, etc. Customers typically use chunk sizes of 50 – 100.

Alternatively, you can eliminate the timeout by using an asynchronous DE web service in conjunction with a callback service or with the publish-subscribe mechanism.

In any case, the available memory (heap size) imposes an upper limit on the chunk size.

12.1.5 Authentication

The selected authentication method has a large impact on performance.

Unsecured mode is the fastest as no authentication/authorization is done. If access to the DE server is restricted by other means (such as through an API gateway), *unsecured* mode may be a viable alternative.

OAuth 2.0 is nearly as fast as *unsecured* mode for each message but does have an extra overhead of obtaining a token. Each token, however, can be used in multiple requests, until it expires. Note that OAuth 2.0 is supported in DE V7.2.3.2 and above.

Basic authentication is the slowest, adding about 60 milliseconds to the processing of each message. It also consumes the most CPU on the DE server.

We recommend OAuth 2.0 for the best combination of performance and security, particularly in high throughput applications.

12.2 Authorization

DE has three predefined user roles. Users can have any combination of these:

- ADMIN – provides ability to execute administrative services such as deploying and undeploying Decisions and Flows
- MONITORING – provides access to DE monitoring
- USER – provides ability to execute Decisions and Flows

DE does not further restrict authenticated users. This means that any user-ID with the USER role can execute any deployed Decision or Flow. If you need to restrict user-IDs to particular artifacts, you will need to use a third-party API gateway tool that can inspect the messages and act based upon the content. You will also need to prevent access to the DE server directly, for example by allow-listing only the applicable DM server(s) for deployment and the API gateway.

As a best practice, separation of duties should be applied, and a single user-ID should not have the ADMIN role and the USER role.

When using DE with basic authentication, the user-IDs and roles are defined in a properties file. Any changes to this file require a server restart. Keeping in mind that DE user-IDs are system users, not end users, and to minimize the size of the properties file and its ongoing maintenance, and provide the minimal authentication time, we recommend defining only two user-IDs, one with the ADMIN and MONITORING roles, one with the USER role. Audit requirements can be satisfied by other means, such as providing the information required for audit via *executionKeyValues* (see section 12.3 *Extending DE Output* below).

12.3 Extending DE Output

DE's output can be extended by including pass-through values. The pass-through values are included in the input message as *executionKeyValues* tags. The *executionKeyValues* can appear at any level in the BIM.

This can be used to pass data that would normally not be included in the logic but is needed in the response message. One common example is identifying information such as *Applicant Name*. Other examples include custom data for analytics and traceability.

To optimize performance, minimize the input message size by passing only the least additional data that is needed.

12.4 Reusing Tags

Although reusing a Tag is possible, it is not recommended prior to DE V7.1.2.3 since those versions of DE did not un-deploy the old Tag first, possibly resulting in a mix of assets from both deployments of the Tag. If you wish to reuse a Tag prior to DE V7.1.2.3, it is best to manually un-deploy the old Tag by calling the *deleteAllArtifactsInTag* DE web service prior to deploying the new Tag. Alternatively, you can manually un-deploy individual DE artifacts by calling the *deleteArtifact* DE web service (currently available in SOAP only) prior to deploying the new Tag.

12.5 Repository Discriminator

The DE Repository Adapter has a repository discriminator feature that appends the Environment Name to the Release Project's name in the deployed package. This segregates deployments of the same Tag to multiple environments by giving them a distinct namespace, essentially partitioning the DE server, and enabling one DE instance (or cluster) to service multiple environments.

As a best practice, the repository discriminator feature should not be used with production (or production-like) DE environments as its use can increase memory consumption significantly due to the same asset getting loaded into memory multiple times – one copy for each namespace.

The use of the repository discriminator feature is recommended to reduce the number of non-production DE servers required. Instead of provisioning and maintaining one server instance (or cluster) for each environment (e.g., DEV, UAT, Hotfix, etc.), a single server instance (or cluster) can be used.

13 Java Format Adapter

13.1 Typed vs. Non-typed

The Java Reference Code supplied by Sapiens includes multiple source code examples for the execution of the Java code generated by the Java Format Adapter. The examples are classified as Typed or Non-typed.

The Typed examples are so named because the code references the specific assets deployed from DM and uses method names that are specific to those assets:

```
import com.sapiens.bdms.views.FL.dmm1_1.PolicyRenewalMethod;


PolicyRenewalMethod policyRenewalMethod = new PolicyRenewalMethod();
policyRenewalMethod.setFloodZone("Y");
```

The Non-typed examples use generic objects and methods, not specific to the assets being referenced:

```
import com.sapiens.bdms.java.exe.helper.base.Decision;


Decision policyRenewalMethod = Decisions.newDecision("PolicyRenewalMethod", "FL", "1.1");
Map<String, Object> input = new HashMap<>();
input.put("FloodZone", "Y");
policyRenewalMethod.setFactTypes(input);
```

The code snippets shown above are functionally identical – they both instantiate a Decision object for *Policy Renewal Method (View: FL) V1.1* and set the value “Y” in the *FloodZone* (mapped name) Fact Type.

The Typed approach is easier to read and understand and has the advantage that many errors (e.g., misspelling a Fact Type name) are caught at compile time. The Non-typed approach features dynamic binding without the need for any code changes and is therefore better suited for generic programming of wrappers and frameworks. It has the disadvantage that many errors (e.g., misspelling a Fact Type name) are not caught until run-time either as exceptions or incorrect results.

The Non-typed approach utilizes either Java Reflection for class instantiation or dynamic class loading and instantiation. In most cases, the static binding of the Typed approach will yield better performance. This can be mitigated by reusing the Decision/Flow objects, after using the *clear()* method to remove all data from the previous execution.

The best practice then is to start with the Typed approach for simplicity and performance and then move to the Non-typed approach for dynamic implementations where new versions are frequently released or the input/output requirements change often (see section 14.2 *Reading the Manifest Dynamically* below).

13.2 Optimizing Run-time Performance

In this section we will look at optimizing the run-time performance of interacting with code generated by the Java Format Adapter. For this discussion, we will exclude any optimization to the Decisions and Flows themselves.

The instantiation of the Decision and Flow objects is relatively expensive. In most cases, the static binding of the Typed approach will yield better performance than the Java Reflection or dynamic class loading and instantiation used in the Non-typed approach (see section 13.1 *Typed vs. Non-typed* above for a discussion of non-performance considerations). High-throughput/low-latency implementations should maintain a pool of Decision/Flow objects for reuse instead of instantiating a new one each time.

In the Non-typed approach, input is provided via a HashMap. To optimize performance, include only the minimal input to run the Decision/Flow. For an individual Decision (but not for a Flow), this HashMap is then processed by calling the individual Fact Type setter methods as needed. From a performance point of view, when executing a single Decision, it is faster to just use the setter methods directly as in the Typed approach.

It is usually sufficient to see the execution results (final Fact Type values and all messages) for a Flow at the end of the execution; there is typically no need to examine the results of each Decision task (Fact Type values and messages). Assuming this is the case, the execution should be run with trace turned off (e.g., *policyRenewalMethodFlow.setEnableTrace(false);*) – the default – and the output should be read directly from the *factTypeMembers* HashMap within the Flow object, rather than looping through the individual *TaskNode* objects in the *executablesTrace* object.

13.3 Events

The code generated by the Java Format Adapter can generate the following events:

- *DvNotFoundEvent*
- *ExecutionFinishedEvent*
- *ExecutionStartedEvent*
- *NoRuleFamilyRowHitEvent*
- *RowConflictEvent*
- *RowHitEvent*

Please see the Java Reference Code supplied by Sapiens for an example of each of these and how to implement an *EventListener*.

Two of these events are of particular usefulness and will be discussed below.

13.3.1 RowConflictEvent

Multiple rows can execute in a Rule Family. If the conclusion operator is *Is* then all the executed rows are supposed to have the same conclusion value, with the result being one conclusion value plus any messages from all the rows that executed. However, it is possible that the executed rows return different values. This is known as a conflict and while the possibility that it will occur is reported by the validation function within DM, it is only a warning, and the models can be approved and deployed without correcting this.

The default execution of a Decision/Flow in Java does not catch conflicts. In other words, the Rule Family will randomly return one of the values with no indication of anything amiss. To catch conflicts, you must implement an *EventListener* and listen for a *RowConflictEvent*. It is a good idea to always do this as there is no other indication of a problem.

13.3.2 NoRuleFamilyRowHitEvent

Per the TDM principles, Rule Families are supposed to be complete, that is, return at least one conclusion value for any set of valid input values. When a Rule Family does not return any conclusion value for some of input values, the resulting null value may propagate through the Decision and even through the Flow, making it difficult to track down amongst a flood of null values.

The *NoRuleFamilyRowHitEvent* helps track these incomplete Rule Families down to assist in the determination of whether there is a problem with the data or a logic gap. The event provides detailed diagnostic information including identifying information for the Decision and Rule Family, the BIM Entity, and the current values of all participating Fact Types. This makes for quick work of an otherwise tedious task.

Note that some Rule Families may be intentionally incomplete, for example as part of the Else Pattern (see section 4.13.2.1 *Else Pattern* above). These would be caught by the *NoRuleFamilyRowHitEvent* as false alarms.

13.4 Null-safe

The Java Format Adapter has a setting to specify whether the code generated for function calls checks for null values prior to calling a function. For more information, please see section 4.12.2 *Null-Safe Execution* above.

13.5 Source Code

The Java Format Adapter has an option to include the generated source code in the output package. This code is meant as a reference only. It is not intended to be recompiled or changed as changing the generated source code breaks the paradigm of using Decision to manage the business logic; any issues should be corrected by changing the logic models themselves and regenerating the code.

14 Integration

14.1 Message Contract

There are three approaches to making Decision/Flow execution calls:

- Static with asset-specific input message
- Static with generic input message
- Dynamic

In the first approach, each executable asset (Flow version or Decision version) is built to its own input requirements. The data preparation and the execution output handling in the calling application are hard coded to the specific structure required for that executable asset. Manual coding changes may be required to integrate any message contract changes in a new version of the executable asset.

In the second approach, a generic input message is used, passing all the potentially relevant data related to the subject of the executable asset. In an insurance application, for example, this could be all the policy data for a particular policy number. All assets that deal with the same subject would typically be built to align with the same generic input structure, but each asset would include only those BIM Entities and Fact Types that are needed for its business logic, a subset of the full input structure. New versions of an asset will require integration coding changes only if they need data that isn't already in the generic input message, or if the output changed (for example, due to new Message Categories or new Decisions in a Flow). Compared to the first approach, the initial setup would require more effort to deal with the larger input structure, but subsequent contract changes would require smaller effort, often none. Furthermore, the same code could be used for all assets that share the same input message, greatly simplifying the integration effort. The downside to this approach is the performance hit of sending data, perhaps lots of data, that is not needed to execute the particular asset.

The third approach uses the manifest file to provide the details of the message contract (see section 14.2 *Reading the Manifest Dynamically* below). The calling application reads the manifest file to determine the required inputs along with where to get them and the outputs along with where to put them. This approach requires a large upfront effort to build the infrastructure to process the manifest files, but once that is done, nearly all executable asset versions can be deployed with no coding changes. Furthermore, the same infrastructure can be leveraged for any executable asset. When using the Java Format Adapter, the dynamic approach works best in conjunction with non-typed calls (see 13.1 *Typed vs. Non-typed* above). The dynamic approach is especially useful for batch implementations or when the knowledge of the message contract can be cached, rather than recreated in each call.

When deploying a new version of an executable asset, it is important to know whether the message contract changed from version to version. This can be determined by comparing the *executionModelHash* from each version's manifest file. If the hashes match, that means that there were no message contract changes, only logic changes.

14.2 Reading the Manifest Dynamically

DM produces a manifest file to describe the message contract for each executable asset. The manifest file describes the BIM structure used by the executable asset and for each participating Fact Type describes its name, data type, allowed values, Model Mapping, whether it is a List Fact Type, and whether it is used as a persistent (input) or an interim (output) Fact Type. The manifest file also includes custom properties for Flows, Decisions, Fact Types, Fact Type Model Mappings, and Message Categories. As we'll see below these custom properties can provide additional information which is critical to being able to integrate with an executable asset dynamically, in particular where to get input and write results.

When using DE, the calling application can determine the message contract to use for a particular request by making a total of three calls:

- *latestTagDetails* (REST) or *findLatestTag* (SOAP) to get the latest tag for the Release Project
- *manifest* (REST) or *getDecisionManifest* (SOAP) to get the manifest file for the executable asset within the specified tag
- execute

The first two calls need only be done once per deployment and then cached by the calling application.

When using the Java Format Adapter, the XML format manifest file can be included in the deployment package by setting the *Include Manifest* option to *Yes* in the format adapter settings for the deployment environment.

14.2.1 Manifest Files for a Decision

All Fact Types that have *isPersistent=true* are inputs.

All Fact Types that have *isConclusion=true* are outputs.

14.2.2 Manifest Files for a Flow

All Fact Types that have *isPersistent=true isConclusion=false* are inputs.

All Fact Types that have *isPersistent=false isConclusion=true* are outputs.

Fact Types that have *isPersistent=true isConclusion=true* are ones that DM cannot currently determine for certain whether they are inputs or outputs. They are inputs in one or more Decisions in the Flow and/or used as gateway Fact Types. They are also outputs of at least one Rule Family in the Flow.

There are 2 cases where this might happen:

- The Fact Type is consumed as an input BEFORE its value is set in a Rule Family
- The Fact Type is consumed as an input AFTER its value is set in a Rule Family

In the latter case, it is not really an input Fact Type from the Flow point of view and should not be provided.

It is impossible to determine which case applies from the standard manifest alone and some additional information is required. This can be provided within the manifest by means of Fact Type or Model Mapping custom properties to specify how the Fact Type is being used, whether it is an input or an output.

14.2.3 Manifest File Custom Properties

The manifest file can be enriched with custom properties to inform the calling application where to get input, which outputs are of interest, and where to write them. This is usually done via Model Mapping custom properties but can be done at the Fact Type level. The custom properties enable a high degree of automation of the message contract processing, moving the effort from code to properties after the initial investment of building the code to process the properties.

As discussed in section 14.2.2 *Manifest Files for a Flow* above, a custom property is needed to specify categorically whether a Fact Type is an input or an output for the Flow. We'll refer to this custom property as *Fact Type Usage*.

For input Fact Types, one or more custom properties are needed to specify where to get the input. These can include things like:

- Resource Type (e.g., database, XML message, JSON message, etc.)
- Database name
- Database table
- Column name (alternatively, use the Fact Type mapped name)
- Join information
- XPath
- JSONPath

These same custom properties could be used for output Fact Types. However, since it is likely that not all output Fact Types will need to be persisted, another custom property (or value of the *Fact Type Usage*) will be needed to indicate which output Fact Types are of interest and which are to be ignored.

Decision model output often also includes messages. These should be handled similarly to output Fact Types by using Message Category custom properties like those described above to specify which Message Categories are of interest and where to write them.

14.3 Integration with Flows

For consistent integration consider deploying only Flows, wrapping even a stand-alone Decision as a Flow. This provides a single integration pattern and provides for maximum flexibility as additional Decisions can be added to the Flow with minimal impact – if any – on the integration.

15 User-Defined Functions (UDFs)

15.1 When Should You Use a UDF?

UDFs are a great way to extend the functionality of DM. When is it appropriate to use a UDF?

As with decision logic, logic that the business is interested in managing and logic that changes frequently should be modeled within DM, not buried in the code of a UDF. The converse is also true. Logic that can be modeled in DM shouldn't necessarily be in DM. For example, validating whether a particular string represents a valid date is of no interest to the business and will never change. The business is only concerned with the final outcome – valid date or not – and not with how that conclusion is reached.

Furthermore, handling the date validation as an *IsDate* UDF has a significant additional benefit: the UDF function is not dependent on any particular Fact Types as a decision model would be, and therefore the UDF can be used to validate any date string – it is reusable.

As a best practice UDFs should be generic pieces of code that can be readily reused.

15.2 UDF Naming and Packaging

A UDF name should be descriptive so that it is easy for the user to understand what the function does. Where an equivalent Excel (or any other commonly used tool) function exists, use the Excel function name.

For packaging UDFs, Sapiens recommends following the same best practices that Sapiens uses in its hosting operations:

1. All UDFs should be in a Java package named: *com.<customername>.decision.custom.functions*
2. All UDFs should be placed in a single jar file
3. The jar file should be versioned.
4. The jar file should be named: *custom-functions-<customername>-<nnn>.jar* (where *nnn* is the version number of the jar file)

15.3 What Can Be Done in a UDF?

There are only a few limitations placed on what can be done in a UDF. Best practices provide a guide as to what should and shouldn't be done in a UDF.

A UDF should be written to minimize its performance impact. Database I/O and service calls could have significant performance implications relative to decision model execution and are discouraged. As a mitigation, minimize the calls to the UDF within the same Decision/Flow. See the discussion on lookup functions in section 4.6 *Lookup Tables* above.

A UDF should not introduce dependencies that make the determination of the decision model conclusion dependent on anything other than the inputs provided to the decision model. This could make the decision model *non-deterministic* (see section 4.21 *Modeling for Deterministic Execution* above) and make testing more difficult.

A UDF should be self-contained in terms of its error-handling. The UDF should always return a result for all inputs and handle any errors that may occur. This will ensure that all Decisions calling this UDF can successfully execute to completion.

15.4 UDF Method Signature

A UDF must satisfy the Java method signature requirements so that it can be called as a function from Decision.

The variables used in a UDF method signature must map to Decision's data types and their cardinality (single value vs. list). This mapping varies based on settings related to the representation Decision uses for numeric values (BigDecimal vs. Double) and date values (Date vs. Long). For more information, see *Appendix: Mapping of Decision Data Types to Java Data Types* in the *DM Adapters Guide*. Use Object… as the input specification for functions that can take an infinite number of values as a mix of single values and lists.

When writing UDFs for general use, it is recommended to use generic methods and/or overload the methods to support all relevant settings variations.

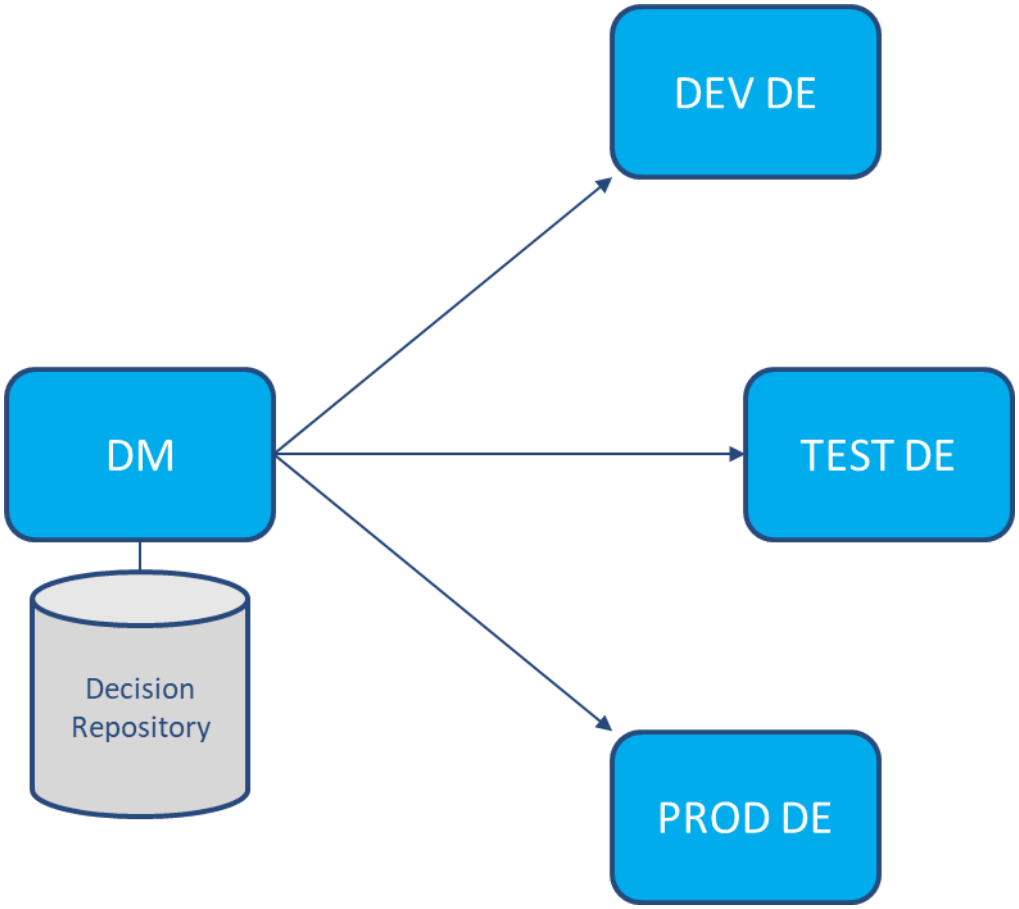
15.5 Java Version Compatibility

Prior to DM V7.7, all UDFs have to be compiled so as to be compatible with Java 8. Either compile with a Java 8 JDK or use javac --release 8.

As of DM V7.7, UDFs must be compiled so as to be compatible with Java 17 for use with DM and DE. If the *Java Compilation Version* in your Java Format Adapter settings is set to 8, you will need a version of the UDF jar that is compatible with Java 8 for use with your Java Format Adapter deployed code.

16 Working with Multiple DM Repositories

DM is designed to be used as a single repository for all software development life cycle (SDLC) activities. As shown in the diagram below, the single DM repository is the source for deployments to the various DE environments.



This architecture provides a comprehensive audit trail and helps ensure that the same assets that were deployed to one environment, are deployed to the other environments as well. See chapter 7 *Release Management* above for additional measures to ensure deployment of the same assets.

Other DM environments can be used in parallel to the above “production” repository for various non-SDLC uses, such as training, proofs of concept, or testing new DM versions.

Sapiens does not recommend exporting from one DM repository and importing into another as part of the normal SDLC process; all SDLC-related work should be done in one DM repository.

17 Minimized Input Optimization (MIO) API

The MIO API is an alternative execution algorithm for Decisions. It provides an API for an interactive user interface to request the next question(s) – each representing an input Fact Type – to be displayed to the user. The MIO API selects which question(s) to ask next to provide the optimal user experience, that is, ask the user the fewest possible number of questions to reach a Decision conclusion. It does this by examining the declarative business logic expressed in a Decision and determining which question(s), given particular responses to each, would yield a Decision conclusion. If the responses received are not the ones that yield a conclusion right away, the process iterates using all the received inputs from previous iterations to determine the next set of questions to ask.

The MIO API enables automated UI generation, replacing scripted questionnaire logic with dynamic scripting based on a Decision. It is a great fit for interactive questionnaires such as insurance claims, loan applications, product selection, etc.

17.1 Using the MIO API with a Decision

When executing a Decision with the MIO API, the algorithm attempts to execute the Decision with the provided data, if any. Rule Family rows are skipped if data was not provided for one or more of its Fact Types. This differs from the standard execution algorithm which always executes all rows and produces all applicable messages. As a result, you may get fewer messages when executing with the MIO API than with the standard execution algorithm.

If a conclusion is not reachable with the provided data, the MIO API algorithm examines each row of the Decision Rule Family to see how many additional Fact Types are required to be able to execute that row and reach a conclusion, assuming values that enable that row to fire are provided. If any of the identified Fact Types have supporting Rule Families, they are not counted. Instead, the algorithm recurses into the supporting Rule Family, looking at each row, counting required Fact Types for each, and recursing into additional supporting Rule Families as needed. Within each Rule Family, the algorithm selects the row that requires the fewest additional Fact Types (including those from the supporting Rule Families). In case of a tie, the row with the lower sequence number – per the saved Perspective at deployment time – is selected. The set of additional Fact Types (including those from the supporting Rule Families) for the selected Decision Rule Family row is then provided as the output for this MIO API execution.

The process iterates by invoking the MIO API again, with a new set of data, ideally including the additional Fact Types requested. Note that the algorithm is stateless and will run with any set of input data – it knows only what was provided to it within the current execution.

As a best practice, model your Decisions as you normally would. Let the Decision logic drive the order of the questions, rather than the other way around. This will result in easier to understand Decisions.

To refine the order of the questions, you have the following options:

- Where there are ties, the order of the questions can be modified by changing the order of the rows in the various Rule Families so that rows to be fired first are above other rows in the Rule Family, saving the Perspective for each Rule Family, and regenerating the deployed code.

- Where there are certain questions that must be asked first, separate them into their own Decision, and use a Flow with a gateway to ensure that they are answered first, before proceeding. See below for more information about using the MIO API with a Flow.

Decisions with child BIM Entities are not currently supported with the MIO API.

17.2 Using the MIO API with a Flow

The MIO API optimizes individual Decisions as described above. When used with a Flow, the MIO API is applied to each Decision independently. It does not optimize the inputs for the Flow as a whole, nor does it request values for gateway Fact Types.

The MIO API algorithm processes the Decision Tasks within the Flow sequentially until it encounters a gateway where it cannot determine the branch to be taken or it reaches the end of the Flow. The required Fact Types to request next are provided for each processed Decision separately. These should be consolidated by the calling application. Note that some Decisions might have conclusion values while others still request additional data, and some might not have been evaluated at all because the Flow execution stopped at a gateway.

Since the Flow is not optimized as a whole, the best practice is to ensure that the Flow requests Fact Types from only one Decision at a time. This is achieved by placing a gateway after each Decision, with the conclusion Fact Type of that Decision as the gateway Fact Type. The Flow execution will not proceed unless that Decision resulted in a conclusion value. The MIO API will list the required Fact Types for just that Decision, as any prior Decisions will already have conclusion values.

17.3 UI Approaches with the MIO API

A UI built with the MIO API needs to translate the Fact Type names and values expected by Decision into questions and answers. This is typically done through lookup tables (or similar means) to enable speed and ease of change and to easily support multiple languages.

The following approaches are commonly used to interact with the MIO API:

5. Display only the question(s) currently requested by the MIO API

   This is the simplest approach, taking the user down a specific path driven by the MIO API. If the user wants to change a prior response, they either need to back up (which may result in different questions being asked than what was asked previously) or start over.

6. Display the cumulative list of questions requested by MIO API

   The currently requested question(s) are appended to the list of questions already asked. This allows a user to change previous responses, which may result in different questions being asked than what was asked previously.

7. Display all questions, highlighting the questions currently requested by the MIO API

   This approach allows a sophisticated user to answer the questions in any order and to easily change previous responses. It is better suited for an internal user, such as an agent, rather than an end-user.

The same Decision or Flow can be used with any combination of the above approaches simultaneously, allowing you to mix and match as needed to support different UI needs. Which approach works best depends on your requirements.

17.4 Omitted Fact Types

Decision's standard execution algorithms handle omitted Fact Types, null values, and empty values as NotPopulated. The MIO API handles omitted Fact Types as questions to be asked. If you want to pass a null or empty value, you must do so explicitly by including the Fact Type in the input message.

17.5 Testing

Testing with the MIO API is not supported currently in DM. Testing with the MIO API must be done externally. As a best practice, deploy your assets as candidates (see section 7.6 *Candidate Deployment* above) for external testing with the MIO API prior to approval.

18 Parameter Management (PaM)

In this section we will discuss best practices specific to the Parameter Management (PaM) module. For a general discussion on working with lookup tables, see section 4.6 *Lookup Tables* above.

PaM provides a user interface for managing parameter tables externally from DM and it provides the ability to perform lookups against these tables at runtime via a lookup function, which as of this writing, is implemented as a Sapiens-provided *LOOKUP* UDF. PaM enables parameterization of the business logic within DM such that a parameter change (e.g., a threshold value) can be done via a data deployment rather than a more tightly governed code change.

There are some key differences between PaM parameter tables and Rule Families. Parameter tables are first-hit, meaning that, unlike Rule Families, row order does matter. Parameter tables can have multiple result columns, although the *LOOKUP* function can only return one at a time.

Since PaM makes database calls at runtime to fetch the current parameter values, it is critical, from a performance point of view, to minimize the *LOOKUP* calls. Use unconditional supporting Rule Families as shown in section 4.5 *Formulas* above to ensure that a lookup is executed only once for each Decision invocation. Similarly, for a Flow, instead of performing the same lookup in multiple Decisions, do it once and store the result in an interim Fact Type which you refer to later in the Flow as needed.

A *LOOKUP* call will return null if a match is not found. This null must be handled in one of the following ways:

1. Add a catch-all row at the bottom of the lookup table

   Since PaM tables are first-hit, adding a row at the bottom of the table where all the *SearchCriteria* columns are empty will effectively catch all lookups that failed to match in any of the preceding rows.

2. Catch the null in the supported Rule Family

   As in the Else Pattern discussed in section 4.13.2.1 *Else Pattern* above, the null can be caught in the supported Rule Family.

3. If the *LOOKUP* function is wrapped with a *VALUE* function to convert the returned text value to a numeric value, use the *VALUE*(*LOOKUP*(….), <default value>) form of the *VALUE* function to return the specified default value when the *LOOKUP* function returns null. This requires DM version 7.5 or later.

   If you use the *VALUE*(*LOOKUP*(….)) form without the default value, the *VALUE* function will result in a null pointer exception when the *LOOKUP* function returns null.

Option 1 is usually preferred as it requires the least effort to implement and maintain, one row per lookup table. Options 2 and 3 need to be implemented for each *LOOKUP* call and if the default value changes, could require logic changes in multiple places.

The performance of each lookup can be optimized by reducing lookup table size and by combining lookup tables. To reduce lookup table size, combine rows where possible by using empty cells to represent any value. Where multiple tables share the same search criteria (columns and values),

combine them into one table with multiple result columns to reduce the volume of the parameter table database.

When using tables with different combinations of empty cells, make sure to order the rows by descending priority of the search criteria (e.g., in a commission rate lookup, does the product take precedence over the state or vice versa?).

19 ModelAI

In this section we will discuss best practices specific to the ModelAI module. ModelAI utilizes generative AI to assist with modeling by inputting business policy in conversational English and automatically translating it into decision models.

You will achieve greater success with ModelAI if you follow these best practices:

- Use small chunks of logic to generate a single Rule Family at a time.
- Iterate by refining your text and regenerating, until you get a satisfactory result.
- Capitalization of the condition/conclusion names and/or using “the” often helps with the identification of the Fact Types.
- Prepend the business context (via the chat) to produce better Fact Type names.
- To reduce the number of unnecessary Fact Types generated in DM, use the *Sync Glossary* feature to map the generated Fact Types to existing ones prior to logic generation.

19.1 Example 1: Vehicle Theft Requirements

Requirements:

> *Vehicles with the following parameters are ineligible to be insured:*
>
> *Manufacturer: Hyundai or Kia*
>
> *Model years:2015-2019*
>
> *Feature list does not include: electronic engine immobilizer*
>
> *Recall Status: None*
>
> *Garaging State: MI, OH, WI*

Generated Rule Family:

Generally, ModelAI produces a Rule Family with the logic expressed in the requirements, but not the negative/completed logic. At this time this is preferred; once we copy the generated Rule Family to our decision model, we can easily create the diagonal.

Insurance Eligibility

MANUFACTURER	MODEL YEAR	FEATURE LIST	RECALL STATUS	GARAGING STATE	INSURANCE ELIGIBILITY
is in:Hyundai,Kia	is in:2015,2016,20...	does not contain:e...	is:None	is in:MI,OH,WI	Ineligible for Insur...

Our recommended Fact Type naming scheme suggests that we provide context to our Fact Type names. ModelAI can easily help with this task. In the above example we may wish to prepend "Vehicle" to each condition and make the conclusion more meaningful. To do so simply send another chat: *"Change Insurance Eligibility to Vehicle Theft Eligibility and prepend the word Vehicle to all conditions."*

19.2 Example 2: Pizza Requirements

Requirement:

> *Pizza Sauce Acceptability is determined by the following factors: Sauce Flavor, Sauce Consistency, Sauce Saltiness, Sauce Temperature. The sauce must be spicy, thick, not too salty, and served hot.*

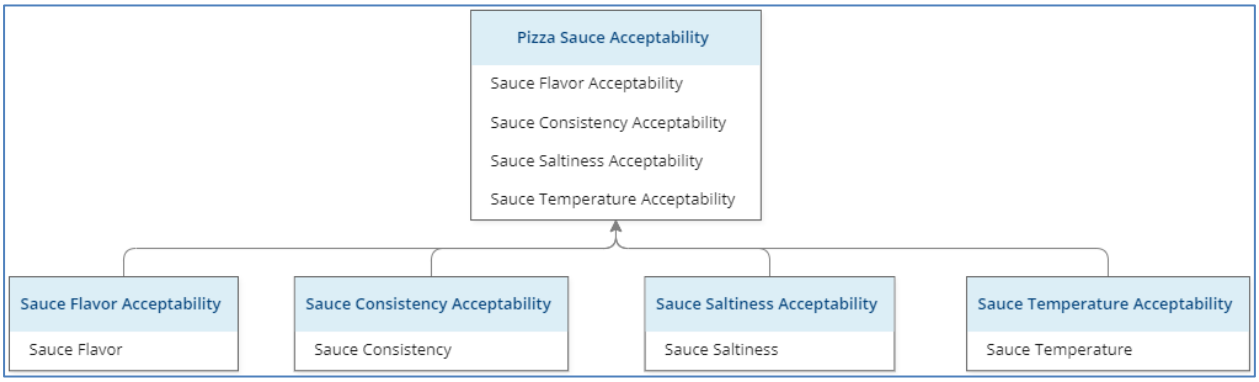
These simple requirements produce a simple Rule Family.

However, the negative logic is not correct. As simple as these requirements are, and while there are only four condition columns, this logic could be decomposed better.

Pizza Sauce Acceptability

SAUCE FLAVOR	SAUCE CONSISTENCY	SAUCE SALTINESS	SAUCE TEMPERATURE	PIZZA SAUCE ACCEPTABILITY
is:Spicy	is:Thick	Is Not:Too salty	is:Hot	Acceptable
Is Not:Spicy	Is Not:Thick	is:Too salty	Is Not:Hot	Not Acceptable

It can be difficult to teach new modelers to decompose the logic. ModelAI can frequently do it well. Send another chat, *"Decompose this logic into multiple supporting tables."*



This is a much better representation of the requirements and because each Rule Family has been simplified, the logic is accurate (but not quite for the Decision Rule Family).

Accurate:

Sauce Flavor Acceptability

SAUCE FLAVOR	SAUCE FLAVOR ACCEPTABILITY
is:Spicy	Acceptable
Is Not:Spicy	Not Acceptable

Inaccurate, but easily adjusted:

Pizza Sauce Acceptability

SAUCE FLAVOR ACCEPTABILITY	SAUCE CONSISTENCY ACCEPTABILITY	SAUCE SALTINESS ACCEPTABILITY	SAUCE TEMPERATURE ACCEPTABILITY	PIZZA SAUCE ACCEPTABILITY
is:Acceptable	is:Acceptable	is:Acceptable	is:Acceptable	Acceptable
is:Not Acceptable	is:Not Acceptable	is:Not Acceptable	is:Not Acceptable	Not Acceptable

Contact Us

For more information, please visit or contact us at:

www.sapiens.com

info.sapiens@sapiens.com

SAPIENS